

GestureFlow

Complete Project, Viva & Interview Preparation Guide

Real-time hand gesture recognition — React + FastAPI + MediaPipe + scikit-learn

Author / GitHub: **prakrash2003-web**

Repository: github.com/prakrash2003-web/gesture-recognition-app

Live app: gesture-recognition-app-tawny.vercel.app

Backend API: gesture-recognition-app-8ksr.onrender.com

Prepared for viva voce, internship, and placement interview preparation.

Generated from the actual project repository — every technology, file, metric, and URL in this guide was verified against the live code before writing.

Table of Contents

Table of Contents	2
1. Full Project Audit & Data Flow	5
1.1 Repository layout (verified)	5
1.2 The verified end-to-end data flow	6
2. Project Overview	8
2.1 Which part is which	8
2.2 Why this is a real-time system	8
2.3 Four spoken explanations	8
A) 30-second version	8
B) 1-minute version	9
C) 3-minute detailed version	9
D) 5-minute complete viva version	10
3. Complete Technology Stack	11
3.1 Quick reference table	11
3.2 Technology-by-technology detail	13
Backend and computer vision	13
Python 3.12	13
FastAPI	13
Uvicorn	13
WebSocket	13
OpenCV (opencv-contrib-python)	13
MediaPipe Hands	14
NumPy	14
scikit-learn	14
joblib	14
Pydantic	14
pytest and Ruff	15
Frontend	15
React 19	15
TypeScript	15
Vite	15
Tailwind CSS v4	15
React Router	15
Recharts	16
Vitest and React Testing Library	16
DevOps and deployment	17
Docker	17
Git and GitHub	17
GitHub Actions (CI)	17
Vercel	17
Render	17
Environment variables and JSON	17
4. Basic Concepts You Must Know	19
4.1 Python fundamentals	19
4.2 React fundamentals	20
4.3 Vite fundamentals	21
4.4 FastAPI fundamentals	21
4.5 MediaPipe and hand landmark detection	22
4.6 Machine learning fundamentals	22
4.7 Logistic Regression	24

4.8 Support Vector Machine (RBF kernel)	24
4.9 Random Forest	24
4.10 The rule-based classifier	24
4.11 WebSocket, in depth	25
4.12 Docker, in depth	25
4.13 Git and GitHub, in depth	26
4.14 Vercel and Render, in depth	26
5. File-by-File Project Explanation	28
5.1 Backend application (backend/app/)	28
5.2 Computer-vision pipeline (backend/app/vision/)	28
5.3 ML pipeline (backend/ml/)	30
5.4 Developer scripts	30
5.5 Frontend (frontend/src/)	31
5.6 Configuration and infrastructure files	33
5.7 Documentation files	33
6. The Machine Learning Pipeline in Detail	35
6.1 Dataset	35
6.2 Feature extraction	35
6.3 Training and model comparison	35
6.4 The five candidates compared	37
6.5 Model selection reasoning, including cross-validation	37
6.6 Debugging story: the fist vs. thumbs-up confusion	37
6.7 Serialization, runtime loading, and fallback	39
7. Real Project Results	40
7.1 Headline results	40
7.2 Logistic Regression confusion matrix (held-out session s2)	40
7.3 What these results mean, honestly	41
8. The Real-Time System, Explained	42
8.1 The pipeline stage by stage	42
8.2 Latency, throughput, and dropped frames	42
8.3 What the session-statistics panel actually measures	42
9. Frontend Explanation	44
9.1 What happens when the user opens the Live page	44
9.2 Browser permissions	44
9.3 Why production webcam access requires HTTPS	44
9.4 Page-by-page summary	44
9.5 Settings and the engine toggle	45
10. Backend Explanation	46
10.1 Application startup	46
10.2 Configuration	46
10.3 Routes	46
10.4 The /ws endpoint in detail	46
10.5 Error handling and the fallback classifier	47
10.6 Why the backend is deployed separately from the frontend	47
11. Deployment, Start to Finish	48
11.1 The chain, end to end	48
11.2 Why HTTPS and WSS specifically	48
11.3 Environment variables, precisely	48
11.4 Render free-tier behavior	49
12. GitHub and Version-Control Journey	50
12.1 Selected commits actually in the repository history	50
12.2 What happens on git push	50
13. Testing	51

13.1 What the backend's 93 tests cover	51
13.2 What the frontend's 93 tests cover	51
13.3 Other verification in the project	51
13.4 What these tests prove, and what they do not	51
14. Limitations (Honest Assessment)	53
15. Future Improvements	53
16. How I Should Explain This Project to an Examiner	54
17. Viva Questions — Basic Level (50)	56
18. Viva Questions — Project-Specific (75)	60
19. Senior Recruiter / Top-Company Level Questions (50)	76
20. Small / Mid-Size Company Interview (50)	90
21. Trick Questions / Cross-Questions (30)	94
22. Debugging / Real-World Troubleshooting Scenarios (30)	98
23. Rapid-Fire Round (100 Questions)	103
General (1-15)	103
ML / CV (16-30)	103
Numbers & Metrics (31-45)	104
Backend / API (46-60)	105
Frontend (61-75)	106
Deployment / DevOps (76-90)	107
Testing & Misc (91-100)	108
24. Important Numbers to Memorize	109
25. Final One-Page Cheat Sheet	111
10 things I MUST remember before viva	111

1. Full Project Audit & Data Flow

This guide is built directly from the repository at github.com/prakrash2003-web/gesture-recognition-app (branch `main`). Every fact below was checked against the actual source files, test runs, the committed ML evaluation report, and the live deployed URLs on the day this guide was generated. Where something is a documented plan rather than working code, it is labelled **"Planned / documented, not implemented."** Where something could not be verified from the repository, it is labelled **"Not determined from the repository."**

1.1 Repository layout (verified)

Path	What it is
<code>frontend/</code>	React 19 + TypeScript + Vite single-page app
<code>frontend/src/pages/</code>	LivePage, DashboardPage, GuidePage, ModelPage, AboutPage, NotFoundPage
<code>frontend/src/components/</code>	CameraView, GesturePanel, SessionStats, ConnectionBanner, SettingsPanel, NavBar, charts/, etc.
<code>frontend/src/hooks/</code>	useWebcam, useGestureSocket, useSettings, useSessionRecorder, useGestures, useModelInfo, useBackendHealth, useTheme
<code>frontend/src/lib/</code>	frameCapture.ts, drawLandmarks.ts, api.ts, session.ts, settings.ts
<code>backend/app/</code>	FastAPI application: main.py, routes.py, ws.py, schemas.py, config.py, gestures.py
<code>backend/app/vision/</code>	decode, landmarks, normalize, geometry, fingers, features, classifier_rules, classifier_ml, smoothing, pipeline, types
<code>backend/ml/</code>	dataset.py, synthetic.py, train.py, evaluate.py, metrics.py, models/, reports/, data/ (git-ignored)
<code>backend/tests/</code>	pytest suite — 93 tests across 15 test files
<code>backend/scripts/</code>	record_fixtures.py (dataset capture), ws_smoke.py (manual WebSocket check)
<code>docs/</code>	decisions.md, DEPLOYMENT.md, progress.md, screenshots/
<code>.github/workflows/ci.yml</code>	GitHub Actions: backend job + frontend job, on every push/PR to main
<code>Dockerfile</code> (repo root)	Backend container image
<code>.dockerignore</code>	Keeps the Docker build context to runtime files only

<code>frontend/vercel.json</code>	Vite build settings + SPA rewrite for Vercel
<code>backend/.env.example, frontend/.env.example</code>	Documented environment variables (no real secrets)
LICENSE	MIT License

1.2 The verified end-to-end data flow

The exact pipeline, as implemented:

1. **User opens the deployed frontend** (Vercel) and lands on the **Live** page (React Router route `/`).
2. User presses **Start**. `useWebcam` calls the browser's `getUserMedia()` API, requesting camera permission.
3. Once granted, the video stream is attached to a `<video>` element in `CameraView.tsx`.
4. `useGestureSocket` opens a WebSocket to `VITE_WS_URL` (the backend's `/ws` endpoint).
5. The backend's `/ws` handler (`app/ws.py`) first checks the browser's `Origin` header against `GESTUREFLOW_ALLOWED_ORIGINS`; if disallowed it closes the socket with code 1008.
6. On accept, the backend builds a per-connection `GesturePipeline` and sends a "ready" JSON message (gesture list, recommended fps, current classifier, whether the ML model is available, current confidence threshold).
7. Every ~100 ms, `lib/frameCapture.ts` draws the current video frame onto a hidden `<canvas>`, downsizes it to about 320 px wide, and encodes it as a JPEG Blob.
8. The Blob is sent over the WebSocket as a binary message. Only one frame is ever "in flight" — if a new frame is captured before the previous one's result comes back, the older one is replaced (never queued), so the system never falls behind under load.
9. On the backend, `app/vision/decode.py` turns the JPEG bytes back into a NumPy image with OpenCV, then downscales again as a safety net.
10. `app/vision/landmarks.py` runs **MediaPipe Hands** on the image and returns **21 (x, y, z) landmark points** for the most prominent hand, or `None` if no hand is found.
11. `app/vision/normalize.py` translates the landmarks so the wrist is at the origin and scales them by the hand's own size, making the result independent of how close to the camera the hand is or where in the

frame it sits.

Depending on `GESTUREFLOW_CLASSIFIER` (or a live "config" message from the client), one of two classifiers scores the normalized hand:

- **Rule-based** (`classifier_rules.py`) — hand-written geometric rules.
- **ML** (`classifier_ml.py`) — a trained scikit-learn Logistic Regression model loaded from `ml/models/gesture_clf.joblib`, fed 25 engineered features from `app/vision/features.py`.

13. `app/vision/smoothing.py` applies a majority vote over the last 6 predictions (with a 2-frame delay before switching to a new gesture) so a single noisy frame can't flicker the displayed result.

14. The backend streams a "result" JSON message back: gesture id (or `null`), confidence, whether a hand was present, handedness, the 21 landmark points (for drawing), per-class scores, server inference time in ms, and how many frames were dropped.

15. The frontend's `drawLandmarks.ts` draws the skeleton onto an overlay `<canvas>` positioned over the video; `GesturePanel.tsx` shows the gesture name, emoji, and a confidence bar; `SessionStats.tsx` shows connection state, throughput, server latency, dropped frames, and a running detection count.

16. Every result is also handed to `useSessionRecorder`, which keeps an in-memory, session-only history used by the **Dashboard** page (gesture-frequency chart, confidence-over-time chart, a gesture-history timeline).

This entire loop (steps 7–15) repeats roughly 10 times per second while the camera is on. The whole system is stateless between browser sessions — nothing about a gesture session is saved on the server or in a database; it lives only in the browser tab's memory for as long as that tab is open.

2. Project Overview

GestureFlow is a real-time hand-gesture recognition web application. A user opens a website, allows their webcam, and holds up one of six hand gestures; the app detects the hand, tracks 21 skeletal points on it, classifies which gesture is being shown, and displays the result live — typically within a few hundred milliseconds — with a session dashboard tracking what was detected over time.

Problem it addresses: demonstrating a complete, real-time computer-vision and machine-learning system with a proper production architecture, not a notebook demo — separate frontend and backend, a live network protocol, a trained and evaluated ML model with an honest baseline comparison, automated tests, and a real public deployment.

Why it was created: as a portfolio and final-year project to show, in one working system, skills across frontend engineering (React), backend engineering (FastAPI, WebSockets), computer vision (MediaPipe, OpenCV), applied machine learning (feature engineering, model comparison, honest evaluation), and DevOps (Docker, CI, cloud deployment).

Who can use it: anyone with a webcam (or a phone used as a webcam) and a browser that supports `getUserMedia` and WebSockets, no installation required, since it is deployed publicly.

2.1 Which part is which

Layer	What it is in this project
Frontend	React + TypeScript app: camera capture, UI, charts, settings, deployed on Vercel
Backend	Python FastAPI app: WebSocket server, orchestrates the vision pipeline, deployed on Render (Docker)
Computer vision	OpenCV (frame decoding) plus MediaPipe Hands (21-landmark detection)
Machine learning	scikit-learn Logistic Regression (shipped), compared against SVM, Random Forest, and a dummy baseline
Deployment and infra	Docker, GitHub Actions CI, Vercel, Render, environment-variable-driven config

2.2 Why this is a real-time system

Frames are captured, sent, processed, and displayed continuously in a loop (about 10 times per second), over a single persistent **WebSocket** connection rather than one-off requests. The system is explicitly designed to prefer freshness over completeness: the frontend only ever has one frame in flight, and the backend always processes the newest frame available, dropping stale ones rather than queuing them, so the displayed gesture reflects what the hand is doing right now, not what it was doing half a second ago.

2.3 Four spoken explanations

A) 30-second version

"GestureFlow is a real-time hand-gesture recognition web app. My React frontend streams webcam frames to a Python FastAPI backend over a WebSocket. The backend uses MediaPipe to find 21 points on the hand, turns those into features, and a trained Logistic Regression model, which I trained myself on 2,258 real samples I recorded, classifies which of six gestures it is, at about 95.5 percent accuracy. The result streams back and the frontend draws the hand skeleton and the gesture live. The app is deployed publicly: React on Vercel, the Python backend on Render as a Docker container."

B) 1-minute version

"GestureFlow recognizes six hand gestures: open palm, fist, thumbs up, victory, pointing up, and OK sign, from a live webcam feed. The browser captures frames, downsizes them, and streams them over a WebSocket to a Python backend. The backend decodes each frame with OpenCV, runs Google's MediaPipe Hands model to get 21 hand-joint coordinates, normalizes them so position and hand size do not matter, and then classifies the gesture. I built two classifiers: a hand-written rule-based one using finger-angle geometry as a zero-dependency baseline, and a trained scikit-learn model. For the ML model I collected a real dataset myself, 2,258 labelled samples across two separate recording sessions, engineered 25 geometric features from the landmarks, and compared four candidate models using a session-held-out train and test split, so the evaluation reflects a genuinely unseen session, not just a random shuffle. Logistic Regression generalized best and is what is shipped, at 95.5 percent accuracy versus about 86 percent for the rule-based baseline. Everything is tested with pytest and Vitest, runs through CI on every push, and is deployed live: frontend on Vercel, backend as a Docker container on Render."

C) 3-minute detailed version

"GestureFlow started as a way to build a complete, realistic computer-vision system rather than a notebook demo. The architecture is a browser-based React frontend and a separate Python FastAPI backend, talking over a WebSocket, because gesture recognition on live video is a continuous, bidirectional, stateful problem; a WebSocket keeps one connection open instead of firing dozens of REST calls per second.

On the frontend, once the user grants camera access, a hook captures a frame roughly every 100 milliseconds, draws it to an off-screen canvas, downsizes it to about 320 pixels wide, and sends it as a compressed JPEG. I deliberately never let more than one frame be in flight: if the backend is still working on the previous frame when a new one is captured, the new one replaces it rather than queuing, so latency cannot build up under load.

On the backend, OpenCV decodes the JPEG, MediaPipe Hands finds 21 (x, y, z) landmark points on the hand, and I normalize those landmarks: translate so the wrist sits at the origin, scale by the hand's own size, so the same gesture produces the same numbers regardless of where the hand is in frame or how close it is to the camera.

For classification I built two engines. The rule-based one looks at finger-extension ratios and joint angles I derived by hand; for example, a thumbs-up needs the thumb pointing up and clearly protruding past the curled fingers, a fix I added after noticing a closed fist was sometimes misread as a thumbs-up, since a fist's thumb is nearly as straight as an extended one. The second engine is a trained scikit-learn model. I recorded my own dataset with a small capture script, 2,258 samples across all six gestures over two separate sessions, and turned each hand pose into 25 engineered features (finger angles, tip distances, extension ratios) rather than raw pixel data or raw coordinates, so the model generalizes with a realistic amount of data. I trained and compared four candidates: a most-frequent dummy baseline, Logistic Regression, an RBF SVM, and a

Random Forest, using a session-held-out split, training on one recording session and testing on the other, so the reported accuracy reflects genuine generalization rather than memorizing near-duplicate frames. Logistic Regression won at 95.5 percent accuracy and 95.4 percent macro F1, beating the rule-based baseline's roughly 86 percent, and interestingly it generalized across sessions better than the more complex models, which scored higher in cross-validation on the training session but lost more accuracy on the unseen one.

The result streams back as JSON: gesture, confidence, the 21 landmarks, per-class scores, server inference time, dropped-frame count, and the frontend draws the skeleton, shows the gesture with a confidence bar, and feeds a session-statistics dashboard with charts and a history timeline. There is also a Model page that shows the real evaluation numbers and confusion matrices live from the backend, and a Settings panel that lets the user switch between the rule-based and ML engines at runtime and tune sensitivity.

The whole thing is deployed for real: the frontend is a static Vite build on Vercel, and the backend is a Docker container on Render, with the trained model baked into the image. I wrote 93 backend tests and 93 frontend tests, all running through GitHub Actions on every push, plus a WebSocket origin check so only my deployed frontend can open the socket in production."

D) 5-minute complete viva version

Use the 3-minute version above as the backbone, then extend it with: (1) the specific reason each technology was chosen over an alternative, see Section 3 and `docs/decisions.md` (10 documented decisions, for example why WebSockets over polling, why server-side vision over in-browser MediaPipe.js, why engineered features over raw coordinates); (2) a walk-through of one concrete bug that was fixed, the fist and thumbs-up confusion covered in Section 6.5, because it shows real debugging using the actual dataset, not guesswork; (3) the deployment story end-to-end from Section 11, including the environment variables and the WebSocket origin check; (4) the honest limitations in Section 14 (single hand, six gestures, one contributor's data, free-tier cold start); and (5) two or three future improvements from Section 15. Speaking all five parts naturally, in your own words, is exactly what Section 16's speaking script is for; rehearse from there, not by memorizing this paragraph.

3. Complete Technology Stack

Every technology below is actually present in the repository (checked in `requirements.txt`, `package.json`, source imports, and config files). Nothing here was added just because it is commonly seen in similar projects.

3.1 Quick reference table

Technology	Category	Where it is used in this project
Python 3.12	Language	Entire backend
FastAPI	Web framework	<code>backend/app/main.py</code> , <code>routes.py</code> , <code>ws.py</code>
Uvicorn	ASGI server	Runs the FastAPI app, in the Dockerfile CMD
WebSockets	Real-time protocol	<code>/ws</code> endpoint, <code>useGestureSocket.ts</code>
OpenCV (opencv-contrib-python)	Computer vision	<code>app/vision/decode.py</code> (JPEG to image array)
MediaPipe Hands	Computer vision (pretrained model)	<code>app/vision/landmarks.py</code>
NumPy	Numerical computing	Landmark math throughout <code>app/vision/</code>
scikit-learn	Machine learning	<code>backend/ml/train.py</code> , <code>classifier_ml.py</code>
joblib	Model serialization	Saves/loads <code>gesture_clf.joblib</code>
Pydantic	Data validation	<code>app/schemas.py</code> (all API/WebSocket message shapes)
pytest	Testing	93 backend tests in <code>backend/tests/</code>
Ruff	Linting/formatting	Backend code style, enforced in CI
React 19	UI library	Entire frontend
TypeScript	Language	Entire frontend
Vite	Build tool / dev server	Frontend dev server and production build
Tailwind CSS v4	Styling	All frontend component styling

React Router	Client-side routing	Live, Dashboard, Guide, Model, About pages
Recharts	Charting	Dashboard's frequency and confidence charts
Vitest + React Testing Library	Testing	93 frontend tests
oxlint	Linting	Frontend code style, enforced in CI
Docker	Containerization	Backend deployment image (repo-root Dockerfile)
Git and GitHub	Version control	Whole repository, GitHub Actions CI
GitHub Actions	CI/CD	.github/workflows/ci.yml
Vercel	Frontend hosting	Deployed frontend
Render	Backend hosting	Deployed backend (Docker)
Environment variables	Configuration	app/config.py , src/config.ts
JSON	Data format	REST responses and every WebSocket message

Not actually used (verified absent):

Deep learning / neural networks, TensorFlow, PyTorch, a database of any kind, Redux, GraphQL, Next.js, Kubernetes, WebRTC, and Hugging Face Spaces (documented in [docs/DEPLOYMENT.md](#) as an alternative backend host, but the project is actually deployed on Render). These are explicitly called out as rejected alternatives in [docs/decisions.md](#), not oversights.

3.2 Technology-by-technology detail

Format for each: What it is · Why chosen here · Where it lives in the code · one concept to know · a likely viva question.

Backend and computer vision

Python 3.12

What: a general-purpose, interpreted, dynamically typed programming language. **Why here:** it has the best ecosystem for both computer vision (OpenCV, MediaPipe) and machine learning (scikit-learn) in one language, so the whole backend, from the web server to the trained model, is one codebase. **Where:** the whole `backend/` folder. **Know:** Python is interpreted (no separate compile step) and dynamically typed (a variable's type is checked at runtime, not before). **Viva Q:** "Why Python and not a faster compiled language for the backend?" Answer: the actual per-frame work is CPU-bound in MediaPipe and NumPy, which are C/C++ under the hood; Python is the orchestration layer, not the hot loop, and the ecosystem match for CV and ML was more valuable than raw language speed.

FastAPI

What: a Python web framework for building APIs, built on the ASGI standard. **Why here:** native, first-class WebSocket support, automatic request/response validation and documentation from Pydantic models, and asynchronous request handling. **Where:** `app/main.py` creates the app and adds CORS; `app/routes.py` defines `/health`, `/gestures`, `/model`; `app/ws.py` defines `/ws`. **Know:** FastAPI auto-generates interactive API docs at `/docs` from the route type hints and Pydantic models. **Viva Q:** "Why FastAPI over Flask?" Answer: Flask's WebSocket support is not built in and needs an extension; FastAPI's is native, plus FastAPI is asynchronous by default, which matters when many WebSocket connections are being served at once.

Uvicorn

What: an ASGI server, the program that actually listens on a network port and runs the FastAPI app. **Why here:** it is the standard, high-performance ASGI server and supports WebSockets out of the box via the `[standard]` extras. **Where:** the Dockerfile's final `CMD` line runs `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. **Know:** FastAPI defines *what* the app does; Uvicorn is *what actually runs it* and talks to the network. **Viva Q:** "What does `--host 0.0.0.0` mean and why is it needed in Docker?" Answer: it tells Uvicorn to accept connections from any network interface, not just `localhost`; inside a container, the request arrives from outside the container's own loopback address, so binding to `127.0.0.1` only would make the app unreachable.

WebSocket

What: a network protocol that upgrades a normal HTTP connection into a single, persistent, two-way connection. **Why here:** gesture recognition on live video is continuous and bidirectional (frames flow up, results flow down) roughly ten times a second; opening a new HTTP request for every frame would add repeated connection and header overhead and has no natural concept of a session. **Where:** backend `app/ws.py` (`/ws` route), frontend `hooks/useGestureSocket.ts`. **Know:** a WebSocket starts life as an HTTP request with an `Upgrade: websocket` header; once the server accepts, both sides can send messages at any time on that same connection. **Viva Q:** "Why not just poll a REST endpoint every 100ms instead?" Answer: polling means a brand-new HTTP request/response cycle (TCP handshake overhead avoided only if keep-alive is used, but still full HTTP headers) for every single frame, with no server-side session state; a WebSocket is one connection that both sides keep open and write to, which is both lower overhead and a more natural fit for a continuous stream.

OpenCV (opencv-contrib-python)

What: the standard open-source computer-vision library. **Why here:** it is used narrowly, for one job: decoding the JPEG bytes the browser sends into a NumPy pixel array, and downscaling as a safety net. **Where:** `app/vision/decode.py` (`decode_jpeg`, `downscale`). **Know:** OpenCV represents images as NumPy arrays in BGR (blue, green, red) channel order, not RGB, which is a common source of color bugs if forgotten. **Viva Q:** "Is OpenCV doing the gesture detection?" Answer: no, OpenCV only decodes and resizes the image; the actual hand detection is done by MediaPipe.

MediaPipe Hands

What: a pretrained computer-vision model from Google that finds a hand in an image and returns 21 (x, y, z) landmark points for its joints. **Why here:** building an equivalent hand-landmark detector from scratch would be a project of its own; MediaPipe is a solved, fast, CPU-friendly building block, and the interesting, hand-written part of this project is what happens *after* the landmarks are found. **Where:** `app/vision/landmarks.py`, using the classic `mediapipe.solutions.hands` API with `model_complexity=0` (the lightest, fastest model variant) and `static_image_mode=False` (it tracks the hand between frames instead of fully re-detecting every time). **Know:** MediaPipe's model is bundled inside the Python package, so there is no separate model file to download or manage for this stage. **Viva Q:** "Why 21 landmarks?" See Section 4 (MediaPipe basics) for the full joint map; in short, MediaPipe's hand model was trained to output one point for the wrist plus four points per finger (base, two middle knuckles, tip), for five fingers: $1 + 4 \times 5 = 21$.

NumPy

What: the standard Python library for fast array and vector math. **Why here:** every landmark is a 3D point, and all the geometry (distances, angles, normalization) is vector arithmetic, which NumPy does far faster than plain Python loops. **Where:** throughout `app/vision/`, especially `geometry.py`, `normalize.py`, and `features.py`. **Know:** a NumPy array of the 21 landmarks has shape `(21, 3)`, 21 rows (one per joint) and 3 columns (x, y, z). **Viva Q:** "Why not just use Python lists?" Answer: NumPy operations run as compiled, vectorized C code under the hood, which matters when this math runs on every frame, about ten times a second, for every connected user.

scikit-learn

What: the standard Python library for classical (non deep-learning) machine learning: models, evaluation metrics, train/test splitting utilities. **Why here:** the problem, 25 numeric features to one of six classes, is a small, structured classification task, exactly what scikit-learn is built for, and it ships every model and metric this project needed (`LogisticRegression`, `SVC`, `RandomForestClassifier`, `DummyClassifier`, `GroupShuffleSplit`, `precision_recall_fscore_support`, `confusion_matrix`). **Where:** `backend/ml/train.py`, `backend/ml/metrics.py`. **Know:** scikit-learn's convention is `model.fit(X_train, y_train)` to train, then `model.predict(X_test)` to get predictions. **Viva Q:** "Why not deep learning?" See Section 21 (trick questions) for the full answer; in short, with 2,258 samples and 25 well-engineered features, a neural network has no real advantage and a much higher risk of overfitting.

joblib

What: a Python library for efficiently serializing (saving/loading) objects, commonly used for scikit-learn models. **Why here:** to save the trained pipeline (scaler plus classifier) to a single file that the backend can load at startup. **Where:** `ml/train.py`'s `_save_model` writes `ml/models/gesture_clf.joblib`; `app/vision/classifier_ml.py` loads it with `joblib.load`. **Know:** the saved file is not just the model; it is a dictionary containing the model, the feature names, a `feature_version` number, the training dataset type, and the evaluation score, so the loader can sanity-check compatibility before trusting it. **Viva Q:** "What happens if the model file is corrupted or from an incompatible version?" Answer: `MLClassifier.__init__` catches load failures and raises `ModelUnavailable`, and separately checks the stored `feature_version` against the code's current one; either problem makes the backend fall back to the rule-based classifier instead of crashing or mispredicting silently.

Pydantic

What: a Python data-validation library that turns a class with typed attributes into a schema that validates, serializes, and documents data. **Why here:** every HTTP response and every WebSocket message has an exact, defined shape, and Pydantic both enforces that shape and generates the `/docs` page automatically from the same class definitions. **Where:** `app/schemas.py` (`HealthResponse`, `Gesture`, `GesturesResponse`, `ModelInfoResponse`, `ReadyMessage`, `ConfigMessage`, `FrameResultMessage`, `ErrorMessage`). **Know:** FastAPI uses Pydantic under the hood for both request validation and response serialization. **Viva Q:** "What happens if the frontend sends a malformed config message over the WebSocket?" Answer: `ConfigMessage.model_validate(payload)` raises a `ValidationError`, which `app/ws.py` catches and simply ignores the message, so a bad message cannot crash the connection.

pytest and Ruff

What: pytest is Python's most widely used test runner; Ruff is a fast linter and code formatter. **Why here:** to catch regressions automatically (pytest) and keep the codebase consistently styled and free of common mistakes (Ruff). **Where:** `backend/tests/` (93 tests across 15 files), `backend/pyproject.toml` configures both. **Know:** pytest auto-discovers any function named `test_*` in any file named `test_*.py`; no manual test registration needed. **Viva Q:** "What do the 93 tests actually cover?" See Section 13 (Testing) for the full breakdown by area.

Frontend

React 19

What: a JavaScript/TypeScript library for building user interfaces out of reusable pieces called components. **Why here:** the UI has real interactive state (camera on/off, live gesture results, settings, charts) that needs to re-render efficiently as data streams in; React's component-plus-state model fits that directly, and it is the most widely used frontend library, which matters for a portfolio project. **Where:** the entire `frontend/src/` tree. **Know:** a component is a function that returns markup (JSX); when its state changes, React re-renders it automatically. **Viva Q:** "What is the difference between props and state?" Answer: props are values passed into a component from its parent (read-only from that component's own point of view); state is data a component owns and can change itself, and changing it triggers a re-render.

TypeScript

What: JavaScript with an added static type system, checked before the code runs. **Why here:** the WebSocket message shapes need to match exactly between what the backend sends and what the frontend expects; TypeScript catches a mismatched field name or wrong type at compile time instead of as a silent runtime bug. **Where:** every file in `frontend/src/`; `types.ts` mirrors the backend's Pydantic models field-for-field. **Know:** TypeScript is not a separate language that runs in the browser; it is compiled (transpiled) down to plain JavaScript by Vite before it ships. **Viva Q:** "How do you keep the frontend types and backend schemas in sync?" Answer: currently by hand, since the project is small enough that `types.ts` is kept deliberately mirroring `schemas.py`; a larger project would generate one from the other automatically from the OpenAPI schema FastAPI already produces.

Vite

What: a modern frontend build tool: an instant-reload development server plus a production bundler. **Why here:** it starts and updates far faster than older tools during development, and produces an optimized, code-split production build. **Where:** `frontend/vite.config.ts`; `npm run dev` and `npm run build`. **Know:** Vite only exposes environment variables to the browser if their name starts with `VITE_`, which is a deliberate safety boundary so a server secret cannot accidentally leak into client code. **Viva Q:** "What does `npm run build` actually produce?" Answer: a `dist/` folder of static HTML, CSS, and JavaScript files, including a separate lazily loaded chunk for the Dashboard page (because it pulls in the Recharts charting library) so the initial page load stays small.

Tailwind CSS v4

What: a utility-class CSS framework; instead of writing custom CSS files, elements are styled by composing small pre-defined classes directly in the markup. **Why here:** fast, and consistent spacing, color, and typography across many components without maintaining separate stylesheets. **Where:** every component's `className` attributes; theme tokens defined in `src/index.css`. **Know:** Tailwind supports both light and dark themes through a `dark:` variant prefix on classes, toggled here by a `.dark` class on the `html` element. **Viva Q:** "How does the light/dark theme toggle work?" Answer: `useTheme.ts` reads a saved preference (or the OS setting) and toggles a `.dark` class on the document root; a custom Tailwind variant makes `dark:`-prefixed utility classes respond to that class rather than only the OS setting.

React Router

What: a client-side routing library; it maps URL paths to React components without a full page reload. **Why here:** the app has five real pages (Live, Dashboard, Guide, Model, About) that should each have their own URL and survive a refresh. **Where:** `App.tsx` defines the route table; `frontend/vercel.json`'s rewrite rule sends every path

back to `index.html` so a hard refresh on, for example, `/model`, does not 404 on Vercel. **Viva Q:** "Why does a hard refresh on `/model` need special hosting configuration?" Answer: because there is no real server-side route for `/model`; the static host must be told to always serve `index.html` and let React Router handle the path client-side once the JavaScript loads, otherwise the host looks for a literal file at that path and returns a 404.

Recharts

What: a React charting library built on SVG. **Why here:** to draw the Dashboard's gesture-frequency bar chart and confidence-over-time area chart from the live session data. **Where:** `components/charts/GestureFrequencyChart.tsx`, `ConfidenceTimelineChart.tsx`. **Know:** the Dashboard page and Recharts are loaded lazily (`React.lazy`) so that library's weight does not slow down the initial load of the Live page. **Viva Q:** "Why lazy-load the Dashboard specifically?" Answer: Recharts is one of the larger dependencies in the project; most users open the app to use the Live page first, so deferring the chart library until `/dashboard` is actually visited keeps the first paint fast.

Vitest and React Testing Library

What: Vitest is a Vite-native test runner; React Testing Library renders components in a simulated browser (jsdom) and lets tests interact with them the way a user would. **Why here:** to test hooks, components, and pure logic (frame capture math, session-data derivations) without a real browser. **Where:** 93 tests across 22 files, colocated with the code they test, for example `useGestureSocket.test.ts` next to `useGestureSocket.ts`. **Viva Q:** "How do you test a WebSocket hook without a real backend?" Answer: a mock `WebSocket` class is substituted for the browser's real one in the test, and the test drives it directly, simulating an open event, a ready message, a result message, to check the hook reacts correctly.

DevOps and deployment

Docker

What: a tool for packaging an application together with everything it needs to run (system libraries, Python packages, code) into a single portable image, then running that image as an isolated container. **Why here:** MediaPipe and OpenCV need specific system-level libraries (`libgl1`, `libgl1-mesa-glx_2.0-0`) that are not present by default on most hosting platforms; Docker guarantees the exact same environment locally and in production. **Where:** the repo-root `Dockerfile`, built and run by Render. **Know:** an "image" is the packaged, unchanging blueprint; a "container" is a running instance of that image. **Viva Q:** "Why does the Dockerfile install `libgl1` and `libgl1-mesa-glx_2.0-0` specifically?" Answer: OpenCV's compiled binary depends on the system graphics library (`libGL`) even when only decoding images, not displaying them, and MediaPipe's dependencies pull in `glib`; without those two packages the container would fail on import, not just at runtime.

Git and GitHub

What: Git is a distributed version-control system that tracks every change to the code over time; GitHub is a hosting service for Git repositories, plus collaboration and automation features. **Why here:** to keep a full, inspectable history of the project's development in logical commits, and because both deployment platforms (Vercel and Render) redeploy automatically from GitHub pushes. **Where:** the whole repository; 16 commits on `main` at the time of writing. **Know:** a commit is a saved snapshot with a message describing what changed and why; a commit hash (for example `a257d05`) is a unique identifier for that exact snapshot. **Viva Q:** "What happens the moment you run `git push`?" See Section 12 (GitHub and version-control journey) for the full push-to-redeploy chain.

GitHub Actions (CI)

What: GitHub's built-in automation service; it runs a defined set of steps on GitHub's own servers whenever something happens in the repository, such as a push. **Why here:** to automatically lint, type-check, test, and build both the backend and frontend on every push and pull request, so a mistake is caught before it reaches `main`. **Where:** `.github/workflows/ci.yml`, two jobs: `backend` and `frontend`. **Know:** the backend job runs `ruff check`, `ruff format --check`, and `pytest`; the frontend job runs `oxlint`, a TypeScript type-check, `vitest run`, and a production build. **Viva Q:** "Does CI deploy the app?" Answer: no, CI only verifies the code; the actual deployment is a separate step done by Render and Vercel, which watch the GitHub repository independently and rebuild when `main` changes.

Vercel

What: a hosting platform specialized in static sites and frontend frameworks. **Why here:** free tier, automatic HTTPS, automatic deployment on every push to `main`, and native support for Vite's build output. **Where:** hosts the deployed frontend at `gesture-recognition-app-tawny.vercel.app`, configured by `frontend/vercel.json`. **Know:** Vercel serves a static build; it has no server-side Python or Node process running, which is why all backend logic must live on Render instead. **Viva Q:** "Could the whole app run on Vercel alone?" Answer: no; Vercel does not run a persistent WebSocket server or the Python/MediaPipe stack, so the backend has to be hosted separately on a platform that runs a long-lived container.

Render

What: a cloud hosting platform that can run arbitrary Docker containers as web services, with a free tier. **Why here:** the backend needs a real, always-listening process (Uvicorn, holding open WebSocket connections), which is what Render's web-service model provides, built from the project's own `Dockerfile`. **Where:** hosts the deployed backend at `gesture-recognition-app-8ksr.onrender.com`. **Know:** Render's free tier puts a service to sleep after roughly 15 minutes without traffic; the first request afterward has to wait for the container to start again, around 30 seconds. **Viva Q:** "How does Render know how to build the app?" Answer: it detects the `Dockerfile` at the repository root and builds it as a Docker image, then runs the image's `CMD`, exactly as `docker build` and `docker run` would locally.

Environment variables and JSON

What: environment variables are named values read by a program from its surrounding process environment rather than hard-coded in source; JSON (JavaScript Object Notation) is a lightweight, text-based data format for structured data. **Why here:** environment variables let the same code run locally and in production with different settings (allowed origins, which classifier to use, API URLs) without editing source code or committing secrets; JSON is the format of every REST response and every WebSocket message, because both the browser and Python can parse and produce it natively. **Where:** `backend/app/config.py` reads `GESTUREFLOW_*` variables; `frontend/src/config.ts` reads `VITE_*` variables; every response body in `app/schemas.py` is serialized to JSON. **Viva Q:** "Are any secrets stored in environment variables here?" Answer: no; every variable in this project (allowed origins, which classifier to run, API URLs) is a configuration value, not a credential, which is explicitly noted in `backend/.env.example`.

4. Basic Concepts You Must Know

Section 3 covered why each technology was chosen for this project. This section covers the underlying fundamentals, in case a basic question is asked in isolation.

4.1 Python fundamentals

Concept	Plain explanation
Variable	A name bound to a value, for example <code>confidence = 0.95</code> . Python figures out the type automatically.
Function	A reusable named block of code, defined with <code>def</code> , for example <code>def classify(points): ...</code> .
Class	A blueprint for creating objects that bundle data and behavior together, for example <code>class MLClassifier:</code> in this project.
List	An ordered, changeable collection, for example the 21 landmark points as a list of coordinates.
Dictionary	A collection of key-to-value pairs, for example the per-gesture confidence scores <code>{"fist": 0.02, "thumbs_up": 0.97, ...}</code> .
Tuple	An ordered, unchangeable collection, used in this project for fixed things like landmark index constants.
Module	A single <code>.py</code> file, importable by name, for example <code>app/vision/features.py</code> .
Package	A folder of modules treated as one importable unit (needs an <code>__init__.py</code>), for example the <code>app</code> and <code>ml</code> packages.
Virtual environment	An isolated, project-only copy of Python so this project's dependencies do not collide with any other project's; this repo's is at <code>.venv/</code> .
pip	Python's package installer, downloads libraries from PyPI, for example <code>pip install -r requirements.txt</code> .
requirements.txt	A text file listing exact package versions to install, so every machine gets an identical set.
Exception	An error object raised when something goes wrong, catchable with <code>try/except</code> ; this project uses a custom <code>ModelUnavailable</code> exception.

API	A defined set of operations a program exposes for other programs to call; here, the REST endpoints and the WebSocket protocol.
JSON	See Section 3.2; Python's built-in <code>json</code> module converts between JSON text and Python dictionaries/lists.
<code>async/await</code>	Syntax for asynchronous code, work that can pause while waiting (for a network message, for example) without blocking the whole program; FastAPI's WebSocket handler is written with <code>async def</code> .
Basic OOP	Object-oriented programming: bundling data and the functions that act on it into classes and objects, for example <code>GesturePipeline</code> , which owns a <code>MediaPipe</code> detector and a smoothing buffer as its state.
Why Python for ML	Its ML/CV ecosystem (scikit-learn, NumPy, MediaPipe) is mature and standard, with a clean, readable syntax for expressing the logic.

4.2 React fundamentals

Concept	Plain explanation
Component	A function that returns UI markup, for example <code>GesturePanel</code> . The app is built by composing components.
Props	Inputs passed into a component from its parent, for example <code><GestureCard gesture={g} /></code> passes <code>gesture</code> as a prop.
State	Data a component owns and can change; changing it causes a re-render, for example the current WebSocket connection status.
Hooks	Functions starting with <code>use</code> that let a component use React features like state and side effects; this project has many custom ones (<code>useWebcam</code> , <code>useGestureSocket</code>).
<code>useState</code>	The built-in hook for a single piece of component state, for example the theme (light/dark).
<code>useEffect</code>	The built-in hook for side effects, code that runs after render and can clean up after itself, for example opening and later closing a WebSocket.
Events	User interactions like clicks, handled with props such as <code>onClick</code> .

Rendering	The process of React turning components and their current state into actual DOM elements the browser displays.
Frontend vs backend	Frontend is code that runs in the user's browser (this React app); backend is code that runs on a server the browser talks to (the FastAPI app).

4.3 Vite fundamentals

Vite is the build tool and development server. During development, `npm run dev` starts a local server with near-instant hot reloading. For production, `npm run build` compiles and bundles everything into a small set of optimized static files in `dist/`, which is what gets uploaded to Vercel. Environment variables prefixed `VITE_` (for example `VITE_API_BASE_URL`) are baked into that build at build time, which is why changing them on Vercel requires a new deployment to take effect.

4.4 FastAPI fundamentals

Concept	Plain explanation
Route / endpoint	One URL plus one HTTP method mapped to one Python function, for example <code>GET /health</code> .
GET	The HTTP method meaning "fetch data, do not change anything"; used by <code>/health</code> , <code>/gestures</code> , <code>/model</code> .
POST	The HTTP method meaning "create or submit data"; not used anywhere in this project's REST API, only GET and the WebSocket.
Request / response	What the client sends and what the server sends back; both are typed with Pydantic models here.
Asynchronous processing	FastAPI can handle many connections concurrently without blocking on slow I/O (like waiting for a WebSocket message).
WebSocket support	Native in FastAPI via <code>@router.websocket("/ws")</code> , no external library needed.
<code>/health</code>	Liveness check, returns <code>{"status": "ok", ...}</code> ; used by the frontend's status badge and by Render's health check.
<code>/model</code>	Returns which classifier is active, whether the ML model is available, and the full evaluation report.
<code>/ws</code>	The WebSocket endpoint that streams frames in and gesture results out.

4.5 MediaPipe and hand landmark detection

MediaPipe Hands takes one image and returns, for each hand it finds, 21 (x, y, z) points. This is **landmark detection**: finding *where* the joints of the hand are. It is a completely different task from **gesture classification**: deciding *what* those 21 points, taken together, mean. MediaPipe only does the first job. Every gesture-specific decision in this project, both the rule-based engine and the trained ML model, is code written for this project, working purely on the 21 numbers MediaPipe hands back.

The 21 landmark map

Point 0 is the wrist. Each of the five fingers then gets four points, numbered from the base of the finger to the tip: 1–4 thumb, 5–8 index, 9–12 middle, 13–16 ring, 17–20 pinky. So 1 (wrist) + 4 x 5 (fingers) = 21. For a non-thumb finger the four points are the knuckle where it joins the palm (MCP), two middle joints (PIP and DIP), and the tip; the thumb has one fewer true knuckle joint so its four points are named CMC, MCP, IP, and tip.

x, y, z meaning: x and y are normalized image coordinates (0 to 1 across the frame width and height, x increasing rightward, y increasing *downward*); z is an approximate depth relative to the wrist, roughly on the same scale as x, negative meaning closer to the camera.

Viva Q: "Why MediaPipe instead of building a hand detector from scratch?" Answer: hand-landmark detection is itself a hard, already-solved computer-vision problem; reproducing it would be a separate multi-month project, and the actual point of this project is the geometry, feature engineering, and classification logic built on top of those landmarks, not re-inventing landmark detection.

4.6 Machine learning fundamentals

Term	Plain explanation, grounded in this project
Supervised learning	Learning a mapping from inputs to known correct outputs, from labelled examples. Here: 25 features in, one of six gesture labels out, learned from 2,258 labelled samples.
Classification	Supervised learning where the output is one of a fixed set of categories (as opposed to a continuous number). Here: which of six gestures.
Feature	One measured input number fed to the model, for example <code>thumb_curl_angle</code> . This project uses 25 features per hand.
Label	The correct answer for one training example, for example "fist".
Train/test split	Dividing the dataset so the model is fit on one part and scored on a part it has never seen, to measure real generalization rather than memorization.
Held-out evaluation	Here, specifically a <i>session</i> -held-out split: the model trains on recording session 1 (1,115 samples) and is tested on session 2 (1,143 samples), a genuinely different recording, not just a random shuffle of the same session.

Accuracy	The fraction of predictions that were exactly correct.
Precision	Of everything the model called a given gesture, the fraction that actually was that gesture.
Recall	Of everything that actually was a given gesture, the fraction the model correctly caught.
F1 score	The harmonic mean of precision and recall, one number balancing both. "Macro F1" averages the F1 of each of the six classes equally, so a rare gesture matters as much as a common one.
Confusion matrix	A grid of true label versus predicted label; the diagonal is correct predictions, everything off the diagonal shows exactly which gestures get mixed up with which.
Overfitting	A model that fits the training data very well but fails to generalize to new data, often from memorizing noise or, here, near-duplicate consecutive frames.
Generalization	How well a model performs on genuinely new data it was not trained on; the entire point of the session-held-out split is to measure this honestly.

4.7 Logistic Regression

What it is: despite the name, it is a *classification* algorithm, not a regression one in the everyday sense. It computes a weighted sum of the input features, then squashes that sum through a sigmoid (or, for more than two classes, a softmax) function to produce a probability for each class. **Basic idea:** the model learns one weight per feature per class; a feature with a large positive weight for "thumbs_up" pushes the prediction toward thumbs_up when that feature is large. **Multiclass:** scikit-learn's default handles the six gesture classes directly (a multinomial approach), producing a probability for every class that sums to 1. **Why selected here:** in this project's own comparison, Logistic Regression scored highest on the held-out session (95.5 percent accuracy) and, notably, generalized *better* across sessions than the more flexible models, which fit the training session more tightly (higher cross-validation score) but lost more accuracy on the unseen session, a sign of mild overfitting to session-specific quirks. A simpler, linear-ish decision boundary transferred better here. It is also fast, small (the saved model file is about 4 KB), and easy to explain.

4.8 Support Vector Machine (RBF kernel)

What it is: a classifier that tries to find the boundary between classes that maximizes the margin (distance) to the nearest points of each class. The RBF (radial basis function) kernel lets that boundary be a flexible, curved shape in feature space rather than a straight line. **Why it was tested:** as a stronger, non-linear alternative to Logistic Regression, to check whether the true decision boundary between gestures needed that extra flexibility. **Result here:** 90.8 percent accuracy, solidly behind Logistic Regression; the extra flexibility did not translate into better generalization on the unseen session.

4.9 Random Forest

What it is: an ensemble of many decision trees, each trained on a random subset of the data and features; the forest's prediction is the majority vote across all the trees. A decision tree itself is a series of yes/no questions on feature values (for example, "is thumb_curl_angle greater than 2.8?") that end in a predicted class. **Why it was tested:** as a strong, commonly used baseline that can capture non-linear feature interactions without much tuning. **Result here:** 84.3 percent accuracy and 83.1 percent macro F1, the weakest of the three real ML candidates on the held-out session, despite scoring well in cross-validation within the training session, again pointing to overfitting to session-specific patterns.

4.10 The rule-based classifier

What it means: a classifier built entirely from hand-written geometric rules and thresholds, with no training data and no learning involved, as opposed to a model that learns its parameters from examples. **How it differs from ML:** every decision is an explicit line of code a human wrote (for example, "a finger is extended if its tip is farther from the wrist than its middle knuckle"), rather than weights learned automatically from data. **Why it exists as a baseline:** it works immediately with zero training data, every decision is explainable in one sentence, and it is the zero-dependency fallback the backend automatically uses if the trained model file is ever missing or incompatible, so the app never simply stops working. **Strengths:** no data needed, fully interpretable, instant to reason about, fast. **Weaknesses:** hand-tuned thresholds, brittle at unusual hand angles, and, as this project's own real evaluation shows, meaningfully less accurate overall (about 86 percent versus the trained model's 95.5 percent).

4.11 WebSocket, in depth

Concept	Plain explanation
HTTP vs WebSocket	HTTP is request-then-response, then the connection is typically done; a WebSocket is one connection that both sides keep open and can write to at any time, in either direction.
Connection lifecycle	Browser opens the socket to <code>/ws</code> ; the server checks the Origin header, accepts, and sends a "ready" message; then binary frame messages flow up and JSON result messages flow down until either side closes it.
<code>ws://</code> vs <code>wss://</code>	<code>ws://</code> is a plain, unencrypted WebSocket, used only for local development; <code>wss://</code> is the encrypted version, required in production because the page itself is served over HTTPS and browsers do not allow an HTTPS page to open a plain <code>ws://</code> connection (a "mixed content" restriction).
Why appropriate here	The video-to-result exchange is continuous, high-frequency, and naturally stateful (the smoothing buffer and the MediaPipe graph persist for the life of one connection); a WebSocket models that directly instead of faking it with repeated HTTP calls.

4.12 Docker, in depth

Concept	Plain explanation
Image	The packaged, unchanging blueprint built from the Dockerfile's instructions.
Container	A running instance of an image; you can start many containers from one image.
Dockerfile	The recipe: base image, system packages, Python packages, application files, and the startup command.
Why useful here	Guarantees the exact same OS-level dependencies (for OpenCV/MediaPipe) locally, in CI if used, and on the hosting platform; Render builds and runs this project's own Dockerfile directly.
Health check	Implemented: the Dockerfile's <code>HEALTHCHECK</code> instruction periodically calls <code>/health</code> inside the container so Docker itself can tell whether the app is actually responding, not just whether the process is alive.

Non-root user

Implemented: the Dockerfile creates an unprivileged `appuser` and switches to it before running the app, a standard security hardening step so a compromised process is not running as root inside the container.

4.13 Git and GitHub, in depth

Term	Plain explanation
Repository	The full tracked project and its history; here, <code>prakrash2003-web/gesture-recognition-app</code> on GitHub.
Commit	A saved snapshot of changes with a message; this project has 16 commits on <code>main</code> .
Branch	A named, independent line of commits; this project has been developed directly on <code>main</code> .
<code>main</code>	The default/primary branch, the one both Vercel and Render deploy from.
push / pull	<code>git push</code> uploads local commits to GitHub; <code>git pull</code> downloads and merges commits from GitHub into the local copy.
clone	<code>git clone <url></code> downloads a full copy of a repository and its history.
Commit hash	A unique identifier for one exact commit, for example <code>a257d05</code> , used to reference or roll back to that exact state.
<code>git status</code>	Shows which files are changed, staged, or untracked in the working copy.
<code>git diff</code>	Shows the exact line-by-line changes that have not yet been committed (or between two commits).
Why it matters	Full history of every change, the ability to see exactly what changed and why, safe collaboration, and here, it is also the trigger for both deployment platforms.

4.14 Vercel and Render, in depth

See Section 3.2 for the project-specific why/where. In short: **Vercel** builds and serves the static React frontend with automatic HTTPS and automatic redeploys on push; **Render** builds the project's Dockerfile into a container and runs it as a persistent web service, also with automatic HTTPS and automatic redeploys on push, and is where the actual Python/MediaPipe/WebSocket backend lives. **Free-tier cold start** (Render specific): after roughly 15 minutes of no traffic the container is stopped to save resources; the next request triggers a fresh start, which can take up to about 30 seconds, during which the frontend shows a "backend may be waking up" message instead of failing

silently.

5. File-by-File Project Explanation

Only real application and configuration files are covered; generated dependency files (`package-lock.json`, `node_modules`, `.venv`) are skipped.

5.1 Backend application (`backend/app/`)

File	Purpose / key contents	Say this if asked
<code>main.py</code>	Creates the FastAPI <code>app</code> object, adds the CORS middleware, includes the REST and WebSocket routers, defines <code>GET /</code> .	"This is where the app is assembled; Uvicorn imports <code>app</code> from here."
<code>routes.py</code>	<code>GET /health</code> , <code>GET /gestures</code> , <code>GET /model</code> .	"Plain request/response endpoints, as opposed to the streaming WebSocket."
<code>ws.py</code>	The <code>/ws</code> WebSocket route: origin check, per-connection receive loop and process loop, the "latest frame wins" logic, config/reset message handling.	"This file is the real-time core of the backend."
<code>schemas.py</code>	Every Pydantic model: REST responses and every WebSocket message shape.	"One place defines every piece of data the API sends or receives."
<code>gestures.py</code>	<code>SUPPORTED_GESTURES</code> , the single source of truth for the six gestures (id, name, emoji, description).	"Both <code>/gestures</code> and the classifier import this same list, so they cannot drift apart."
<code>config.py</code>	Reads all <code>GESTUREFLOW_*</code> environment variables: allowed origins, WS origin-check toggle, classifier choice, model/report paths.	"All production configuration is environment variables, nothing hard-coded."

5.2 Computer-vision pipeline (`backend/app/vision/`)

File	Purpose / key contents	Say this if asked
<code>decode.py</code>	<code>decode_jpeg</code> (JPEG bytes to NumPy image via OpenCV), <code>downscale</code> (safety-net resize).	"Turns the raw bytes the browser sent back into pixels."
<code>landmarks.py</code>	<code>LandmarkDetector</code> class wrapping MediaPipe Hands; <code>detect()</code> returns 21 points or None.	"The only file that talks to MediaPipe directly."

<code>normalize.py</code>	<code>normalize_landmarks</code> : translate to the wrist, scale by hand size, deliberately no rotation.	"Makes the same gesture produce the same numbers regardless of position or distance from the camera."
<code>geometry.py</code>	<code>distance</code> and <code>angle</code> vector-math helpers, shared by the finger analysis and the features module.	"Small, reusable math, not gesture-specific logic."
<code>fingers.py</code>	<code>finger_states</code> : is each of the five fingers extended, based on tip-versus-knuckle distance from the wrist (rotation-independent), with a separate joint-straightness test for the thumb.	"Bridges raw geometry into a human-readable five-finger pattern."
<code>features.py</code>	<code>extract_features</code> : the 25-value feature vector, plus <code>FEATURE_VERSION</code> used by both training and inference.	"The single feature definition both the training script and the live backend use, so they can never drift apart."
<code>classifier_rules.py</code>	<code>classify()</code> : scores the finger pattern against a template per gesture, applies geometric checks (thumb-points-up-and-protrudes, pinch distance for OK sign), enforces a minimum confidence and a runner-up margin.	"The rule-based baseline; every threshold here came from looking at real recorded data."
<code>classifier_ml.py</code>	<code>MLClassifier</code> : loads the joblib model, checks its <code>feature_version</code> , runs <code>predict_proba</code> , raises <code>ModelUnavailable</code> on any problem.	"The inference-only wrapper around the trained model; training itself lives in <code>backend/ml/</code> ."
<code>smoothing.py</code>	<code>GestureSmoother</code> : majority vote over the last 6 predictions, requires a new gesture to win 2 frames in a row before switching.	"Removes single-frame flicker so the UI does not jump between gestures."
<code>pipeline.py</code>	<code>GesturePipeline</code> : ties every stage together end to end for one connection; <code>process_frame()</code> ; <code>set_classifier()</code> to switch rule/ml live with automatic fallback.	"One object per WebSocket connection; this is what <code>ws.py</code> actually calls per frame."
<code>types.py</code>	Shared dataclasses (<code>HandLandmarks</code> , <code>GesturePrediction</code> , <code>FrameResult</code>) and the MediaPipe landmark index constants.	"Keeps every stage talking about landmarks the same way, by name, not magic numbers."

5.3 ML pipeline (backend/ml/)

File	Purpose / key contents	Say this if asked
<code>dataset.py</code>	The on-disk dataset format (CSV of normalized landmarks plus label and session), <code>load_dir</code> , <code>validate</code> , <code>summary</code> .	"Stores normalized landmarks, never raw images; a dataset row is one hand pose."
<code>synthetic.py</code>	Hand-built synthetic 21-point hands for unit tests, plus a jittered synthetic-dataset generator used to smoke-test the training pipeline before real data existed.	"Used for testing the code, not for the shipped model, which is trained on real recorded data."
<code>train.py</code>	Loads the dataset, splits it by session, trains and compares four candidates plus the rule-based baseline, picks the best by macro F1, saves the model and writes the reports.	"Running <code>python -m ml.train</code> is the entire training pipeline, end to end."
<code>evaluate.py</code>	Re-scores an already-saved model against a dataset directory, for checking on fresh data later.	"Lets you evaluate without retraining."
<code>metrics.py</code>	Shared accuracy/precision/recall/F1/confusion-matrix computation used by both <code>train.py</code> and <code>evaluate.py</code> , so the two always report identically.	"One metrics implementation, not two that could disagree."
<code>models/gesture_clf.joblib</code>	The committed, trained Logistic Regression pipeline (about 4 KB); the only model file tracked in Git.	"Small enough and real enough to commit directly for deployment."
<code>reports/comparison.json</code> , <code>EVALUATION.md</code>	The committed, real evaluation report: every candidate's metrics, the confusion matrices, which model was selected.	"Served live by <code>GET /model</code> and shown on the frontend's Model page."

5.4 Developer scripts

File	Purpose
<code>scripts/record_fixtures.py</code>	Opens the webcam, runs MediaPipe live, and on a keypress saves normalized landmarks with a gesture label and session name; this is exactly how the real 2,258-sample dataset was collected.

scripts/ws_smoke.py

A small standalone WebSocket client for manually exercising `/ws` against a running server, without needing a browser.

5.5 Frontend (`frontend/src/`)

File	Purpose
<code>main.tsx</code>	Entry point: mounts React, wraps the app in the router and the settings/session-recorder context providers.
<code>App.tsx</code>	The route table: Live, Dashboard, Guide, Model, About, and a 404 page, all inside a shared <code>Layout</code> .
<code>config.ts</code>	Reads <code>VITE_API_BASE_URL</code> and <code>VITE_WS_URL</code> ; logs a console error in a production build if either is missing.
<code>types.ts</code>	TypeScript interfaces mirroring every backend Pydantic model.
<code>pages/LivePage.tsx</code>	Orchestrates the webcam, the socket, and the overlay; owns the small bits of session state like fps and detection count.
<code>pages/DashboardPage.tsx</code>	Renders the charts and history timeline from <code>useSessionRecorder</code> 's data.
<code>pages/GuidePage.tsx</code>	Fetches <code>GET /gestures</code> and lists them; never hard-codes the gesture list.
<code>pages/ModelPage.tsx</code>	Fetches <code>GET /model</code> and renders the metrics table and confusion matrices.
<code>pages/AboutPage.tsx</code>	Static explanation of the architecture and pipeline.
<code>hooks/useWebcam.ts</code>	<code>getUserMedia</code> wrapper with distinct states for idle, requesting, active, denied, not found, in use, unavailable, error; reacts if the camera track ends unexpectedly.
<code>hooks/useGestureSocket.ts</code>	Owns the WebSocket connection: send loop throttled to the server's recommended fps, reconnect with exponential backoff, pushing config changes (threshold, classifier) live.
<code>hooks/useSettings.tsx</code>	React Context for mirror, overlay, target fps, sensitivity, and engine choice, persisted to <code>localStorage</code> .
<code>hooks/useSessionRecorder.tsx</code>	React Context recording a throttled stream of results for the Dashboard, in memory only, cleared on refresh.

<code>lib/frameCapture.ts</code>	Draws the video frame to a reused canvas, downsizes it, and produces a JPEG Blob .
<code>lib/drawLandmarks.ts</code>	Draws the 21-point skeleton and its bone connections onto the overlay canvas.
<code>lib/api.ts</code>	Thin <code>fetch</code> wrappers for the REST endpoints.
<code>lib/session.ts</code>	Pure functions turning raw result samples into chart-ready data: gesture segments, frequency counts, a confidence time series.

5.6 Configuration and infrastructure files

File	Purpose
<code>Dockerfile</code> (repo root)	Builds the backend image: <code>python:3.12-slim</code> base, installs <code>libgl1/libglib2.0-0</code> , installs <code>requirements.txt</code> only (no dev/test tools), copies the app code plus the trained model and report, creates and switches to a non-root user, declares a <code>HEALTHCHECK</code> on <code>/health</code> , runs Uvicorn on <code>\$PORT</code> .
<code>.dockerignore</code>	Excludes tests, the raw dataset, dev-only config, and the frontend from the Docker build context, keeping the image small.
<code>frontend/vercel.json</code>	Declares the Vite framework preset and a catch-all rewrite to <code>index.html</code> so client-side routes survive a hard refresh.
<code>.github/workflows/ci.yml</code>	Two GitHub Actions jobs, <code>backend</code> and <code>frontend</code> , running lint, type-check, tests, and build on every push and pull request to <code>main</code> .
<code>backend/requirements.txt</code>	Exact, pinned runtime dependencies (FastAPI, Uvicorn, MediaPipe, NumPy, scikit-learn, joblib): what actually ships to production.
<code>backend/requirements-dev.txt</code>	<code>requirements.txt</code> plus <code>pytest</code> and <code>Ruff</code> , for local development and CI only.
<code>backend/pyproject.toml</code>	<code>Ruff</code> and <code>pytest</code> configuration in one file.
<code>frontend/package.json</code>	Frontend dependencies and npm scripts (<code>dev</code> , <code>build</code> , <code>lint</code> , <code>typecheck</code> , <code>test</code>).
<code>backend/.env.example</code> , <code>frontend/.env.example</code>	Document every environment variable the project reads, with no real secrets, since none of these values are secret.
<code>LICENSE</code>	MIT License.

5.7 Documentation files

File	Purpose
<code>README.md</code>	The project's public front page: overview, live demo links, features, results, tech stack, architecture, setup, deployment, limitations.

<code>docs/decisions.md</code>	A running log of ten non-obvious engineering decisions and the reasoning and alternatives considered for each; written specifically to be interview-readable.
<code>docs/DEPLOYMENT.md</code>	The full deployment runbook: environment variables, exact Render and Vercel steps, a post-deploy checklist.
<code>docs/progress.md</code>	An internal phase-by-phase build log kept across the project's development.
<code>docs/screenshots/</code>	Real screenshots of the deployed application (Live, Dashboard, Model, About pages).

6. The Machine Learning Pipeline in Detail

6.1 Dataset

The dataset was collected personally with `backend/scripts/record_fixtures.py`: the script opens the webcam, runs the same MediaPipe landmark detector the live app uses, and on a keypress writes one row, the 21 normalized landmarks plus the gesture label and a session name, to a CSV file. No images are ever saved, only the 63 numbers (21 points times x, y, z) per sample.

Fact	Verified value
Total samples	2,258
Recording sessions	2 (named s1 and s2 in the data)
Samples in session s1	1,115
Samples in session s2	1,143
Gesture classes	6 (open_palm, fist, thumbs_up, victory, pointing_up, ok_sign)
Samples per class	fist 384, ok_sign 352, open_palm 372, pointing_up 381, thumbs_up 398, victory 371
Data type stored	Normalized landmarks (never raw images)

6.2 Feature extraction

Rather than feeding the model raw pixels or even the raw 63 landmark coordinates directly, each normalized hand is converted into **25 engineered geometric features** by `app/vision/features.py`: five finger extension ratios, five finger curl angles, four adjacent-fingertip distances, a thumb-to-index-knuckle distance, the maximum fingertip spread, the mean fingertip reach from the wrist, a count of fingertips above the wrist, an index of which fingertip is highest, the palm width, and the five individual fingertip y-coordinates.

Why engineered features rather than raw coordinates

They are rotation- and scale-tolerant by construction (they are ratios and angles, not absolute positions), so a dataset of a few thousand samples can generalize; raw coordinates would need far more data to learn the same invariances. Every feature is also individually explainable in one sentence, and the same feature set is used for both training and live inference, guarded by a `FEATURE_VERSION` number so a stale model can never silently be fed a different feature layout than it was trained on.

6.3 Training and model comparison

`python -m ml.train` is the entire pipeline: load the dataset, extract features for every sample, split into train and test, fit four candidate models plus score the existing rule-based classifier on the same test set, then save the best model and write the reports.

Avoiding data leakage: the session-held-out split

Consecutive frames of the same held gesture look almost identical. A naive random train/test split would put near-duplicate frames on both sides of the split, letting the model effectively memorize rather than generalize, and inflate the reported accuracy. Instead, `train.py` uses scikit-learn's `GroupShuffleSplit` keyed on the recording session: whichever session a sample came from, all of that session's samples go entirely to train or entirely to test. In this run, session `s1` (1,115 samples) became the training set and session `s2` (1,143 samples), a separately recorded session, became the test set. The reported accuracy therefore reflects the model seeing genuinely new footage, not memorized near-duplicates.

6.4 The five candidates compared

Model	Accuracy	Macro F1	Role
Most-frequent (dummy)	15.6%	4.5%	Sanity floor: if a model cannot beat always guessing the single most common class, it has learned nothing.
Rule-based classifier	86.4%	83.3%	The existing, zero-training baseline this project already had.
Random Forest	84.3%	83.1%	Ensemble of decision trees; tested for non-linear feature interactions.
SVM (RBF kernel)	90.8%	90.8%	Non-linear margin-based classifier; tested as a stronger alternative to a plain linear model.
Logistic Regression	95.5%	95.4%	Selected and shipped to production.

6.5 Model selection reasoning, including cross-validation

Logistic Regression was chosen purely because it scored highest on the held-out session, which is the number that matters. It is worth understanding *why*, because it is a genuinely interesting result: during training, `train.py` also computes 5-fold cross-validation macro F1 on the training session alone, and there the Random Forest and SVM actually scored *higher* than Logistic Regression (around 0.96 versus about 0.94). But both of those models lost more ground on the truly unseen session 2, while Logistic Regression's score barely moved. That gap is a textbook sign of mild overfitting to patterns specific to one recording session, lighting, exact hand distance, subtle camera framing, that a more flexible model can latch onto but a simpler, closer-to-linear model resists. This is exactly why the split-by-session evaluation exists: a random shuffle would very likely have hidden this and could have shipped a worse-generalizing model.

6.6 Debugging story: the fist vs. thumbs-up confusion

A real, verified bug and its fix — good viva material

After deployment, manual testing showed a closed fist sometimes read as a thumbs-up on the rule-based engine. Investigating with the real dataset showed the rule-based classifier misread about 95 percent of real fists as thumbs-up. The cause: the rule that decided "is the thumb extended" only checked whether the thumb's joints were nearly straight, and a closed fist's thumb rests almost as straight as a raised one, it simply lies alongside the curled fingers instead of sticking out. The fix added one more geometric check, specifically for the thumbs-up rule: the thumb tip must reach noticeably farther from the wrist than the average of the other, curled fingertips (a "protrusion" test), with the exact threshold, 0.55 in normalized units, chosen by directly plotting that value across the real fist and thumbs-up samples and picking the gap between them. This raised

the rule-based classifier's overall accuracy from about 70 percent to about 86 percent and fixed fist recall from about 5 percent to about 87 percent, without touching any of the other five gestures' logic. The same idea was also tried as a new ML feature and retrained; it did not improve the already strong ML model (test F1 moved slightly the wrong way, from 0.954 to 0.949), so that change was reverted, and the ML model shipped is unmodified by this fix. This is a strong example to describe in a viva because it shows debugging grounded in real data rather than guessing, and the discipline to revert a change that did not actually help.

6.7 Serialization, runtime loading, and fallback

`train.py` saves a single `joblib` file containing a dictionary: the fitted scikit-learn pipeline (a `StandardScaler` plus the `LogisticRegression`), the list of feature names, the `FEATURE_VERSION` the model was trained with, the gesture labels, the scikit-learn version, the dataset type ("real"), whether it is provisional, and the held-out test macro F1. At backend startup, each new WebSocket connection builds a `GesturePipeline`; if `GESTUREFLOW_CLASSIFIER=ml`, it tries to construct an `MLClassifier`, which loads the `joblib` file and checks the stored `feature_version` against the code's current one. If the file is missing, unreadable, or version-mismatched, `MLClassifier` raises `ModelUnavailable`, the pipeline logs a warning, and it transparently falls back to the rule-based classifier for that connection, so a model problem degrades the app rather than crashing it. The same switch can also be triggered live, mid-session, via a `{"type": "config", "classifier": "ml"}` WebSocket message from the frontend's Settings panel.

7. Real Project Results

Every number below is read directly from the committed `backend/ml/reports/comparison.json`, re-verified against a fresh run of the backend test suite and a live fetch of `gesture-recognition-app-8ksr.onrender.com/model` before this guide was written.

7.1 Headline results

Metric	Value
Dataset	real (not synthetic), <code>provisional: false</code>
Total samples	2,258
Sessions	2 (session-held-out split: train on s1, test on s2)
Train / test sizes	1,115 / 1,143
Selected model	Logistic Regression
Logistic Regression accuracy	95.45%
Logistic Regression macro F1	95.41%
Logistic Regression macro precision	95.49%
Logistic Regression macro recall	95.38%
SVM (RBF) accuracy / F1	90.81% / 90.81%
Random Forest accuracy / F1	84.25% / 83.06%
Rule-based baseline accuracy / F1	86.35% / 83.27%
Most-frequent dummy accuracy / F1	15.57% / 4.49%
scikit-learn version used	1.6.1
Feature version	1 (25 features)

7.2 Logistic Regression confusion matrix (held-out session s2)

True \ Predicted	open_palm	fist	thumbs_up	victory	pointing_up	ok_sign
open_palm	199	1	0	0	0	13
fist	0	195	2	4	0	2

thumbs_up	0	0	200	0	0	0
victory	0	1	0	177	0	0
pointing_up	0	8	4	1	165	0
ok_sign	14	0	0	1	1	155

Reading it: rows are the true gesture, columns are what the model predicted; the diagonal is correct predictions. **thumbs_up** was perfect (200/200). The two largest confusions are **open_palm** occasionally read as **ok_sign** (13 times) and **ok_sign** occasionally read as **open_palm** (14 times), both make geometric sense since an OK sign is essentially an open palm with the thumb and index finger touching, and **pointing_up** confused with **fist** 8 times, plausible if the index finger was not fully extended in some frames.

7.3 What these results mean, honestly

- 95.5 percent is the accuracy on a genuinely unseen recording session from the same person, same approximate setup; it is **not** a claim about accuracy for a different person, a different camera, or very different lighting (see Section 14, Limitations).
- Macro F1 (95.4 percent) being close to accuracy (95.5 percent) indicates performance is fairly even across the six classes, not skewed by the model doing very well on common gestures and poorly on rare ones.
- The rule-based baseline's 86.4 percent accuracy (after the fist/thumbs-up fix) is a meaningful, honest comparison point: it shows the trained model earns a real, non-trivial improvement, not just a marginal one.

8. The Real-Time System, Explained

This is a real-time system because it forms a continuous loop, capture, send, process, respond, display, that repeats roughly ten times per second for as long as the camera is on, over one persistent connection, with an explicit design choice to always prefer the newest data over completeness.

8.1 The pipeline stage by stage

Stage	What happens
Webcam	The browser's <code>getUserMedia</code> API provides a live video stream once permission is granted.
Frame capture	Roughly every 100ms, the current video frame is drawn to an off-screen canvas.
Downscaling	The captured frame is resized to about 320 pixels wide before being compressed to JPEG; a large image is not needed for landmark detection, and a small one uploads and decodes far faster.
Frame rate	10 frames per second is the server's <code>recommended_fps</code> , sent in the <code>"ready"</code> message; it is adjustable in Settings (3 to 15 fps).
WebSocket transport	Each downsized JPEG is sent as one binary WebSocket message; only one is ever in flight per connection.
Backend processing	Decode, MediaPipe landmark detection, normalize, classify, smooth, typically a few milliseconds to tens of milliseconds of server-side work per frame.
Response	A JSON <code>"result"</code> message streams back with the gesture, confidence, landmarks, and timing.
Visualization	The skeleton overlay and the gesture panel update as soon as each result arrives.

8.2 Latency, throughput, and dropped frames

The backend's "latest frame wins" design means the client never queues frames. If a new frame is captured while the previous one is still being processed on the server, the frontend simply overwrites the pending one, so the round trip never grows an ever-increasing backlog under load. On the server side, if a new frame arrives while the previous one is mid-processing, the connection's receive loop keeps only the newest one and the older frame is counted in `frames_dropped`, reported back to the client in every result message. This means dropped frames are an explicit, visible signal of load, not a silent failure.

8.3 What the session-statistics panel actually measures

Stat	What it means in this implementation
------	--------------------------------------

Connection: Live	The WebSocket is open and has received a "ready" message from the backend.
Connection: Idle	The camera/session has not been started (no socket is open yet).
Connection: Connecting / Reconnecting	The socket is being opened, or was unexpectedly closed and the frontend is retrying with exponential backoff.
Throughput	Results actually received per second, measured client-side over the trailing one-second window, not just the frames sent.
Server latency	<code>inference_ms</code> from the last result message: the backend's own measured processing time for that one frame (decode through classify).
Frames dropped	Cumulative count of frames the backend discarded because a newer one arrived before the previous one finished processing.
Detections	A running count of gesture <i>changes</i> (new stable gesture recognized), not a count of frames.

9. Frontend Explanation

9.1 What happens when the user opens the Live page

1. React Router renders `LivePage` inside the shared `Layout` (nav bar, backend status badge, theme toggle).
2. `useGestures` fetches `GET /gestures` in the background to know the six gestures' names and emoji, for later display.
3. `useBackendHealth` polls `GET /health` periodically to drive the "backend online/offline" badge.
4. The camera and the WebSocket are **not** started automatically; the page shows an idle state and a Start button until the user opts in.
5. Pressing Start calls `useWebcam().start()`, which calls `navigator.mediaDevices.getUserMedia`, triggering the browser's own permission prompt.
6. Once granted, the returned `MediaStream` is attached to a `video` element; `useGestureSocket` then opens the WebSocket and the frame-sending loop begins.

9.2 Browser permissions

Camera access in a browser is permission-gated per origin (per website): the first time a page calls `getUserMedia`, the browser shows a prompt the user must explicitly allow. If denied, `getUserMedia` rejects with a `NotAllowedError`, which `useWebcam` catches and turns into a "denied" state, shown in the UI with a clear message and a way to retry. Other distinct states are handled the same way: no camera present (`NotFoundError`), the camera already in use by another app (`NotReadableError`), and the API being entirely unavailable (for example, an insecure, non-HTTPS context).

9.3 Why production webcam access requires HTTPS

Browsers only allow `getUserMedia` on what they consider a "secure context": pages served over `https://`, or `http://localhost` for local development. This is a browser security policy, not something this project's code enforces or could bypass; a webcam feed is sensitive data, so browsers refuse to hand it to a page an attacker could have intercepted or altered in transit over plain HTTP. Both Vercel (frontend) and Render (backend) serve everything over HTTPS automatically, which is why camera access works from the deployed URLs.

9.4 Page-by-page summary

Page	What it shows
Live (/)	The camera feed, the 21-point skeleton overlay, the current gesture with a confidence bar, connection/session stats, Start/Stop/Reset, and the Settings panel.
Dashboard (/dashboard)	Gesture-frequency bar chart, confidence-over-time area chart, and a gesture-history timeline, all built from the current in-browser session, cleared on refresh.
Guide (/guide)	The six supported gestures with their names, emoji, and descriptions, fetched live from <code>GET /gestures</code> .
Model (/model)	The real evaluation report from <code>GET /model</code> : the metrics table for every candidate and two confusion-matrix heatmaps.

9.5 Settings and the engine toggle

The Settings panel (a disclosure opened from the Live page) exposes: mirror the video, show or hide the skeleton overlay, target frame rate, sensitivity (mapped to the backend's confidence threshold), and a **Recognition engine** toggle between **Rule-based** and **ML model**. The ML option is automatically disabled in the UI if `GET /model` reports no trained model is available. Changing the engine sends a `{"type": "config", "classifier": "ml"}` (or `"rule"`) message over the already-open WebSocket, so the switch takes effect on the very next frame, with no reconnect needed. All settings persist to `localStorage` so they survive a page reload.

10. Backend Explanation

10.1 Application startup

`app/main.py` constructs the FastAPI app, adds the `CORSMiddleware` (configured from `GESTUREFLOW_ALLOWED_ORIGINS`), and includes the REST router (`routes.py`) and the WebSocket router (`ws.py`). Uvicorn then imports this `app` object and starts serving. No database connection or other heavyweight setup happens at startup; MediaPipe is instead initialized *per WebSocket connection* inside `GesturePipeline`, because its internal tracking state is not safe to share between concurrent users.

10.2 Configuration

`app/config.py` reads every setting from environment variables at import time: `GESTUREFLOW_ALLOWED_ORIGINS` (CORS and WebSocket origin allow-list), `GESTUREFLOW_WS_ALLOW_ANY_ORIGIN` (disables the origin check, for local tooling only), `GESTUREFLOW_CLASSIFIER` (default engine, `rule` or `ml`), `GESTUREFLOW_MODEL_PATH` and `GESTUREFLOW_MODEL_REPORT` (override file locations). None of these are secrets.

10.3 Routes

Endpoint	Behavior
GET /	A friendly landing JSON response pointing at <code>/docs</code> .
GET /health	Returns <code>{"status":"ok", "service":"gestureflow-backend", "version"}</code> used by the frontend badge and by Render's health check.
GET /gestures	Returns the six supported gestures, straight from <code>app/gestures.py</code> .
GET /model	Returns which classifier is active by default, whether the ML model file is present, and the full committed evaluation report.
GET /docs, /redoc, /openapi.json	Auto-generated interactive API documentation from the Pydantic schemas, provided by FastAPI itself.
WS /ws	The streaming endpoint; see below.

10.4 The `/ws` endpoint in detail

- On connect, before accepting, `_origin_allowed()` checks the browser's `Origin` header against `GESTUREFLOW_ALLOWED_ORIGINS`; a request with no Origin header (a non-browser client) is allowed, a present-but-unlisted Origin is rejected with WebSocket close code 1008 (policy violation).
- Once accepted, a `GesturePipeline` is created for this connection (loading MediaPipe and, if configured, the ML model), and a `"ready"` message is sent.
- Two concurrent tasks then run for the life of the connection: a **receive loop** that reads every incoming message and keeps only the newest binary frame (plus handling `"reset"/"config"` text messages), and a **process loop** that, whenever a new frame is waiting, runs it through the pipeline in a worker thread (since MediaPipe is synchronous/blocking, this keeps it off the main async event loop) and sends back a `"result"` or, on a decode

failure, an "error" message without closing the connection.

4. On disconnect, the pipeline is closed, releasing the MediaPipe resources for that connection.

10.5 Error handling and the fallback classifier

A malformed WebSocket control message is validated with Pydantic and simply ignored if invalid, never crashing the connection. An undecodable frame produces an "error" message, not a disconnect. A request to switch to the ML classifier when no model is available (or the model file is incompatible) is met with an "error" message reporting why, while the connection quietly keeps running on the rule-based classifier, so the video stream itself never stops because of a classifier problem.

10.6 Why the backend is deployed separately from the frontend

The frontend is a static site, plain HTML/CSS/JS files with no server-side process, which is exactly what Vercel is built to serve efficiently and cheaply. The backend needs a real, always-running process holding open WebSocket connections and running MediaPipe, which requires a platform that runs containers, not a static file host. Splitting them also means each half can be scaled, redeployed, and rolled back independently.

11. Deployment, Start to Finish

Live, verified production URLs

Frontend (Vercel)	https://gesture-recognition-app-tawny.vercel.app/
Backend (Render)	https://gesture-recognition-app-8ksr.onrender.com
Health check	https://gesture-recognition-app-8ksr.onrender.com/health
Model info	https://gesture-recognition-app-8ksr.onrender.com/model

11.1 The chain, end to end

1. **GitHub repository** ([prakrash2003-web/gesture-recognition-app](#), branch `main`) is the single source of truth both platforms deploy from.
2. **Render** watches the repository. On a push to `main`, it builds the repo-root `Dockerfile` into a new image (installing `libgl1/libglib2.0-0`, the pinned Python dependencies, and copying in the app code, the trained model, and the report) and starts a new container from it, on a URL that does not change between deploys.
3. **Backend environment variables** are set once in the Render dashboard: `GESTUREFLOW_CLASSIFIER=m1` (serve the trained model) and `GESTUREFLOW_ALLOWED_ORIGINS` set to the exact deployed frontend origin. Render itself injects `PORT`.
4. **Vercel** separately watches the same repository, with its project root set to `frontend/`. On a push, it runs `npm run build` and serves the resulting `dist/` as a static site behind its own CDN and automatic HTTPS.
5. **Frontend environment variables**, `VITE_API_BASE_URL` and `VITE_WS_URL`, are set in the Vercel dashboard to the Render backend's `https://` and `wss://.../ws` URLs; because Vite bakes these in at build time, changing them requires a new deploy to take effect.
6. The deployed frontend now calls the deployed backend over HTTPS for REST calls and WSS for the gesture stream, both encrypted, both required for camera access to even be allowed by the browser.

11.2 Why HTTPS and WSS specifically

HTTPS is required because browsers refuse camera access on an insecure page (Section 9.3), and because a page served over HTTPS is itself blocked from opening a `plain ws://` connection, a "mixed content" restriction, so the WebSocket must be `wss://` to match. **WSS** is simply the WebSocket protocol running over the same TLS encryption as HTTPS.

11.3 Environment variables, precisely

Variable	Where	Purpose
<code>GESTUREFLOW_ALLOWED_ORIGINS</code>	Render (backend)	CORS allow-list for REST, and the WebSocket Origin allow-list for <code>/ws</code> .
<code>GESTUREFLOW_CLASSIFIER</code>	Render (backend)	Which classifier a new connection starts with, <code>m1</code> in production.
<code>PORT</code>	Render (injected automatically)	Which port Uvicorn should listen on inside the container.

VITE_API_BASE_URL	Vercel (frontend)	The base URL for REST calls (/health, /gestures, /model).
VITE_WS_URL	Vercel (frontend)	The full WebSocket URL, including the /ws path.

11.4 Render free-tier behavior

On Render's free tier, the backend container is stopped after roughly 15 minutes with no incoming traffic, to save resources. The next request after that has to wait for a fresh container start, observed to take up to about 30 seconds. This is documented in [docs/DEPLOYMENT.md](#) and handled explicitly in the UI: the Live page shows a "backend may be waking up" message during a slow first connection rather than presenting a bare error.

12. GitHub and Version-Control Journey

The project lives at github.com/prakrash2003-web/gesture-recognition-app, developed directly on the `main` branch with 16 commits at the time of writing, each a logical, self-contained step (a scaffolded backend, the offline vision pipeline, the WebSocket endpoint, the frontend shell, connecting them end to end, dashboard and settings, the ML classifier, deployment scaffolding, a real-data model finalization, a targeted bug fix, and documentation passes).

12.1 Selected commits actually in the repository history

Commit	What it did
fbf82b5	Initial repository structure and tooling
c31f505	Offline computer-vision pipeline (landmarks to gesture)
09b82bf	The <code>/ws</code> WebSocket endpoint
8c9b254	Live webcam to WebSocket to gesture, connected end to end
b88c0f3	The optional trained ML classifier alongside the rule baseline
6e31e6b	Finalized the real (non-synthetic) model evaluation
21c76e3	Fixed the fist/thumbs-up misclassification; deployment hardening
a257d05	Added real screenshots of the deployed app to the README

12.2 What happens on `git push`

Pushing to `main` uploads the new commits to GitHub. Two independent things then react to that: **GitHub Actions** runs the CI workflow (lint, type-check, test, build for both halves) and reports pass/fail directly on the commit; separately, **Render** and **Vercel** each detect the change on `main` and automatically rebuild and redeploy their half of the app from the new code, with no manual deploy step required. CI passing is not what triggers deployment; the two are independent, deployment happens regardless of CI's result unless the platform is specifically configured to gate on it.

13. Testing

Re-verified at the time of writing this guide: **93 backend tests pass** (pytest, 15 test files) and **93 frontend tests pass** (vitest run, 22 test files); Ruff and oxlint both report clean.

13.1 What the backend's 93 tests cover

Area	Example test files
Vision math	<code>test_geometry.py</code> , <code>test_normalize.py</code> , <code>test_fingers.py</code> , <code>test_features.py</code>
Rule-based classifier	<code>test_classifier_rules.py</code> , including the fist/thumbs-up regression tests
Frame decoding, landmark detection, smoothing, pipeline	<code>test_decode.py</code> , <code>test_landmarks.py</code> , <code>test_smoothing.py</code> , <code>test_pipeline.py</code>
ML dataset and training	<code>test_ml_dataset.py</code> , <code>test_ml_train.py</code> , <code>test_classifier_ml.py</code> (uses a small model trained on synthetic data within the test, never the committed real model)
REST routes	<code>test_routes.py</code> (/health, /gestures, /model)
WebSocket behavior	<code>test_ws.py</code> (ready message, streaming a frame, bad frames, reset/config messages, ML-unavailable fallback, allowed/disallowed/missing Origin)
CORS	<code>test_cors.py</code>

13.2 What the frontend's 93 tests cover

Pure logic (frame-capture size math, landmark-drawing calls against a fake canvas context, session-data derivations like gesture segmentation and confidence series), hooks (`useWebcam`, `useGestureSocket` against a mock WebSocket, `useSettings`, `useSessionRecorder`), and components (`GesturePanel`, `ConnectionBanner`, `SettingsPanel`, `NavBar`, `ConfusionMatrix`, and each page).

13.3 Other verification in the project

- **Ruff** (lint and format) and **oxlint** run in CI on every push, alongside the tests.
- `scripts/ws_smoke.py`: a manual, non-pytest WebSocket client used to exercise `/ws` against a real running server by hand.
- **Render's Docker HEALTHCHECK** and its dashboard health-check path both call `/health` to confirm the container is actually serving, not merely running.
- Before writing this guide, the deployed `/health` and `/model` endpoints were fetched live and checked against the committed report, and the test suites were re-run fresh, rather than trusting older numbers.

13.4 What these tests prove, and what they do not

Be precise about this in a viva

They prove: the vision math, the classifiers, the WebSocket protocol, the REST endpoints, and the frontend logic behave correctly against known inputs, including several genuinely tricky edge cases (a missing model file, a disallowed Origin, a malformed message, a fist that should not be a thumbs-up). **They do not prove:** that the trained model will perform at 95.5 percent accuracy for a different person, camera, or lighting setup (that would need a much larger, more diverse dataset); they also do not measure production-scale concurrency or load, since none of the 93 tests spin up dozens of simultaneous real WebSocket connections against a live server.

14. Limitations (Honest Assessment)

Limitation	Detail
Free-tier cold start	The Render backend sleeps after about 15 minutes idle; the first request afterward can take up to about 30 seconds.
Single hand only	The MediaPipe detector is configured with <code>max_num_hands=1</code> ; a second hand in frame is simply ignored.
Six gestures only	Anything outside <code>open_palm</code> , <code>fist</code> , <code>thumbs_up</code> , <code>victory</code> , <code>pointing_up</code> , and <code>ok_sign</code> is reported as unrecognized, never guessed as one of the six.
Dataset size and diversity	2,258 samples from one person across two sessions is enough to demonstrate a real, working, honestly-evaluated pipeline, but is not enough data to claim strong generalization across different people, skin tones, hand sizes, or cameras.
Extreme rotation	Both classifiers' accuracy degrades at extreme in-plane hand rotation, documented in docs/decisions.md and docs/progress.md as roughly beyond plus or minus 45 degrees.
Session-only history	Gesture history and dashboard statistics live only in the browser tab's memory; nothing is persisted server-side or across a page refresh, there is no database and no user accounts.
No automated load testing	Concurrent-user scalability has not been measured (see Section 19 for how this would be approached).

15. Future Improvements

Grounded in the project's own documented plans ([docs/progress.md](#), [README.md](#)), not invented for this guide:

- **Expand the training dataset** with more contributors and sessions, to test and improve generalization across different people, hand sizes, and skin tones.
- **Support two-hand gestures**, which would need `max_num_hands` raised and the classifier and message protocol extended to carry more than one hand's data.
- **Add more gesture classes**, which the architecture already supports cleanly since the gesture list, the rule templates, and the ML labels are all driven from one place ([app/gestures.py](#)).
- **Automated visual regression or screenshot testing in CI**, to catch UI regressions the current unit/component tests would not.
- **Optional user accounts to persist gesture history** across sessions, which the current architecture deliberately does not have (no database, no accounts).
- **A CI job that actually builds the Docker image**; today the Dockerfile is validated only by the deploy platform's own build, not by GitHub Actions.

16. How I Should Explain This Project to an Examiner

A natural, first-person speaking script, written the way a student would actually talk, not marketing copy. Practice it out loud, section by section, in your own words rather than memorizing it word for word; the point is to internalize the flow, not recite this exact text.

Problem and objective. "I wanted to build something that was not just a machine-learning notebook, a complete, real, deployed system that actually demonstrates the full stack: a real-time frontend, a real backend, real computer vision, and a real trained model with an honest evaluation. So GestureFlow is a hand-gesture recognition app: you show your webcam one of six hand gestures, and it recognizes it live."

Architecture. "The architecture is a React frontend talking to a Python FastAPI backend over a WebSocket, because this is a continuous, real-time problem, not a one-off request. The frontend captures and streams webcam frames; the backend runs the actual computer vision and classification and streams results back."

Data. "For the machine-learning side, I collected my own dataset. I wrote a small script that uses the same MediaPipe detector the live app uses, and I recorded myself performing all six gestures, over two separate sessions, 2,258 samples in total. I stored the landmark coordinates, not raw images, which keeps the dataset small and avoids storing any actual video of me."

MediaPipe. "The hand detection itself uses Google's MediaPipe Hands, a pretrained model that gives me 21 (x, y, z) points on the hand for every frame. I did not build that part myself, it is a solved problem, and I want to be upfront about that in an interview, what I built is everything that happens after those 21 points."

Feature extraction. "From those 21 points, I engineered 25 features, things like how far each finger extends relative to the wrist, the bend angle at each finger joint, and the distances between fingertips, instead of feeding the model raw coordinates. That makes the features tolerant to where the hand is in frame, how far from the camera it is, and to some rotation, so a dataset of a couple thousand samples is actually enough to train on."

ML models and model selection. "I trained and compared four models: a dummy baseline that just guesses the most common class, Logistic Regression, an RBF SVM, and a Random Forest, and I also scored my existing rule-based classifier on the exact same test set. The important design decision was how I split the data: I split by recording session, not randomly, training on one session and testing on the other, because a random split would let near-identical consecutive frames leak between train and test and give an inflated, dishonest accuracy number. Logistic Regression won, 95.5 percent accuracy, and interestingly it beat the more complex models specifically because they overfit a bit to one session and generalized worse to the other, which is a genuinely useful thing to have seen firsthand."

Backend. "The backend is FastAPI. It has REST endpoints for health checks and for exposing which gestures and which model results it has, and it has one WebSocket endpoint that does the real work: receive a frame, decode it, run MediaPipe, normalize the landmarks, run the classifier, smooth the result over the last few frames so it does not flicker, and send the result back."

WebSocket. "I chose a WebSocket over repeatedly calling a REST endpoint because this is a continuous stream in both directions, frames going up, results coming down, roughly ten times a second, and a WebSocket is one connection that stays open for that, instead of paying the overhead of a new HTTP request every single frame."

Frontend. "On the frontend, once you grant camera permission, I capture a frame, downscale it, and send it as a small JPEG over the socket. I only ever let one frame be in flight, if a new one is ready before the backend replies, I send the newer one instead of queuing, so the whole system stays responsive under load rather than falling further and further behind. The result comes back and I draw the hand skeleton over the video and show the gesture and its confidence, and I also feed every result into a small in-memory session store that powers a dashboard with charts and a history timeline."

Deployment. "It is actually deployed, not just running on my laptop. The frontend is a static build on Vercel, the backend is a Docker container on Render, and they talk to each other over HTTPS and a secure WebSocket. I had to think about things like CORS and validating the WebSocket's Origin header so only my frontend can connect to the backend in production, and about environment variables so the same code works locally and in production without changing anything."

Results. "The trained model gets 95.5 percent accuracy and 95.4 percent macro F1 on a genuinely held-out session, compared to about 86 percent for my rule-based baseline, and I can show the confusion matrix and explain exactly which gestures still get confused and why."

Limitations. "I am upfront about the limitations: the dataset is from one person over two sessions, so I would not claim this generalizes perfectly to anyone's hands; it only recognizes six gestures and one hand at a time; and the backend is on a free hosting tier, so the very first request after it has been idle can be slow while it wakes up."

Future scope. "Given more time, I would collect data from more people to test real generalization, support two hands, and add a small CI step that actually builds the Docker image, not just tests the code."

17. Viva Questions — Basic Level (50)

#	Question	Model answer
1	What is Python?	An interpreted, dynamically typed, general-purpose programming language, used here for the entire backend.
2	Why Python for this project?	It has mature libraries for computer vision (OpenCV, MediaPipe) and machine learning (scikit-learn) in one ecosystem.
3	What is React?	A JavaScript/TypeScript library for building UIs from reusable components that re-render when their state changes.
4	What is FastAPI?	A Python web framework for building APIs, with native WebSocket support and automatic validation/docs from Pydantic models.
5	What is MediaPipe?	A pretrained computer-vision toolkit from Google; here, its Hands model finds 21 landmark points on a hand.
6	What is machine learning?	Building a system that learns patterns from data rather than being explicitly programmed with fixed rules for every case.
7	What is supervised learning?	Learning a mapping from inputs to known correct outputs, using labelled training examples.
8	What is classification?	Supervised learning where the output is one of a fixed set of categories, here one of six gestures.
9	What is a feature?	One measured input number given to a model, for example a finger's curl angle.
10	What is a label?	The correct, known output for a training example, for example the gesture name "fist".
11	What is accuracy?	The fraction of predictions that were exactly correct.
12	What is precision?	Of everything predicted as a class, the fraction that was actually that class.
13	What is recall?	Of everything that actually was a class, the fraction the model correctly found.
14	What is F1 score?	The harmonic mean of precision and recall; macro F1 averages it equally across all classes.

15	What is a confusion matrix?	A grid of true label versus predicted label showing exactly which classes get mixed up with which.
16	What is a WebSocket?	A protocol that upgrades an HTTP connection into one persistent, two-way connection.
17	What is a REST API?	An API style built on plain HTTP requests and responses; one request, one response, then done.
18	What is Docker?	A tool that packages an app and its dependencies into a portable image, run as an isolated container.
19	What is Git?	A distributed version-control system that tracks every change to code over time as commits.
20	What is GitHub?	A hosting service for Git repositories, with collaboration and automation (Actions) features.
21	What is Vercel?	A hosting platform for static sites/frontends; hosts this project's React build.
22	What is Render?	A cloud host that can run Docker containers as web services; hosts this project's backend.
23	What is HTTPS?	HTTP encrypted with TLS; required by browsers for camera access and for opening a secure WebSocket.
24	What is WSS?	WebSocket over TLS, the encrypted counterpart of <code>ws://</code> , required from an HTTPS page.
25	Why do we need environment variables?	So the same code runs locally and in production with different settings, without editing or hard-coding values in source.

#	Question	Model answer
26	Why 21 landmarks specifically?	MediaPipe's hand model was trained to output one wrist point plus four points per finger (base, two knuckles, tip), for five fingers: $1 + 4 \times 5 = 21$.
27	Why not do all the processing only in the browser?	The project's goal is to demonstrate a real Python computer-vision and ML backend; doing everything client-side would make Python a bystander, plus it keeps the interesting logic in one tested codebase.
28	Why use a trained ML model at all if a rule-based classifier already works?	The rule-based engine is a good zero-data baseline, but the real, evaluated data shows the trained model is meaningfully more accurate (95.5% vs about 86%).

29	Why compare multiple models instead of picking one?	To make an evidence-based choice; the comparison revealed the "obvious" more complex models actually generalized worse, which would have been missed otherwise.
30	What is a variable in Python?	A name bound to a value; Python infers its type automatically at runtime.
31	What is a function?	A reusable, named block of code, defined with <code>def</code> .
32	What is a class?	A blueprint for creating objects that bundle data and behavior together, for example <code>GesturePipeline</code> .
33	What is a dictionary in Python?	A collection of key-to-value pairs, used here for things like per-gesture confidence scores.
34	What is JSON?	A lightweight, text-based data format for structured data, used for every API response and WebSocket message.
35	What is a virtual environment?	An isolated, project-only Python installation, so dependencies do not clash across projects.
36	What is pip?	Python's package installer, used to install dependencies listed in <code>requirements.txt</code> .
37	What is a React component?	A function that returns UI markup; the app is built by composing many small components.
38	What is state in React?	Data a component owns and can change itself; changing it triggers a re-render.
39	What is a prop in React?	An input passed into a component from its parent.
40	What is a React hook?	A function starting with <code>use</code> that lets a component use features like state or side effects.
41	What is TypeScript?	JavaScript with an added static type system, checked before the code runs.
42	What is Vite?	A frontend build tool: an instant-reload dev server and a production bundler.
43	What does <code>npm install</code> do?	Downloads and installs the JavaScript dependencies listed in <code>package.json</code> .
44	What is CORS?	A browser security rule that blocks a page on one origin from reading a response from a different origin unless the server explicitly allows it.
45	What is a commit hash?	A unique identifier for one exact saved snapshot of the code, for example <code>a257d05</code> .

46	What does <code>git status</code> show?	Which files are changed, staged, or untracked in the current working copy.
47	What does <code>git diff</code> show?	The exact line-by-line changes that have not yet been committed.
48	What is CI/CD?	Continuous Integration (automatically testing every change) and Continuous Deployment (automatically shipping it); here, GitHub Actions is the CI, Render/Vercel provide the CD.
49	What is a health check endpoint?	A simple endpoint (<code>/health</code>) that reports whether the service is alive, used by monitoring and by the frontend's status badge.
50	What is model serialization?	Saving a trained model's learned parameters to a file (here, a <code>.joblib</code> file) so it can be loaded later without retraining.

18. Viva Questions — Project-Specific (75)

Format for each: Answer, Key points, Possible follow-up.

Q1. Why did you choose MediaPipe for hand detection?

It is a proven, fast, CPU-friendly, pretrained model that solves landmark detection well; building an equivalent detector myself would be a project on its own and would distract from the actual contribution: the classification logic built on top.

Key points: pretrained, bundled model, no separate download, CPU-only, real-time capable.

Follow-up: "Could you swap it for a different detector?" Yes, as long as it produces the same 21-point shape, since `normalize.py` and everything after it only depends on that shape.

Q2. Why 21 landmarks and not fewer or more?

That is simply what MediaPipe's hand model outputs: one wrist point plus four points per finger across five fingers.

Key points: $1 + 4 \times 5 = 21$; not a design choice made by me, a property of the pretrained model.

Follow-up: "Do you use all 21 in every calculation?" No, some features use a subset, for example only the fingertips for spread and reach calculations.

Q3. How are landmarks converted into ML features?

First `normalize_landmarks` translates the 21 points so the wrist is at the origin and scales them by the hand's own size; then `extract_features` computes 25 geometric values from those normalized points, extension ratios, joint angles, tip distances, tip heights, and so on.

Key points: normalization first, then feature engineering; same function used for training and live inference.

Follow-up: "Why normalize before extracting features rather than after?" Because several features are ratios or angles that already need consistent scale to be meaningful; normalizing first keeps the feature code simpler and consistent.

Q4. Why Logistic Regression specifically, given the name suggests regression?

Despite the name, it is a classification algorithm; it produces class probabilities from a weighted sum of features passed through a sigmoid/softmax. It was selected purely because it scored highest, 95.5% accuracy, on the held-out session, and because it generalized better across sessions than the more flexible models.

Key points: name is historical, not descriptive of the task; selection was evidence-based, not a default guess.

Follow-up: "Was it the fastest to train too?" Yes, it is also the fastest and smallest (about 4 KB) of the trained candidates.

Q5. Why did SVM perform differently from Logistic Regression?

The RBF SVM can draw a more flexible, curved decision boundary; here that extra flexibility (90.8% accuracy) did not translate into better generalization to the unseen session, it was simply outperformed by the plainer model.

Key points: more flexibility is not automatically better; results, not theory, decided the winner here.

Follow-up: "Did you tune the SVM's hyperparameters?" A fixed C and gamma="scale" were used; more extensive tuning was not the point of this comparison, an honest baseline comparison was.

Q6. Why did Random Forest perform differently?

It scored 84.3% accuracy on the held-out session despite scoring well in cross-validation within the training session, a sign it fit some session-specific quirks that did not carry over to the second, genuinely different recording session.

Key points: high training-session cross-validation score but the largest drop on the truly unseen session, of the three ML candidates.

Follow-up: "Would more trees have helped?" Possibly marginally, but the gap looked structural (overfitting to session-specific patterns), not simply a matter of more trees.

Q7. Why is the rule-based classifier still useful if the ML model is better?

It needs zero training data, every decision is explainable in one sentence, and it is the automatic fallback the backend uses if the ML model file is ever missing or incompatible, so the app degrades gracefully instead of breaking.

Key points: zero-dependency baseline; production safety net; still meaningfully accurate (about 86%) after the fist/thumbs-up fix.

Follow-up: "Is it still maintained, or was it replaced?" Still maintained and tested; it was actively improved in the fist/thumbs-up bug fix, not abandoned.

Q8. Why is the Most-Frequent baseline useful?

It is a sanity floor: if a real model cannot beat always guessing the single most common class (15.6% accuracy here, since the six classes are roughly balanced), it has not actually learned anything useful.

Key points: scikit-learn's `DummyClassifier`; every other model must clear this bar to be worth using.

Follow-up: "Why is 15.6%, not about 16.7% (1/6)?" Because the six classes are not perfectly balanced in the test session, so the single most common class in the training data does not appear at exactly 1/6 frequency in the test session either.

Q9. How did you prevent data leakage?

By splitting the dataset by recording session rather than randomly, using scikit-learn's `GroupShuffleSplit` keyed on the session id, so all of one session's samples go entirely to train or entirely to test, never split across both.

Key points: consecutive frames of a held gesture are near-duplicates; a random split would leak them across train/test and inflate accuracy.

Follow-up: "What would the number have looked like with a random split?" Almost certainly higher and less trustworthy, since the model could partly memorize near-duplicate frames instead of truly generalizing.

Q10. How exactly was the model evaluated?

Trained on session s1 (1,115 samples), evaluated on session s2 (1,143 samples), a session the model never saw during training, computing accuracy, precision, recall, F1 (macro and per-class), and a confusion matrix, using the same `metrics.py` code for every candidate so results are directly comparable.

Key points: session-held-out, not random; identical metric code for every model, including the rule-based baseline.

Follow-up: "Why also compute 5-fold cross-validation?" To see how each model behaved *within* the training session alone, which is what revealed the Random Forest and SVM's overfitting relative to Logistic Regression.

Q11. Why use macro F1 rather than plain accuracy to pick the model?

Macro F1 averages the F1 score of each of the six classes equally, so a model that does very well on common gestures but poorly on a rarer one is penalized, unlike accuracy, which a large class can dominate.

Key points: macro F1 = fair across classes; accuracy can be misleading when class sizes are unequal.

Follow-up: "Are the six classes actually balanced here?" Reasonably, 352 to 398 samples per class, but macro F1 was still used as the principled choice.

Q12. What is the difference between validation data and test data here?

The 5-fold cross-validation inside `train.py` happens only within the training session (s1) and is used to compare candidates during development; the held-out test set is the entirely separate session (s2), touched only once, to report the final, honest number.

Key points: cross-validation is for model comparison and tuning; the held-out test session is for the final, reported result.

Follow-up: "Was the test session ever used to pick the model?" No, selection was by cross-validation and by scoring on the test session afterward for reporting, not by repeatedly tuning against the test session.

Q13. What would happen with a completely different person's hand?

Likely lower accuracy than 95.5%; the dataset is from one person, so the model has not seen the range of hand sizes, skin tones, or personal gesture "style" a different person would bring, this is explicitly listed as a limitation.

Key points: single-contributor dataset; generalization to new people is untested, not claimed.

Follow-up: "How would you actually test that claim?" Record a small held-out set from several different people and evaluate the existing model against it without retraining.

Q14. Why could the model fail under hand rotation?

The 25 features are largely rotation-tolerant by construction (ratios and angles), but not perfectly rotation-invariant, extreme in-plane rotation, documented at roughly beyond plus or minus 45 degrees, changes the geometry enough that some features drift outside the range the model was trained on.

Key points: engineered features reduce but do not eliminate rotation sensitivity; documented, tested behavior (Section 6.6's rotation investigation), not a guess.

Follow-up: "Did you test this directly?" Yes, by taking real recorded fist samples and rotating them programmatically, which is exactly how the fist/thumbs-up bug was diagnosed.

Q15. Is 95.5% accuracy the same as production accuracy?

No; 95.5% is the accuracy on a held-out recording session from the same person and setup. Real production accuracy for a new, unseen user would likely be lower until the dataset is expanded to cover more people.

Key points: an honest evaluation number is not the same claim as a universal accuracy guarantee.

Follow-up: "Why report it this way instead of overstating it?" Because an inflated, unearned claim would not survive a follow-up question, and understanding the difference is exactly what this project is meant to demonstrate.

Q16. Why can accuracy alone be misleading?

A model can score high accuracy by doing very well on common classes while failing on rarer ones; it also says nothing about *which* gestures get confused with which, which is exactly what the confusion matrix and per-class precision/recall are for.

Key points: use accuracy alongside macro F1, precision, recall, and the confusion matrix, not accuracy alone.

Follow-up: "Give a concrete example from this project." The rule-based classifier's overall accuracy before the fix (about 70%) hid that fist recall specifically was only about 5%, invisible from the accuracy number alone.

Q17. How does the WebSocket connection actually work here, step by step?

Browser opens a socket to `/ws`; backend checks the Origin header, accepts, and sends a "ready" message; the frontend then streams JPEG frames as binary messages roughly every 100ms; the backend replies with a JSON "result" message per frame it processes; either side can close it.

Key points: one persistent connection; binary frames up, JSON results down; origin-checked before accept.

Follow-up: "What closes the connection?" The user stopping the camera, a network drop, or the backend rejecting a disallowed Origin.

Q18. What happens when no hand is detected in a frame?

MediaPipe returns no hand; the pipeline resets the smoothing buffer and returns a result with `hand_present: false`, `gesture: null`, and no landmarks, and the frontend shows "No hand detected" instead of a stale or guessed gesture.

Key points: explicit "no hand" state, not a silent failure or a leftover previous gesture.

Follow-up: "Why reset the smoothing buffer specifically?" So that when a hand reappears, the majority-vote window is not still full of old, now-irrelevant predictions.

Q19. What happens when the ML model cannot be loaded?

`MLClassifier` raises `ModelUnavailable`; the pipeline catches it, logs a warning, and falls back to the rule-based classifier for that connection; if the failure happens from a live "switch to ML" request, the client also receives an explicit error message.

Key points: graceful degradation, not a crash; the frontend is told, not left guessing.

Follow-up: "What specifically can cause this?" A missing file, a corrupted file, or a `feature_version` mismatch between the saved model and the current code.

Q20. How does the frontend know the backend is online?

`useBackendHealth` polls `GET /health` on an interval and drives a status badge in the navigation bar; separately, the WebSocket connection state (connecting/live/reconnecting/error) is shown on the Live page itself.

Key points: two independent signals, general backend health and this specific connection's health.

Follow-up: "What if the health check succeeds but the WebSocket fails?" Both are shown independently, so that specific case would show "backend online" but a reconnecting/error state on the Live page, correctly distinguishing the two.

Q21. How is the skeleton actually drawn?

Each result message includes the 21 landmark points in 0-1 frame-relative coordinates; `drawLandmarks.ts` scales them to the overlay canvas's pixel size and draws lines between the known finger-joint pairs plus a dot at each point, redrawn on every new result.

Key points: coordinates are relative (0 to 1), not pixels, so they scale correctly to any canvas size.

Follow-up: "Why send coordinates 0 to 1 instead of pixels?" Because the frame sent to the backend is downscaled, and the display canvas can be a different size; relative coordinates decouple the two.

Q22. Why is the backend a separate service from the frontend?

The frontend is a static build with no server process, suited to a CDN like Vercel; the backend needs a persistent, always-running process (Uvicorn, holding WebSocket connections, running MediaPipe), which needs a container host like Render. Different requirements, different platforms.

Key points: static hosting versus a long-running server process; independent scaling and redeploys.

Follow-up: "Could you combine them onto one host?" Technically, on a platform that supports both a static build and a long-running Python process, but the project deliberately used the simplest, most standard tool for each half.

Q23. Why use Docker for the backend specifically?

MediaPipe and OpenCV need specific system libraries (`libgl1`, `libglu1`) that are not guaranteed present on a generic Python host; Docker bakes them in so the exact same environment runs locally and on Render.

Key points: reproducible system-level dependencies, not just Python packages.

Follow-up: "Would it work without Docker on Render?" Possibly with Render's native Python runtime and manual system-package configuration, but Docker is more explicit and portable to other hosts too.

Q24. Why Render specifically, and why Vercel specifically?

Render because it runs arbitrary Docker containers as always-listening web services with a free tier, exactly what the WebSocket backend needs; Vercel because it is purpose-built for static frontend frameworks like Vite, with automatic HTTPS and zero-config deploys from GitHub.

Key points: each platform matched to what that half of the app actually needs.

Follow-up: "What was the documented alternative for the backend?" Hugging Face Spaces (Docker SDK) is documented in `docs/DEPLOYMENT.md` as an alternative; Render is what was actually used.

Q25. Why does webcam access require HTTPS?

Browsers only grant `getUserMedia` on a "secure context" (HTTPS, or localhost for development), a browser policy protecting sensitive camera data from being exposed over an interceptable plain HTTP connection.

Key points: browser-enforced, not something the app's own code controls; Vercel provides HTTPS automatically.

Follow-up: "Does this apply to `localhost` too?" No, browsers make an explicit exception for `localhost` so local development works without setting up TLS.

Q26. Why WSS instead of WS in production?

A page served over HTTPS is blocked by the browser from opening a plain, unencrypted `ws://` connection (a mixed-content restriction); it must be `wss://`, the encrypted counterpart, to match.

Key points: mixed-content policy; `VITE_WS_URL` is set to a `wss://` URL in production.

Follow-up: "What would happen if you tried `ws://` from the deployed site?" The browser would refuse to open the connection at all; it never even reaches the server.

Q27. How are allowed origins protected?

Two layers: FastAPI's `CORSMiddleware` restricts which origins can read REST responses, and a custom check in `app/ws.py` reads the WebSocket handshake's Origin header and rejects (close code 1008) anything not in `GESTUREFLOW_ALLOWED_ORIGINS`, since CORS middleware does not cover WebSockets.

Key points: CORS for REST, a separate explicit check for WebSocket, because they are different mechanisms.

Follow-up: "What about a client with no Origin header at all?" Explicitly allowed, since that is how non-browser tools like `scripts/ws_smoke.py` connect; browsers always send an Origin, so this does not weaken protection against unauthorized web pages.

Q28. What does `GESTUREFLOW_ALLOWED_ORIGINS` do?

A comma-separated environment variable listing exactly which browser origins may call the REST API and open the WebSocket; in production it is set to the deployed Vercel URL only.

Key points: one variable, two enforcement points (CORS and WS origin check).

Follow-up: "What happens if it is misconfigured?" Either the real frontend gets blocked (too strict) or an unintended site could call the API (too permissive); it was manually verified against the real Vercel URL after deployment.

Q29. What happens when Render's free tier puts the backend to sleep?

The container is stopped after about 15 minutes of no traffic; the next request triggers a fresh container start, observed to take up to about 30 seconds, during which the frontend shows a "backend may be waking up" message instead of a bare connection error.

Key points: documented, expected behavior, handled explicitly in the UI, not a bug.

Follow-up: "How would you avoid this in a paid setup?" An always-on paid Render instance, or a scheduled ping to keep it warm (with its own cost trade-off).

Q30. What does `GET /health` do exactly?

Returns a small fixed JSON body, `{"status": "ok", "service": "gestureflow-backend", "version": "0.1.0"}`, with no side effects; used by the frontend's status badge and by Render's own health check (and the Docker `HEALTHCHECK` instruction) to confirm the app is actually serving requests.

Key points: liveness check, not a readiness or dependency check.

Follow-up: "Does it check the ML model is loaded?" No, model loading happens per WebSocket connection, not at startup, so `/health` deliberately only reports that the server process itself is up.

Q31. What does `GET /model` do exactly?

Returns which classifier the backend defaults to, whether the trained model file is actually present, and the full committed evaluation report (metrics for every candidate, confusion matrices), read straight from `ml/reports/comparison.json` on disk.

Key points: powers the frontend's Model page and the Settings panel's "is ML available" check.

Follow-up: "Is this data computed live or precomputed?" Precomputed by `python -m ml.train` and committed to the repository; the endpoint just serves that file.

Q32. What happens when a user refreshes `/dashboard` or `/model` directly?

Vercel's rewrite rule in `frontend/vercel.json` serves `index.html` for any path, so the React app loads and React Router then renders the correct page client-side based on the URL; without that rule, the host would look for a literal file at that path and 404.

Key points: SPA hosting needs a fallback rewrite; this project's is explicit and tested.

Follow-up: "Does the Dashboard's data survive that refresh?" No, session data is in-memory only and resets, which is a documented, intentional limitation.

Q33. How are environment variables used across the two halves?

Backend: `app/config.py` reads `GESTUREFLOW_*` variables at import time for origins, classifier choice, and file paths. Frontend: `src/config.ts` reads `VITE_*` variables, baked in at build time by Vite, for the API and WebSocket base URLs.

Key points: backend reads at runtime (can change without rebuilding); frontend reads at build time (needs a rebuild to change).

Follow-up: "What happens if a frontend env var is missing in a production build?" `config.ts` logs a console error and falls back to a localhost default, rather than failing silently.

Q34. What happens if the backend goes offline entirely?

`useBackendHealth`'s polling starts failing and the status badge turns to offline; a new WebSocket attempt fails and `useGestureSocket` retries with exponential backoff, showing a "reconnecting" state rather than a blank or broken UI.

Key points: explicit offline/reconnecting states, not silent failure; backoff avoids hammering the server with retries.

Follow-up: "Is there a maximum retry limit?" The backoff grows but the hook keeps retrying as long as the page stays open; there is no hard cap in the current implementation.

Q35. What do dropped frames mean in this project specifically?

A frame the backend received but did not process because a newer frame had already arrived and replaced it in the connection's single-slot buffer; the count is reported back in every result message as `frames_dropped`.

Key points: intentional design (freshness over completeness), reported transparently, not an error.

Follow-up: "Is a high dropped-frame count a problem?" It signals the backend or network cannot keep up with the requested frame rate; the fix would be lowering the target fps in Settings, not treating it as a bug.

Q36. How would you improve accuracy further?

Primarily by collecting more and more diverse data, more people, more sessions, varied lighting and cameras, since the current model's ceiling is set by what one person's two sessions can teach it; secondarily, revisiting feature engineering with that larger dataset.

Key points: data quantity/diversity is the biggest lever here, not necessarily a fancier model.

Follow-up: "Would a neural network help?" Only with substantially more data than 2,258 samples; with the current dataset size it would likely overfit worse than the simpler models already tested.

Q37. How would you support two hands?

Raise MediaPipe's `max_num_hands`, extend `HandLandmarks`/the WebSocket message schema to carry a list of hands instead of one, and decide a UX for it (independent gestures per hand, or a combined two-hand gesture set), plus retrain or extend the classifier accordingly.

Key points: touches the detection layer, the message protocol, and the classifier; not a one-line change.

Follow-up: "Would the existing 25 features still work?" They would need to be computed per hand; a genuinely two-hand gesture would need new features describing the relationship between the two hands.

Q38. How would you increase generalization across different users?

Collect a dataset from multiple people, and evaluate with a held-out-*person* split (not just held-out session), the same discipline already applied for sessions, extended to people, to get an honest read on cross-person accuracy before claiming it.

Key points: apply the same "held-out split" principle at a person level, not just a session level.

Follow-up: "Would the current model architecture need to change?" Not necessarily; Logistic Regression on the same 25 features might still work well, but that would need to be proven with real cross-person data, not assumed.

Q39. What is the actual difference between rule-based classification and the ML model, in the code?

`classifier_rules.py` is entirely hand-written thresholds and template matching on the finger-state pattern; `classifier_ml.py` loads a scikit-learn pipeline and calls `predict_proba` on the 25-value feature vector; both implement the same interface (`classify(normalized_points) -> GesturePrediction`) so `GesturePipeline` can swap between them without the rest of the code changing.

Key points: same interface, two different implementations; genuinely swappable at runtime.

Follow-up: "Do they share any code?" Yes, both call into `fingers.py/geometry.py`-derived quantities conceptually, and the ML feature set is designed to mirror what the rule engine reasons about, for a fair comparison.

Q40. How is the confidence score computed for each engine?

Rule-based: a heuristic match score, how cleanly the observed finger pattern and geometry fit the winning gesture's template, not a calibrated probability. ML: the actual class probability from `predict_proba`, which Logistic Regression produces naturally from its softmax output.

Key points: two different kinds of "confidence"; the ML one is a genuine probability, the rule-based one is a match-quality heuristic.

Follow-up: "Are they directly comparable?" Not perfectly, they are on the same 0-1 scale but come from different processes; this is disclosed rather than implied to be equivalent.

Q41. Why smooth predictions over multiple frames instead of showing every frame's raw result?

A single noisy frame (motion blur, a half-formed hand shape) can flip the label for an instant; `GestureSmoother` takes a majority vote over the last 6 predictions and requires a new gesture to win 2 frames in a row before switching, which removes that flicker while staying responsive.

Key points: majority vote plus a switch delay; tuned for about 0.6 seconds of context at 10fps.

Follow-up: "Does this add noticeable lag?" A small amount by design, a deliberate trade-off between responsiveness and stability, tuned to feel immediate while still killing flicker.

Q42. What does the sensitivity setting actually change on the backend?

It maps to `min_confidence`, the score a gesture must reach to be reported at all; higher sensitivity lowers that threshold (accepts more borderline gestures), lower sensitivity raises it (stricter). Changing it in Settings sends a live `{"type": "config", "min_confidence": x}` WebSocket message.

Key points: a single number, sent live, applied immediately, no reconnect needed.

Follow-up: "Does sensitivity affect both engines the same way?" Yes, both `classify()` functions accept the same `min_confidence` parameter.

Q43. What is the "runner-up margin" in the rule-based classifier?

Beyond meeting the minimum confidence, the winning gesture must also beat the second-best scoring gesture by a small margin; otherwise the result is reported as unrecognized rather than flickering between two near-tied gestures.

Key points: a second safeguard against ambiguous frames, on top of the confidence threshold.

Follow-up: "Does the ML model have an equivalent?" No, its probabilities already sum to 1 across classes and naturally separate close calls; the margin concept is specific to the rule engine's heuristic scoring.

Q44. How is the gesture-history timeline built on the frontend?

`useSessionRecorder` stores a throttled stream of results in memory; `lib/session.ts`'s pure functions turn that into "segments" (continuous runs of the same gesture), which the Dashboard renders as a chronological list.

Key points: derived, pure-function logic, unit-tested separately from the React components that display it.

Follow-up: "Is this data sent to the server?" No, it never leaves the browser tab; there is no backend storage for session history.

Q45. Why is the ML classifier's inference wrapper separate from the training code?

`app/vision/classifier_ml.py` only loads and runs a model, kept minimal since it runs on every request in production; all the heavier logic, data loading, splitting, training, comparison, lives in `backend/ml/`, which is never imported by the running server.

Key points: production code path is small and fast; training code is a separate, offline concern.

Follow-up: "Does the Docker image include the training code?" No, the Dockerfile only copies `app/` plus the trained model and report, not `ml/*.py`, keeping the image smaller.

Q46. What does `FEATURE_VERSION` protect against?

If the feature-extraction code ever changes (a feature added, removed, or redefined), an old model file trained on the previous feature layout would silently produce garbage predictions if just loaded and fed the new features; `FEATURE_VERSION` is checked on load, so a mismatch raises `ModelUnavailable` instead.

Key points: prevents silent, hard-to-diagnose prediction corruption after a code change.

Follow-up: "Has this actually been exercised?" Yes, during the fist/thumbs-up investigation a 26-feature version 2 was tried, found not to help, and reverted back to version 1, exactly the scenario this guard is for.

Q47. Why does `record_fixtures.py` store landmarks instead of images?

Landmarks are the only data the classifier ever needs; storing images would be unnecessary personal data (a face or background could appear in frame) and would make the dataset far larger for no benefit, since training only ever consumes the 21 points.

Key points: minimal necessary data; a privacy-conscious default, not an afterthought.

Follow-up: "Could you reconstruct an image from the stored landmarks?" No, 21 sparse points carry no photographic detail, they cannot be used to reconstruct what the person or background looked like.

Q48. Why record two sessions instead of one longer session?

A single long session would still let the train/test split leak near-duplicate frames and shared conditions (lighting, distance, camera angle) across both halves; two genuinely separate recording sessions, ideally with varied conditions, is what makes the held-out evaluation honest.

Key points: session count, not just sample count, is what enables a leakage-safe split.

Follow-up: "Would three or more sessions be better?" Likely yes, for both more robust evaluation and, potentially, better generalization if the model were retrained on more varied data.

Q49. How do the backend tests avoid needing a real camera?

Most vision tests feed hand-built, synthetic 21-point arrays (from `ml/synthetic.py`) directly into the `normalize/classify` functions, bypassing `MediaPipe` and the camera entirely; the ML-specific tests train a small model on synthetic data within the test itself, never touching the committed real model.

Key points: synthetic fixtures test the code's logic; they are never used to claim real-world accuracy.

Follow-up: "So how was real-world behavior verified at all?" Manually, with the actual deployed app and a real webcam (including via a phone used as a webcam), plus the real 2,258-sample dataset for the ML evaluation.

Q50. What does the CI pipeline actually run, exactly?

Two GitHub Actions jobs on every push/PR to `main`: backend runs `ruff check`, `ruff format --check`, and `pytest`; frontend runs `oxlint`, a TypeScript type-check, `vitest run`, and `npm run build`.

Key points: lint, format, type-check, test, and build, both halves, every time.

Follow-up: "Does CI test the Docker image?" No, that is a documented gap; the Dockerfile is currently validated only by Render's own build, listed in Future Improvements.

Q51. Why split `requirements.txt` and `requirements-dev.txt`?

So the production Docker image installs only what the running app needs (FastAPI, MediaPipe, scikit-learn, and so on), not pytest or Ruff, keeping the image smaller and closer to what actually ships.

Key points: runtime versus development dependencies, kept explicitly separate.

Follow-up: "Does CI use both files?" Yes, CI installs `requirements-dev.txt` (which includes the runtime file) since it needs the test tools too; the Dockerfile installs only `requirements.txt`.

Q52. Why does the Dockerfile create a non-root user?

Standard container security hardening: if the application process were somehow compromised, running as an unprivileged user limits what it could do inside (and, to a lesser extent, outside) the container compared to running as root.

Key points: defense in depth, not a functional requirement of the app itself.

Follow-up: "Did this require any other changes?" Yes, file ownership had to be set correctly (`chown`) before switching users so the app can still read its own files.

Q53. What does the Docker `HEALTHCHECK` actually do?

Runs a small Python one-liner inside the container every 30 seconds that calls `/health` and fails if it does not get a response, so Docker (and, by extension, anything orchestrating the container) can tell the difference between "process is running" and "process is actually serving requests".

Key points: an in-container liveness probe, complementary to Render's own external health check.

Follow-up: "Would this catch a hung MediaPipe process?" Only if it also blocks the event loop enough to prevent `/health` from responding; MediaPipe runs in a worker thread specifically so it does not block that loop.

Q54. Why is MediaPipe processing done in a worker thread rather than directly in the async handler?

MediaPipe's `process()` call is synchronous and CPU-bound; calling it directly inside an `async def` handler would block FastAPI's whole event loop for that duration, stalling every other connection the server is handling. `run_in_threadpool` moves that blocking work off the event loop.

Key points: keeps the server responsive to many connections while one frame is being processed.

Follow-up: "Why not use multiprocessing instead of threads?" Threads are lighter weight for this; the actual CPU-heavy work happens inside MediaPipe's own C++ implementation, which releases the Python GIL during that work, so threads are an effective, simple choice here.

Q55. Why does each WebSocket connection get its own `GesturePipeline` instead of sharing one?

Both MediaPipe's internal hand-tracking state and the smoothing buffer are stateful and specific to one continuous stream; sharing them between different users' connections would mix up tracking and smoothing across unrelated video feeds.

Key points: per-connection state, deliberately not shared, for correctness with multiple simultaneous users.

Follow-up: "What is the memory/CPU cost of that?" Each connection carries its own MediaPipe graph; this is a real scalability limit for very high concurrent user counts, discussed further in Section 19.

Q56. How does the "latest frame wins" design actually get implemented in code?

The receive loop always overwrites a single `latest_frame` variable on the connection's shared state rather than appending to a list/queue; the process loop, whenever woken, reads and clears that single slot, so there is structurally never more than one frame waiting.

Key points: a one-slot buffer, not a queue, is the actual mechanism, deliberately simple.

Follow-up: "Could this ever lose a frame the user cares about?" Yes, by design, under load an intermediate frame can be skipped entirely; the trade-off accepted is that freshness matters more than seeing every single frame.

Q57. Why does normalization translate and scale but deliberately not rotate the landmarks?

Some gestures are defined by real-world orientation, thumbs-up needs the thumb actually pointing up in the image, pointing-up needs the index finger pointing up; rotating every hand to a canonical upright pose would erase exactly that signal.

Key points: a deliberate trade-off, documented in `docs/decisions.md`, not an oversight.

Follow-up: "Does this hurt rotation tolerance?" Yes, partly, this is precisely why extreme rotation is a documented limitation and why the fist/thumbs-up bug happened in the first place.

Q58. Why does the project use both REST and WebSocket instead of just one?

The video stream is a continuous, bidirectional problem, a natural WebSocket use case; but `/health`, `/gestures`, and `/model` are simple, stateless, one-off lookups, which REST fits more simply and which also lets FastAPI auto-generate interactive docs for them.

Key points: the right tool for each specific job, not a single protocol used everywhere out of habit.

Follow-up: "Could `/gestures` be delivered over the WebSocket instead?" Technically yes, but that would couple a one-off lookup to needing an open connection, with no benefit.

Q59. Why TypeScript instead of plain JavaScript for the frontend?

The WebSocket message contract between frontend and backend needs to match exactly; TypeScript catches a typo'd field name or wrong type at compile time, and gives better editor autocomplete across a codebase this size.

Key points: catches integration bugs earlier; documented in `docs/decisions.md`.

Follow-up: "Is the type safety enforced at the API boundary, or just assumed?" Assumed by convention today, `types.ts` is hand-kept in sync with `schemas.py`; a stronger setup would generate one from the other.

Q60. Why Tailwind CSS instead of hand-written CSS files?

Utility classes keep spacing, color, and typography consistent across many components without maintaining separate stylesheets, and made building the light/dark theme and the responsive layout faster.

Key points: consistency and speed of development, at the cost of more verbose class lists in the markup.

Follow-up: "Any downside you noticed?" Class lists can get long on complex components; that is the standard, accepted trade-off of the utility-class approach.

Q61. Why did the project reject using a database?

Nothing in the app actually needs to persist beyond one browser session, gesture history and stats are explicitly session-only; adding a database would add real operational complexity (hosting, migrations, backups) for no feature the app currently offers.

Key points: explicitly listed as a rejected technology in [docs/decisions.md](#), a deliberate scope decision.

Follow-up: "When would you add one?" If user accounts and cross-session history became a real requirement, listed as a Future Improvement.

Q62. Why did the project reject WebRTC?

WebRTC is built for peer-to-peer real-time media and needs signaling, STUN/TURN servers, and SDP negotiation, far more infrastructure than "browser sends compressed frames to my own server" requires; a plain WebSocket is simpler and sufficient here.

Key points: WebRTC solves a harder problem (peer-to-peer) than this project has (client-to-my-own-server).

Follow-up: "Would WebRTC ever make sense for this app?" If the vision processing needed to run peer-to-peer between two users' browsers directly, not through a central server; not the case here.

Q63. Why exactly these six gestures?

A mix of gestures that are visually and geometrically distinct enough to be reliably separable from 21 landmarks (open palm, fist, thumbs up, victory, pointing up, OK sign), while being small enough to record a solid real dataset for in a reasonable amount of time.

Key points: chosen for geometric distinctiveness and feasibility, not arbitrarily.

Follow-up: "Which two are closest to each other geometrically?" Open palm and OK sign, both have most fingers extended; the confusion matrix (Section 7.2) shows this is exactly where the model's remaining errors mostly occur.

Q64. What is the difference between the frontend's and backend's test suites in what they can verify?

Backend tests can exercise the real vision math, the real classifiers, and the real WebSocket protocol logic in-process; frontend tests can exercise real component rendering and hook behavior against a mocked WebSocket/camera, but neither suite drives an actual browser against the actual deployed backend end to end.

Key points: strong unit/integration coverage on each side; true end-to-end (real browser, real deployed backend) verification was done manually.

Follow-up: "Would you add real end-to-end tests?" Listed conceptually under Future Improvements as automated visual/regression testing in CI.

Q65. Why is the ML model file committed to Git when datasets and other models are not?

It is small (about 4 KB), it is the real, trained-on-real-data model actually used in production, and committing it means a fresh clone or a fresh deploy has a working ML classifier immediately, without requiring anyone to run training first; the raw dataset and any experimental model files remain git-ignored since they are large, personal, or

not what ships.

Key points: a deliberate exception, justified by size and by being the genuine production artifact.

Follow-up: "Is this good practice generally?" For a small, stable model in a small project, yes; for a large or frequently retrained model, a model registry or artifact store would be more appropriate.

Q66. Why did the project reject Next.js in favor of plain Vite plus React Router?

Next.js's main strengths, server-side rendering and API routes, are not needed here: the app is a client-side SPA whose real backend is a separate Python service, not a Node API layer; Vite plus React Router gives a simpler, purpose-fitting setup.

Key points: explicitly listed as rejected in `docs/decisions.md`; matched tool to actual need.

Follow-up: "Would SSR ever help this app?" Unlikely, the app's content is inherently dynamic/client-driven (a live camera feed), so SSR's main benefit, faster first paint of static content, does not apply much here.

Q67. How were the classifier's numeric thresholds (like the 0.55 thumb-protrusion value) chosen?

Empirically, by computing that value across every real fist and thumbs-up sample in the actual dataset and picking a threshold in the gap between the two distributions (fist's 95th percentile around 0.53, thumbs-up's 5th percentile around 0.61), not guessed or tuned by trial and error on live video.

Key points: data-driven threshold selection, documented and reproducible.

Follow-up: "Is this threshold specific to this dataset, or general?" It is derived from this dataset; a much larger, more diverse dataset might justify revisiting it.

Q68. Is the project accessible (in the UI/UX sense)?

Reasonably: keyboard-operable settings, `aria-live` regions announcing the current gesture for screen readers, and respect for the OS "reduce motion" preference are implemented; it has not gone through a full formal accessibility audit.

Key points: real accessibility features present, described honestly as a partial effort, not a certified/audited one.

Follow-up: "What is the biggest accessibility gap?" The core feature itself, gesture recognition via webcam, is inherently visual/motor-dependent and is not an alternative-input-accessible interaction by nature.

Q69. Why oxlint for the frontend instead of the more common ESLint?

It is a fast, Rust-based linter that was the default from the Vite scaffolding tool used to start the frontend; it is faster than ESLint and covers the React/TypeScript rules this project needed.

Key points: a tooling choice inherited from the modern Vite scaffold, evaluated and kept because it worked well.

Follow-up: "Any limitation you hit?" A false-positive-prone rule (React Fast Refresh export shape) was worked around with a scoped override for test files rather than disabled project-wide.

Q70. What would you check first if the deployed frontend showed a blank page?

Open the browser console for a JavaScript error first (often a missing/incorrect environment variable causing a crash on load), then check the Network tab to see if the build's JS/CSS files actually loaded (a routing/rewrite misconfiguration would 404 them).

Key points: console first, then network; matches Section 22's structured debugging approach.

Follow-up: "Would this project's own `config.ts` guard help here?" Partially, it logs a clear console error for a missing API/WS URL specifically, narrowing that one common cause quickly.

Q71. How would you roll back a bad deployment?

On Vercel, redeploy a previous successful build from its dashboard history; on Render, redeploy a previous commit, or simply `git revert` the bad commit and push, which triggers a fresh, correct build on both platforms automatically.

Key points: both platforms keep deploy history; Git itself is also a safety net via revert.

Follow-up: "Would CI have caught the bad change first?" If it was a lint/type/test failure, yes; CI does not catch every possible production issue (for example, a misconfigured environment variable), which is why manual post-deploy verification is also part of the process (Section 11, the post-deploy checklist).

Q72. Is there any production monitoring beyond the health check?

Not determined from the repository beyond what is described here: the `/health` endpoint, Render's own platform-level health checks and logs, and the Docker `HEALTHCHECK`. There is no separate application-level monitoring/alerting or log aggregation service configured in the project.

Key points: honest gap; be direct that this is basic-tier monitoring, not a full observability stack.

Follow-up: "What would you add first?" Structured logging plus a basic uptime/alerting service (see Section 19 for the fuller answer on observability).

Q73. Why does the README include real, verified metrics instead of just a general claim of "high accuracy"?

Specific, sourced numbers (95.5% accuracy, 2,258 samples, a session-held-out split) are checkable and defensible in an interview; a vague claim like "high accuracy" invites the very follow-up question ("how high, measured how?") that specific numbers already answer.

Key points: precision builds credibility; every number in the README traces back to `ml/reports/comparison.json`.

Follow-up: "What would you do if a number in your docs and the code ever disagreed?" Treat the code and its live output as the source of truth and correct the documentation, exactly the verification approach used to prepare this guide.

Q74. Why MIT license for this project?

A short, permissive, widely understood license, appropriate for a portfolio project meant to be freely readable, forkable, and reused as a learning reference, with minimal restriction.

Key points: standard choice for open portfolio work; not a commercial product needing a more restrictive license.

Follow-up: "What does MIT actually permit?" Essentially anything, including commercial use and modification, as long as the original copyright notice is kept.

Q75. If you had to demo this project live with no internet, what would you show?

Run it fully locally: `uvicorn app.main:app --reload` in one terminal and `npm run dev` in another, open `localhost:5173`, and walk through Live, Dashboard, Guide, and Model exactly as in production, since the whole app runs identically locally, only the URLs differ.

Key points: the local setup is a complete, faithful copy of the production app, not a stripped-down version.

Follow-up: "What is different locally versus in production?" Only configuration, no HTTPS is needed on `localhost`, and the classifier defaults to rule-based unless `GESTUREFLOW_CLASSIFIER=ml` is set.

19. Senior Recruiter / Top-Company Level Questions (50)

Format: Strong answer, Why it is asked, Likely follow-up, What a weak answer sounds like.

Q1. Your backend creates a new MediaPipe instance per WebSocket connection. How does that scale to 100 concurrent users?

Strong answer: It does not scale indefinitely on a single instance; each connection holds its own MediaPipe graph and thread, so memory and CPU grow roughly linearly with concurrent users, on a free-tier single container this would degrade well before 100 real-time users. The real fix is horizontal scaling, multiple backend instances behind a load balancer that supports sticky WebSocket routing, which the stateless-per-connection design (no shared server state) makes straightforward to add.

Why asked: tests whether the design was reasoned about beyond "it works on my machine".

Follow-up: "What would you measure before assuming it is a problem?" CPU/memory per connection under load testing, not guessing.

Weak answer: "It should be fine, cloud servers are powerful," with no numbers or reasoning.

Q2. Walk me through the actual latency budget from webcam to displayed result.

Strong answer: Roughly: 0-100ms frame capture interval, a few ms to encode/downscale, network upload of a small JPEG, backend decode plus MediaPipe (the dominant cost, observed low tens of milliseconds), classification (sub-millisecond, it is 25 numbers through a linear model), network back, then a canvas redraw. `inference_ms` in every result message is the measured server-side number, giving a real, not estimated, figure.

Why asked: separates candidates who reasoned about where time actually goes from those who only know "it feels fast".

Follow-up: "Which stage would you optimize first if latency became a problem?" Network round-trip time to the user's specific region, since MediaPipe itself is already fairly fast on the lite model setting.

Weak answer: "It's real-time so it's fast," with no breakdown.

Q3. What is your actual data-leakage prevention, and how would you prove it worked?

Strong answer: Session-grouped splitting (`GroupShuffleSplit` on session id), so no sample from the test session's recording ever appears in training. Proof: the reported metrics come specifically from that held-out session, and the cross-validation-versus-held-out gap for Random Forest and SVM is itself evidence the split is doing real work, if it were leaking, that gap would likely be smaller, not larger.

Why asked: leakage is the single most common way ML evaluation numbers are quietly wrong; this checks if the candidate actually understands why, not just that a split exists.

Follow-up: "Could there still be leakage you have not accounted for?" Possibly subtle ones, like both sessions sharing the same physical camera/room, which is a generalization limitation, distinct from frame-level leakage, and it is disclosed as such.

Weak answer: "I used `train_test_split`," without addressing why a random split would be wrong here.

Q4. Your model gets 95.5% on your own held-out data. Why should I trust that number generalizes at all?

Strong answer: You should trust it as evidence the pipeline and evaluation methodology are sound, not as a universal accuracy claim; it is explicitly one person, two sessions. The honest claim is "a properly evaluated, leakage-safe 95.5% on unseen-session data from this setup," and the documented next step to strengthen the claim is exactly what is missing: more contributors.

Why asked: to see if the candidate overclaims or is precise about what a metric actually proves.

Follow-up: "What is the minimum additional data you would want before trusting this in a real product?" Multiple people, multiple cameras/lighting setups, ideally dozens of sessions, evaluated the same leakage-safe way.

Weak answer: "95.5% is really good so it will work for everyone."

Q5. Why Logistic Regression and not a small neural network, given you already have the data pipeline?

Strong answer: With 2,258 samples and 25 already-informative engineered features, a linear-ish model is close to the right capacity for the problem; a neural network adds many more parameters with no architectural advantage here (no spatial/sequential structure the features do not already expose) and a real overfitting risk at this data size. If data grew to tens of thousands of samples and multiple people, revisiting model capacity would be reasonable.

Why asked: checks whether "more advanced" is being confused with "better", a common junior mistake.

Follow-up: "What would change your mind?" Evidence a linear-ish decision boundary is actually insufficient, which the SVM/RF results here do not show, they underperformed, not the opposite.

Weak answer: "Neural networks are more powerful so they would be better."

Q6. Your WebSocket has no message-size limit mentioned. What happens if a client sends a huge payload?

Strong answer: Not explicitly capped in the current code beyond what Starlette/Uvicorn's own defaults allow; a hostile or buggy client sending an oversized frame is a real, currently un-hardened gap. A production hardening pass would add an explicit max-message-size check and reject/close on violation, plus rate-limiting per connection.

Why asked: probes for security-mindedness and whether the candidate can identify gaps in their own code rather than only defending it.

Follow-up: "How would you add that without breaking legitimate large-ish frames?" Set the cap comfortably above the largest expected downscaled JPEG (well under 100KB) with margin, and log/monitor rejections to catch a misbehaving client.

Weak answer: claiming this is already handled when it is not, rather than acknowledging the gap.

Q7. Is storing landmark data from a webcam a privacy concern?

Strong answer: Lower risk than storing images or video, 21 sparse points carry no reconstructable photographic detail, but they are still biometric-adjacent data (hand geometry) tied to a person; the project mitigates this by never storing raw video/images at any stage, keeping the dataset local and out of the deployed service, and the deployed app itself does not persist any user's session data server-side at all.

Why asked: tests awareness that "not an image" does not automatically mean "not sensitive".

Follow-up: "Would you need consent/disclosure for a real product?" Yes, a real product handling any biometric-adjacent data would need explicit user disclosure and likely regulatory consideration depending on jurisdiction.

Weak answer: "It's just numbers, not a photo, so there's no privacy issue."

Q8. Walk me through what happens, precisely, if two users open the Live page at the same moment.

Strong answer: Two independent WebSocket connections, each gets its own `GesturePipeline` (own `MediaPipe` instance, own smoothing buffer), handled by `asyncio` concurrently on the event loop, with the actual `MediaPipe` work offloaded to the thread pool so neither user blocks the other's frame processing.

Why asked: checks real understanding of the concurrency model, not just "it's async so it's fine".

Follow-up: "What is shared between them?" Nothing application-level; the only shared resource is the underlying process's CPU/memory and the thread pool itself.

Weak answer: "FastAPI handles it automatically," without explaining the per-connection pipeline or the thread offload.

Q9. If this needed to handle 10x the traffic tomorrow, what is your actual scaling plan?

Strong answer: Vertically, a bigger `Render` instance first (cheapest, fastest); horizontally, multiple backend instances behind a load balancer with WebSocket-aware (sticky) routing, since each connection's state lives in that one process; separately, consider a lighter `MediaPipe` model complexity setting or lowering the default frame rate under load, and add real load testing before assuming any of this is sufficient.

Why asked: distinguishes "it would probably just scale" from an actual plan with real trade-offs named.

Follow-up: "What would you measure to know when to scale?" CPU/memory per connection, p95 `inference_ms`, and `frames_dropped` trends under real load.

Weak answer: "Just add more servers," with no mention of WebSocket session affinity or what to measure first.

Q10. Your model is 4KB and committed to git. What is your model-versioning strategy as the project evolves?

Strong answer: Today, versioning is implicit, one file, one Git history, guarded by a `FEATURE_VERSION` compatibility check so an incompatible model cannot silently corrupt predictions. For a larger project this is not enough, a model registry (tracking which dataset, code commit, and metrics produced each model) would be the real next step; the current setup is honestly sized to the project's current scale, not presented as production-grade MLOps.

Why asked: tests whether the candidate can name the gap between "works for a portfolio project" and "production ML practice".

Follow-up: "What breaks first if you don't add that?" Losing track of exactly which dataset/commit produced a deployed model once there are several retraining iterations; `FEATURE_VERSION` only catches one specific kind of mismatch, not a full provenance story.

Weak answer: "Git tracks it, so it's versioned," without engaging with what a model registry would add.

Q11. What is your actual test coverage for the WebSocket concurrency logic specifically?

Strong answer: `test_ws.py` covers behavior per connection thoroughly, ready message, streaming, bad frames, config/reset, ML fallback, origin checks, all through FastAPI's `TestClient`, which runs in-process, not as separate real network connections. It does not include a test that opens many simultaneous real WebSocket connections and measures behavior under that concurrency; that would need a separate load-testing setup.

Why asked: separates "tests exist" from "tests exist for the specific thing being asked about".

Follow-up: "How would you add that?" A load-testing tool (for example, a script opening N real WebSocket clients against a running instance) measuring latency and error rate as N grows.

Weak answer: "Yes, it's fully tested," pointing at the 93 tests without distinguishing unit/integration coverage from concurrency/load coverage.

Q12. Your feature engineering was done by hand. How do you know these 25 features are the right ones, not just ones that happened to work?

Strong answer: They were designed to mirror the same geometric reasoning the rule-based classifier already used (finger extension, joint angles, tip distances), which is domain knowledge, not arbitrary; and the actual comparison against a most-frequent baseline and against a genuinely held-out session is the evidence they work, not just intuition. There was no formal feature-importance or feature-selection study done beyond that.

Why asked: checks whether feature choices are grounded in reasoning and evidence or just "I tried some things".

Follow-up: "How would you find out if some of the 25 are actually useless?" Look at Logistic Regression's learned coefficients, or run a feature-importance/ablation study, removing features one at a time and re-evaluating on the held-out session.

Weak answer: "They just worked well," with no reasoning about why they were chosen in the first place.

Q13. Your Origin check trusts the browser's Origin header. Can that be spoofed?

Strong answer: A real browser cannot be scripted by arbitrary JavaScript on a page to forge the Origin header, it is set by the browser itself based on the page's actual origin, so this check is effective against unauthorized *web pages* connecting. It does not stop a non-browser client (a script, curl-equivalent) from simply omitting or setting any Origin it wants, since no-Origin is explicitly allowed and there is no additional authentication layer; the check's real purpose is browser-based CSRF-style protection, not a full authentication system.

Why asked: tests whether the candidate understands the actual security boundary of what they built, not just that "a check exists".

Follow-up: "Does that matter for this app?" The API has no secrets or user data behind it, so an unauthenticated non-browser client hitting `/ws` is a low-severity concern here, but the distinction is worth stating precisely rather than overclaiming security.

Weak answer: "The Origin check makes it fully secure."

Q14. What is your actual cost profile for this deployment?

Strong answer: Both Render and Vercel's free tiers are used, so the direct cost is \$0; the real cost is the free tier's own limitation, the backend's cold start after idle. Not determined from the repository: exact resource usage numbers (CPU/memory) since no load testing or platform-level metrics dashboard review was part of this project.

Why asked: production engineers are expected to know what something costs, not just that it works.

Follow-up: "What would the first paid tier buy you?" An always-on backend with no cold start, and higher CPU/memory limits for more concurrent users.

Weak answer: not knowing which tier is being used, or claiming "it's free forever" without the free-tier trade-off.

Q15. What is your observability story if something breaks in production right now?

Strong answer: Basic: Render's own platform logs and its health check, plus the app's own `/health` and `/model` endpoints for a manual check, and the frontend's visible connection-state UI. There is no structured application

logging, error tracking service, or metrics dashboard configured; diagnosing an issue today would mean reading Render's raw logs directly.

Why asked: production readiness is judged heavily on "can you find out what went wrong", not just "does it usually work".

Follow-up: "What would you add first?" Structured logging with request/connection IDs, and a basic error-tracking service, before anything more elaborate.

Weak answer: "I'd just check if it's working," with no concept of logs, metrics, or structured diagnosis.

Q16. Are your Python dependencies pinned, and why does that matter?

Strong answer: Yes, every package in `requirements.txt` and `requirements-dev.txt` is pinned to an exact version, so a fresh install (locally, in CI, or in the Docker build) always gets an identical set of dependencies, preventing "works on my machine, breaks in CI/production" from an upstream package silently changing behavior.

Why asked: a basic but frequently-skipped reproducibility practice; tests attention to detail.

Follow-up: "How do you know when it's safe to bump a version?" Not automated here (no Dependabot/renovate configured); currently a manual decision, a real gap for a longer-lived project.

Weak answer: not knowing whether versions are pinned or not.

Q17. Your API design mixes REST and WebSocket. Would you call that a good API design decision?

Strong answer: Yes, deliberately: REST for stateless, cacheable, one-off lookups (`/health`, `/gestures`, `/model`), which also get free interactive documentation from FastAPI; WebSocket specifically for the one genuinely continuous, bidirectional, stateful piece of functionality. Using one protocol for everything would either force awkward polling for the stream or an awkward one-shot-request feel for the video pipeline.

Why asked: tests API design judgment, not just protocol trivia.

Follow-up: "What would you add if this needed a public third-party API?" Versioning the REST routes (for example `/v1/...`) and authentication, neither of which exists today since there has been no external consumer.

Weak answer: "WebSocket for everything is simpler," missing why REST is still the better fit for the simple lookups.

Q18. What browser limitations does this app run into, beyond HTTPS for the camera?

Strong answer: `getUserMedia` and WebSocket support are both required and not universal on very old browsers; mobile browsers can behave differently for camera constraints and autoplay of the video element; and the app has not been formally tested across every browser/device combination, cross-browser QA is not claimed as exhaustive.

Why asked: checks whether "it works in Chrome on my laptop" is being mistaken for "it works everywhere".

Follow-up: "How would you find real compatibility issues?" Manual testing across a defined browser/device matrix, or an automated cross-browser testing service, neither currently in place.

Weak answer: "It's just HTML5 APIs so it works everywhere."

Q19. If a customer complained the app "feels laggy", how would you actually diagnose it, not guess?

Strong answer: Check the Session Stats panel first, live data: server latency (`inference_ms`), throughput, and dropped frames; a high dropped-frame count points to the network or client keeping up, a high `inference_ms`

points to the backend (possibly a cold-started free-tier instance); then check whether the target fps in Settings is set higher than the connection can sustain.

Why asked: distinguishes candidates who reach for the app's own built-in diagnostics from those who would only guess.

Follow-up: "What if server latency is low but it still feels laggy?" Suspect network round-trip time or client-side rendering, check the browser's own network tab for WebSocket frame timing.

Weak answer: "I'd just restart the server," without using the data the app already exposes.

Q20. Your CI does not build the Docker image. Isn't that a real gap?

Strong answer: Yes, honestly, it is a documented gap, listed explicitly in Future Improvements. Today the Dockerfile is only validated by Render's own build step at deploy time, meaning a Docker-specific breakage (a bad system dependency, a broken `COPY` path) would only be caught during an actual deploy, not before merging to `main`.

Why asked: tests whether the candidate defends every choice or can honestly identify a real weakness in their own project.

Follow-up: "How would you add it?" A CI job running `docker build` (and ideally a smoke test hitting `/health` inside the built container) on every push, before Render ever sees the change.

Weak answer: claiming CI already covers this when it does not.

Q21. Why is the classifier decision (rule vs ML) made per-connection instead of a single global setting?

Strong answer: A per-connection default (from `GESTUREFLOW_CLASSIFIER`) that can also be switched live per connection gives every user independent control from the Settings panel, which is a better user experience and also directly enables the Model comparison page's premise, letting one user compare both engines without affecting anyone else's session.

Why asked: checks whether a design choice was made deliberately for a reason, versus being incidental.

Follow-up: "Is there a cost to that flexibility?" A very small one: building an `MLClassifier` instance is not shared across connections, though the model file itself is tiny (about 4KB), so this cost is negligible in practice.

Weak answer: "It just works that way," without connecting it to a real design reason.

Q22. What is the single biggest piece of technical debt in this project right now?

Strong answer: Most honestly, the dataset: 2,258 samples from one person across two sessions is enough to build and evaluate a correct pipeline, but is the real ceiling on any generalization claim; everything downstream (model choice, thresholds) is only as good as that data allows. Close behind: no Docker build in CI, and no production monitoring/alerting beyond a health check.

Why asked: senior engineers are expected to know their own project's real weak points, not just its strengths.

Follow-up: "If you had one week, which would you fix first?" The Docker CI gap, since it is fast to add and directly de-risks every future deploy; the dataset limitation needs ongoing, longer-term data collection.

Weak answer: "There isn't really any debt, it's all pretty clean," an implausible claim for any real project.

Q23. How would you add rate limiting to protect the backend from abuse?

Strong answer: None exists today, a real gap for a public endpoint. I would add a per-IP or per-connection cap on WebSocket connection attempts and REST requests (for example, a token-bucket limiter middleware in FastAPI, or at the platform/reverse-proxy level on Render), plus the message-size cap discussed in Q6, before treating the API as hardened against abuse.

Why asked: checks awareness that "it works for legitimate users" is different from "it is hardened against misuse".

Follow-up: "What would you rate-limit differently for `/model` versus `/ws`?" `/model` is a cheap, cacheable read; `/ws` triggers real CPU work (a MediaPipe instance) per connection, so it deserves a stricter limit.

Weak answer: "Render probably handles that," without knowing whether or how.

Q24. Explain a trade-off in this project where you deliberately chose simplicity over a "more correct" solution.

Strong answer: Keeping `frontend/src/types.ts` hand-written rather than generating it from the backend's OpenAPI schema: the more correct, drift-proof solution is codegen, but for a project this size, manually keeping a small, stable set of types in sync was simpler to set up and understand, at the accepted risk of them silently drifting if the project grew.

Why asked: senior interviewers want evidence of conscious trade-off reasoning, not an assumption that more automation is always better.

Follow-up: "At what project size would you switch?" Once the API surface or the team grew enough that manual synchronization started causing real bugs, a concrete, not arbitrary, trigger.

Weak answer: unable to name any deliberate trade-off, implying every choice was simply "the right way to do it".

Q25. What happens to model accuracy over time in production, and how would you detect drift?

Strong answer: Nothing currently monitors this; the model's accuracy is a fixed, offline evaluation number, there is no mechanism collecting live production predictions with ground truth to detect if real-world accuracy degrades (for example, if usage patterns or cameras shift over time). This is a real production-ML gap in the current setup, not something to claim is handled.

Why asked: separates "trained a model once" from understanding production ML lifecycle.

Follow-up: "How would you even get ground truth in production to detect drift?" Optional in-app feedback (was this gesture correct?), which the current UI does not collect, would be the first step.

Weak answer: "The model doesn't change so it can't drift," conflating the model being static with real-world performance being static.

Q26. Why fit the `StandardScaler` only on the training data, and what breaks if you scale before splitting?

Strong answer: The pipeline fits the scaler on training data only, inside the same `fit()` call as the classifier, via a scikit-learn `Pipeline`; if you instead scaled the whole dataset before splitting, the scaler's mean/variance would be computed using information from the test set, a subtle form of data leakage that inflates the reported score.

Why asked: a classic, specific ML-pipeline-correctness gotcha; tests real hands-on scikit-learn understanding, not just theory.

Follow-up: "Does the saved model include the fitted scaler?" Yes, the whole pipeline, scaler and classifier together, is what gets saved and loaded, so inference applies the exact same transformation training used.

Weak answer: not knowing scaling before splitting is a leakage risk at all.

Q27. How would this app behave if the user wore a glove, or had an unusual hand shape MediaPipe was not trained well on?

Strong answer: Not determined from the repository through direct testing; MediaPipe's own accuracy on non-typical hand appearances is outside this project's control, and if landmark detection itself degrades, everything downstream (normalization, classification) inherits that error, since the classifiers only ever see whatever 21 points MediaPipe reports.

Why asked: tests whether the candidate understands failure can originate upstream of their own code, not just within it.

Follow-up: "Would you add a confidence check on the landmark detection itself?" MediaPipe does return its own hand-presence confidence, which is not currently surfaced to the user; that would be a reasonable addition.

Weak answer: assuming it would work fine with no basis for that claim.

Q28. Is there any risk of overfitting the classifier's thresholds to your specific test data, given how they were tuned?

Strong answer: Yes, a real risk worth naming: the 0.55 thumb-protrusion threshold, and the rule-based classifier's other thresholds, were derived by looking directly at the same dataset later used for evaluation, which is a milder, rule-based analog of tuning on the test set. This is disclosed rather than hidden; a stronger process would tune thresholds on a separate validation slice, distinct from the final reported test session.

Why asked: tests self-awareness of methodology weaknesses, a hallmark of genuine understanding versus rote reporting.

Follow-up: "Does this affect the ML model the same way?" Less directly, the ML model's parameters are learned automatically from the training split only; the rule-based thresholds are the part hand-tuned by looking at data.

Weak answer: insisting there is no such risk at all.

Q29. Your app has zero backend persistence. What is the actual risk of that architecture?

Strong answer: The main risk is simply a feature limitation, not a reliability one: no cross-session history, no analytics over time, nothing to recover if a browser tab closes. It is not a data-loss risk in the traditional sense since nothing was ever meant to be durable; it was a deliberate scope decision (see Section 18, Q61) given the project has no accounts or multi-device requirement.

Why asked: checks whether "no database" is understood as a deliberate, bounded trade-off or an oversight.

Follow-up: "What is the smallest change that would add persistence?" Store session summaries against a generated anonymous session id in a lightweight database, without necessarily adding full user accounts.

Weak answer: not being able to explain why there is no database at all.

Q30. How maintainable is this codebase for someone who is not you?

Strong answer: Reasonably: the vision pipeline is split into small, single-purpose modules (decode, landmarks, normalize, features, classify, smooth), each independently tested; [docs/decisions.md](#) explains the non-obvious "why" behind choices, which is exactly the kind of context that is otherwise lost. The main onboarding friction would be the ML side, understanding the feature engineering and the session-split evaluation requires reading Section 6/7 of a guide like this one, or the equivalent code comments, not just skimming file names.

Why asked: senior engineers care about codebases outliving their original author.

Follow-up: "What is the first thing you would ask a new contributor to read?" [docs/decisions.md](#), since it front-loads the reasoning that is not visible from the code alone.

Weak answer: "I know it well so it's fine," ignoring the question of a different reader's experience.

Q31. Give me a back-of-envelope estimate: how many concurrent users could your current single Render instance realistically serve?

Strong answer: Not measured directly (a real gap, worth admitting), but reasoning through it: each connection's dominant cost is a MediaPipe "lite" model inference roughly every 100ms, observed around 15-30ms of server compute per frame in testing; on a small free-tier instance (roughly one shared vCPU), that suggests single-digit to low-double-digit concurrent active users before latency visibly degrades, without hard load-test data to confirm the exact number.

Why asked: forces reasoning under uncertainty with real numbers, exactly what a senior engineer does before scaling decisions, rather than either refusing to estimate or making up a confident-sounding wrong number.

Follow-up: "How would you get a real number instead of an estimate?" A load-testing script opening N concurrent real WebSocket clients sending frames, ramping N until latency or error rate crosses a threshold.

Weak answer: either "thousands, it's cloud-hosted" (no basis) or refusing to attempt an estimate at all.

Q32. Your reconnect logic uses exponential backoff. Is that actually the right choice here?

Strong answer: Yes, for the free-tier cold-start scenario specifically, backoff avoids hammering a container that is still starting up with rapid-fire reconnect attempts; the trade-off is a slower recovery on a truly transient blip (a one-second network hiccup) since the first retry still waits a base delay. Capped exponential backoff (which this implementation uses, capping at a maximum delay) balances both cases reasonably.

Why asked: checks the candidate is not just naming a pattern ("I used exponential backoff") without understanding why it fits this specific scenario.

Follow-up: "Would you ever want an immediate first retry?" Possibly, a common pattern is one immediate retry, then backoff from there, which this implementation does not currently do.

Weak answer: "It's a standard pattern," with no reasoning about this app's specific failure mode (cold starts).

Q33. Convince me, in 30 seconds, that this project is actually production-ready, or tell me honestly that it is not.

Strong answer: Honestly, it is production-ready for its actual scope, a demonstrated, deployed, tested single-service app with real users able to use it today, but it is not production-ready in the enterprise sense: no monitoring/alerting, no load-tested concurrency ceiling, no rate limiting, and a model trained on one person's data. Being able to state precisely where that line is, rather than defaulting to "yes" or "no", is the actually production-relevant skill.

Why asked: this is the closing "can you self-assess honestly under pressure" question senior interviewers often use.

Follow-up: "What is the smallest change that would move it meaningfully closer?" Adding structured logging plus basic error alerting, the cheapest, highest-leverage gap to close first.

Weak answer: an unqualified "yes, it's fully production ready" with no caveats, or an unqualified "no" with no acknowledgment of what does already work well.

Q34. Does `localStorage` for settings introduce any real risk?

Strong answer: Low risk: only non-sensitive UI preferences (theme, mirror, target fps, sensitivity, engine choice) are stored there, nothing that could be misused if read by another script on the same origin; every read/write is wrapped in a try/catch since `localStorage` can throw or be unavailable in some browser privacy modes, and the app degrades gracefully (falls back to defaults) rather than crashing if so.

Why asked: tests whether "just settings" is dismissed without checking, versus actually verifying nothing sensitive lives there.

Follow-up: "Would this be a problem if you added authentication later?" Yes, a token would need a different, more careful storage strategy (not casually alongside UI prefs); today there is no authentication at all, so this does not apply yet.

Weak answer: not knowing what is actually stored in `localStorage` or why it is safe.

Q35. If I told you the deployed backend needed to support authenticated users tomorrow, what would you change first?

Strong answer: Add an auth layer (JWT or session-based) in front of both the REST routes and the WebSocket handshake, since currently anyone who reaches the deployed URL can use every endpoint; then decide what, if anything, needs to become per-user (likely gesture history, moving it from browser-only memory to a real backend store keyed by user), which would also be the point at which a database becomes genuinely necessary.

Why asked: tests the ability to reason about a real feature addition, not just describe the current state.

Follow-up: "How would you authenticate a WebSocket specifically?" A token passed at connection time (a query parameter or the first message after connect), validated before accepting or before treating the connection as fully live.

Weak answer: "Just add a login page," without addressing the WebSocket or the backend routes at all.

Q36. How would you actually reduce server-side inference time further if you needed to?

Strong answer: MediaPipe already runs at `model_complexity=0` (its fastest setting), so the next lever is reducing work per frame rather than per-model tuning: a smaller input resolution (currently about 320px), or skipping full re-detection when tracking is already locked onto a hand (MediaPipe's own tracking-versus-detection mode already does some of this). The classifier step itself, rule-based or Logistic Regression, is already sub-millisecond and not the bottleneck.

Why asked: checks the candidate knows where the actual time goes, not just "optimize the model", to avoid optimizing the wrong stage.

Follow-up: "Would a GPU help?" Possibly for the MediaPipe stage at larger scale, but adds real infrastructure cost/complexity disproportionate to this project's current needs.

Weak answer: "Use a faster programming language," ignoring that the actual hot path is already MediaPipe's compiled C++ code, not Python.

Q37. Are any of your dependencies scanned for known vulnerabilities?

Strong answer: Not determined from the repository, no Dependabot, `pip-audit`, `npm audit` automation, or similar is configured in CI today; dependencies are pinned for reproducibility (Q16) but pinning alone does not protect against a vulnerability discovered in an already-pinned version.

Why asked: pinning and vulnerability scanning are often conflated by juniors; this question separates the two cleanly.

Follow-up: "What would you add first?" GitHub's own free Dependabot alerts, the lowest-effort, highest-value first step.

Weak answer: "Pinned versions are secure," treating reproducibility as if it were the same thing as security.

Q38. Walk me through exactly what you would change to add a seventh gesture.

Strong answer: Add it to `app/gestures.py`'s `SUPPORTED_GESTURES` list (automatically propagates to `/gestures` and the frontend Guide page); write a new template and geometric rule in `classifier_rules.py` for the rule-based engine; record real samples for it with `record_fixtures.py` across at least two sessions; and retrain with `python -m ml.train`, which handles the rest (feature extraction, comparison, evaluation, saving) automatically since none of that code is gesture-count-specific.

Why asked: a concrete "can you actually extend your own system" check, better than abstract description.

Follow-up: "What is the riskiest part of that change?" Picking a seventh gesture geometrically distinct enough from the existing six that it does not quietly degrade the others' accuracy, worth checking via the confusion matrix after retraining.

Weak answer: vague, "I'd just add it to the code," without naming the specific files or the retraining step.

Q39. Is your test suite a healthy testing pyramid, or too heavy in one layer?

Strong answer: Mostly unit and component-level tests (fast, focused, the bulk of the 93+93), a reasonable amount of integration-style testing (`TestClient`-driven WebSocket and route tests), and effectively no true end-to-end tests (a real browser against a real deployed backend), which were instead done manually. That is a defensible shape for a project this size, but is worth naming precisely rather than claiming "full coverage".

Why asked: tests whether the candidate can critique their own test suite's shape, not just its count.

Follow-up: "Would you add Playwright/Cypress-style E2E tests next?" Yes, listed conceptually in Future Improvements as automated visual/regression testing.

Weak answer: "186 tests total, so it's very well tested," treating test count alone as the measure of quality.

Q40. What is the most interesting bug you found in this project, and what did it teach you?

Strong answer: The fist/thumbs-up confusion (Section 6.6): a closed fist's thumb is nearly as straight as an extended one, so a straightness-only rule could not tell them apart; the fix needed a genuinely different geometric signal (protrusion past the other fingertips), found by plotting the real dataset rather than guessing. It taught me to debug classification bugs by going back to real data and looking at actual distributions, not by tweaking thresholds until one example passes.

Why asked: a classic "tell me about a bug" interview question; tests depth of understanding of your own debugging process.

Follow-up: "How did you confirm the fix generalized, not just fixed the one case you noticed?" Re-ran the fix against all 384 real fist samples and all 398 real thumbs-up samples, and tested it under synthetic in-plane rotation, not just the one example that prompted the investigation.

Weak answer: describing a trivial typo/syntax bug instead of a real design/logic bug, missing the chance to show debugging depth.

Q41. Is there a licensing concern in bundling MediaPipe and scikit-learn into a project you distribute?

Strong answer: Both are permissively licensed (Apache 2.0 for MediaPipe, BSD-3-Clause for scikit-learn), compatible with this project's own MIT license and with commercial or portfolio use; no copyleft obligations are triggered. Not independently re-verified line-by-line for every transitive dependency as part of this project, a reasonable but not exhaustive check.

Why asked: senior engineers are expected to at least know the licensing category of their major dependencies, not just that "it's open source".

Follow-up: "Would this matter more for a commercial product?" Yes, at commercial scale a full dependency-license audit (often automated) would be standard practice.

Weak answer: "It's open source so there's no issue," treating all open-source licenses as equivalent.

Q42. Why not use gRPC instead of a JSON-over-WebSocket protocol?

Strong answer: gRPC's advantages, a compact binary format and strongly typed contracts via protobuf, matter more at high message volume or cross-service internal communication; here the messages are small (a downscaled JPEG, a short JSON result) and the client is a browser, where gRPC-Web adds real complexity (a proxy layer) for limited benefit at this scale. Plain WebSocket plus JSON keeps the browser-side implementation simple and debuggable directly in devtools.

Why asked: tests whether "more modern/enterprise" technology choices are understood as trade-offs, not defaults.

Follow-up: "At what point would gRPC start to make sense?" If this became a multi-service backend with heavy internal service-to-service traffic, a different problem than a single backend talking to a browser.

Weak answer: not having considered the alternative at all, or claiming gRPC is "always better".

Q43. How do you know your 25 features are not redundant with each other?

Strong answer: Not formally verified with a correlation/multicollinearity analysis; some are very likely correlated by construction (for example, a finger's extension ratio and its curl angle both respond to the same underlying finger position). Logistic Regression tolerates correlated features reasonably well for prediction accuracy, though it does make individual coefficients harder to interpret in isolation; this was accepted rather than addressed with formal feature selection.

Why asked: a specific, technically precise ML-methodology question that filters for genuine hands-on experience.

Follow-up: "Would removing redundant features change the result?" Possibly simplify the model with little accuracy change; worth testing with an ablation study, not currently done.

Weak answer: claiming the 25 features were carefully verified to be independent when they were not.

Q44. If Render's Docker build silently used a different Python patch version than your local environment, how would you notice?

Strong answer: The Dockerfile pins `python:3.12-slim` as the base image, so the major/minor version is fixed explicitly; a patch-level drift within 3.12.x could still happen since the base image tag is not pinned to an exact patch digest. In practice, pinned package versions plus the passing CI/deploy build are the current signal; an explicit digest pin would remove this residual uncertainty entirely.

Why asked: tests attention to a subtle but real reproducibility gap between "pinned dependencies" and "pinned everything".

Follow-up: "Would you pin the exact digest in a stricter setup?" Yes, for maximum reproducibility, at the cost of needing to manually bump it for security patches instead of getting them automatically.

Weak answer: assuming the Dockerfile guarantees byte-identical environments with no residual uncertainty at all.

Q45. What would you do differently if you rebuilt this project from scratch today?

Strong answer: Collect the real dataset earlier, from the start, instead of building and shipping the rule-based classifier first and the ML model later, since the real data ended up informing both the ML model and, ultimately, a bug fix to the rule-based engine itself; earlier real data would have caught the fist/thumbs-up issue sooner. I would also set up the Docker CI build from day one rather than as a documented gap.

Why asked: tests genuine retrospective thinking, a strong signal of engineering maturity, versus a rehearsed "no regrets" answer.

Follow-up: "Would you still build the rule-based engine at all, in hindsight?" Yes, it earned its place as the zero-dependency fallback and as an interpretable baseline, that decision held up.

Weak answer: "I wouldn't change anything," an unconvincing answer for any real project.

Q46. Is a 4KB model file really "machine learning" worth talking about in an interview, or is it too simple?

Strong answer: Model complexity is not what makes ML work legitimate; a correctly-collected dataset, a leakage-safe evaluation, a principled comparison against baselines, and an honest interpretation of the results are what matter, and this project has all four. A small, fast, well-justified model that clearly beats its baselines is a stronger result than a large model used carelessly.

Why asked: tests whether the candidate conflates "impressive-sounding" with "well-engineered".

Follow-up: "What would make you more confident recommending a bigger model?" Meaningfully more and more diverse data, and evidence (like an ablation study) that the current feature set is actually the bottleneck, not model capacity.

Weak answer: apologizing for the model being "too simple" without defending why it was the right choice.

Q47. How would you defend the choice to compute per-class metrics rather than only an overall accuracy number?

Strong answer: Overall accuracy hides which specific gestures are weak; per-class precision/recall (Section 7) shows, precisely, that `open_palm` and `ok_sign` are where the model's remaining confusion concentrates, actionable information a single accuracy number cannot provide.

Why asked: reinforces whether the candidate actually uses their own detailed metrics, or just quotes the headline number.

Follow-up: "What would you do with that specific insight?" Consider adding features that better separate an open palm from an OK sign specifically, for example something capturing thumb-to-index proximity more directly.

Weak answer: only ever citing the 95.5% headline number without engaging with the confusion matrix at all.

Q48. What is a scenario where your "latest frame wins" design would actually be the wrong choice?

Strong answer: If completeness mattered more than freshness, for example, if the goal were counting exact gesture occurrences for a precise offline record rather than showing a live result, dropping frames would lose real information. For this app's actual purpose, live, responsive feedback, freshness is unambiguously the right priority, but it is a real trade-off, not a universally correct pattern.

Why asked: tests whether a design choice is understood as context-dependent, not as a universally "correct" pattern to apply everywhere.

Follow-up: "Does the Dashboard's detection count suffer from this?" Slightly, it counts stable gesture changes, not raw frames, from the samples that survive smoothing, so it is a reasonable session summary, not a frame-perfect log.

Weak answer: insisting "latest frame wins" is simply the correct approach in every situation.

Q49. If I gave you write access to this repository right now and 2 hours, what would you actually fix first?

Strong answer: Add the Docker build to CI (Q20), the cheapest, fastest, highest-confidence improvement that directly closes a documented gap without touching application behavior at all, low risk, clear value, and testable within that time budget.

Why asked: a practical prioritization question; tests judgment about impact versus effort under a real constraint.

Follow-up: "What would you do with a full week instead?" Start real load testing to replace the Q31 estimate with actual numbers, since that unlocks an informed answer to nearly every scalability question in this section.

Weak answer: picking something unrelated or purely cosmetic (like a UI color tweak) over a substantive, already-identified gap.

Q50. In one sentence, what is the single strongest engineering decision in this whole project?

Strong answer: The session-held-out train/test split, because it is the one decision that makes every other number in the project (the 95.5% accuracy, the model comparison, the claim that Logistic Regression generalizes best) actually trustworthy rather than merely plausible-sounding.

Why asked: a closing question testing whether the candidate can identify what actually matters most, versus listing surface-level features.

Follow-up: "What is the second strongest?" The rule-based classifier as an automatic fallback, since it means an ML problem degrades the app rather than breaking it in production.

Weak answer: naming something cosmetic (dark mode, charts) as the single strongest engineering decision.

20. Small / Mid-Size Company Interview (50)

Practical questions testing whether you can actually build, debug, and explain your own code.

#	Question	Answer
1	Show me the file that decodes an incoming frame.	<code>backend/app/vision/decode.py</code> , function <code>decode_jpeg</code> , uses OpenCV's <code>cv2.imdecode</code> .
2	Where would you add a new REST endpoint?	<code>backend/app/routes.py</code> , define a new <code>@router.get(...)</code> function returning a Pydantic model from <code>schemas.py</code> .
3	How would you add a new field to the WebSocket result message?	Add it to <code>FrameResultMessage</code> in <code>schemas.py</code> , populate it in <code>app/ws.py</code> 's result-building code, and mirror the field in <code>frontend/src/types.ts</code> .
4	How do you run the backend locally?	From <code>backend/</code> , with the venv active: <code>uvicorn app.main:app --reload</code> .
5	How do you run the frontend locally?	From <code>frontend/</code> : <code>npm install</code> then <code>npm run dev</code> .
6	How do you run the backend tests?	From <code>backend/</code> : <code>pytest</code> (or <code>pytest -v</code> for detail).
7	How do you run the frontend tests?	From <code>frontend/</code> : <code>npm run test:run</code> .
8	How would you lint the whole project?	Backend: <code>ruff check .</code> ; frontend: <code>npx oxlint</code> .
9	What command trains the ML model?	<code>python -m ml.train</code> , from <code>backend/</code> .
10	How would you check what git branch you're on?	<code>git status</code> (shows the branch at the top) or <code>git branch</code> .
11	How would you undo an uncommitted change to one file?	<code>git restore <file></code> .
12	How would you see what changed in the last commit?	<code>git show</code> or <code>git log -p -1</code> .
13	How would you find which commit introduced a bug?	<code>git log</code> to review history, or <code>git bisect</code> for a larger history to binary-search automatically.
14	What Python version does this project require?	3.12, pinned in the Dockerfile's base image and matching the local <code>.venv</code> .
15	What Node.js version does the frontend need?	20 or newer (per <code>frontend/README.md</code>); CI uses Node 22.
16	How would you add a new Python dependency?	Install it in the venv, add the exact version to <code>requirements.txt</code> (or <code>requirements-dev.txt</code> for a dev-only tool).

17	How would you add a new npm dependency?	<code>npm install <package></code> , which updates <code>package.json</code> and <code>package-lock.json</code> automatically.
18	What does a 404 on <code>/model</code> after a hard refresh usually mean?	The SPA rewrite is missing or misconfigured; check <code>frontend/vercel.json</code> 's rewrite rule.
19	What HTTP status code does a successful <code>/health</code> call return?	200.
20	What does WebSocket close code 1008 mean in this project?	Policy violation, used specifically when the connecting Origin is not in the allow-list.
21	How would you check if the deployed backend is actually up right now?	Open https://gesture-recognition-app-8ksr.onrender.com/health in a browser or with <code>curl</code> .
22	How would you check what model is currently deployed?	Open <code>/model</code> on the deployed backend and read <code>selected_model</code> in the JSON response.
23	Where do you set the frontend's backend URL for production?	Vercel project settings, environment variable <code>VITE_API_BASE_URL</code> (and <code>VITE_WS_URL</code>).
24	Where do you set which origins the backend accepts?	Render environment variable <code>GESTUREFLOW_ALLOWED_ORIGINS</code> .
25	What is the file that actually starts the production server?	The Dockerfile's <code>CMD</code> , which runs <code>uvicorn app.main:app --host 0.0.0.0 --port \$PORT</code> .
#	Question	Answer
26	How would you change the smoothing window from 6 frames to something else?	Change the <code>window</code> parameter where <code>GestureSmoother</code> is constructed in <code>app/vision/pipeline.py</code> .
27	How would you add a new gesture end to end?	Add it to <code>app/gestures.py</code> ; write a rule for it in <code>classifier_rules.py</code> ; record real samples with <code>record_fixtures.py</code> ; retrain with <code>python -m ml.train</code> .
28	How would you debug why a specific gesture is not being recognized?	Check the Model page's confusion matrix for that class first, then, if needed, temporarily log the raw feature values or scores for that gesture during a local run.
29	What does <code>ruff format --check .</code> do versus <code>ruff format .</code> ?	<code>--check</code> reports what would change without modifying files (used in CI); without it, files are actually reformatted in place.
30	What is the difference between <code>npm run build</code> and <code>npm run dev</code> ?	<code>dev</code> starts a local hot-reloading server; <code>build</code> produces the optimized static <code>dist/</code> folder used in production.

31	How would you check the frontend TypeScript compiles with no errors?	<code>npm run typecheck</code> .
32	Where is the gesture list defined so both the API and the frontend agree?	<code>backend/app/gestures.py</code> ; the frontend always fetches it live from <code>GET /gestures</code> rather than hard-coding it.
33	What would you check if <code>npm install</code> failed?	Node/npm version compatibility, network connectivity, and the exact error message, most commonly a version mismatch or a broken lockfile.
34	What would you check if <code>pip install -r requirements.txt</code> failed?	Python version compatibility (this project needs 3.12), and whether the virtual environment is actually activated.
35	How do you know the venv is active in the terminal?	The prompt shows <code>(.venv)</code> at the start of the line.
36	What is inside the committed <code>gesture_clf.joblib</code> file?	A dictionary: the fitted scikit-learn pipeline, feature names, feature version, labels, dataset type, and the test macro F1.
37	How would you check the ML model's real accuracy yourself?	Open <code>backend/ml/reports/EVALUATION.md</code> , or fetch <code>/model</code> on the deployed backend.
38	What would happen if you deleted <code>ml/models/gesture_clf.joblib</code> and ran the app?	With <code>GESTUREFLOW_CLASSIFIER=ml</code> , the pipeline would log a warning and fall back to the rule-based classifier; the app would keep working, not crash.
39	How is the confidence bar's width calculated on the Live page?	Directly from the result message's <code>confidence</code> value, shown as a percentage-width filled bar in <code>GesturePanel.tsx</code> .
40	What would you check if the camera permission prompt never appears?	Whether the page is served over HTTPS (or localhost); a plain HTTP page in production would silently fail to even offer the prompt.
41	How would you find all the places <code>GESTUREFLOW_CLASSIFIER</code> is used?	<code>grep -r GESTUREFLOW_CLASSIFIER backend/</code> , or search the codebase in an editor.
42	What does the frontend do if <code>VITE_API_BASE_URL</code> is not set in a production build?	Logs a console error and falls back to a localhost default, which will then fail to reach a real backend, visibly, not silently.
43	How would you check which tests exist for the WebSocket endpoint?	Open <code>backend/tests/test_ws.py</code> and read the test function names.
44	What is the quickest way to see all six gestures and their descriptions?	Open the deployed app's Guide page, or read <code>backend/app/gestures.py</code> directly.
45	How would you check the current git remote URL?	<code>git remote -v</code> .

46	How would you clone this repository fresh and get it running?	<code>git clone</code> the repo, then follow <code>README.md</code> 's "How to run locally" section for both halves.
47	What would you check first if CI failed on a pull request?	Open the failed job's log in GitHub Actions and read the actual error, lint, type-check, test, or build, rather than guessing.
48	How would you verify a fix actually resolved a failing test, not just changed the error?	Re-run the specific test (<code>pytest path/to/test_file.py::test_name -v</code>) and confirm it now passes, then run the full suite to check nothing else broke.
49	Can you explain, in your own words, what <code>normalize_landmarks</code> does?	Moves the wrist to the coordinate origin and scales the hand by its own size, so the same gesture produces the same numbers regardless of position or distance from the camera.
50	Can you explain, in your own words, why the split is by session and not random?	Because consecutive frames of one gesture look almost identical, so a random split would let near-duplicate frames leak between train and test and produce an inflated accuracy number.

21. Trick Questions / Cross-Questions (30)

Designed to catch someone who only memorized the project rather than understood it.

#	Question	Answer
1	Why "Logistic Regression" if it is not doing regression?	The name is historical; it models the log-odds of a class with a linear function (a regression-style equation), but the final output is passed through a sigmoid/softmax to become a classification, so calling it "regression" describes its internal mechanics, not its task.
2	Why not deep learning, since it is the modern standard?	With 2,258 samples and 25 already-informative features, a neural network has no real capacity advantage and a genuinely higher overfitting risk; "modern" is not the same as "appropriate for this data size".
3	Is MediaPipe itself machine learning?	Yes, MediaPipe Hands is itself a pretrained neural network for landmark detection; this project does not train it, but it is absolutely a machine-learning component of the pipeline.
4	Is this really "AI"?	It uses machine learning (a trained classifier) and a pretrained computer-vision model, both reasonably called AI/ML by standard usage; it is not a large language model or anything resembling general intelligence, an important distinction to make precisely.
5	Is the model trained on images or landmarks?	Landmarks, specifically 25 engineered features derived from 21 normalized landmark points; no raw image ever reaches the classifier.
6	What exactly is the input to Logistic Regression, precisely?	A 25-element numeric vector (after standard scaling) for one hand pose, not an image, not raw landmarks, the engineered feature vector from <code>features.py</code> .
7	Why isn't 95.5% accuracy necessarily "production accuracy"?	It is measured on one person's second recording session; without evaluating on different people, cameras, and lighting, claiming it as a universal production number would be overstating the evidence.
8	Why can accuracy alone be misleading?	It can be high while a specific class performs poorly, invisible without per-class metrics or a confusion matrix; the pre-fix rule-based classifier's roughly 70% overall accuracy hid a first recall of about 5%.
9	What is the actual difference between validation and test data in this project?	Cross-validation happens only within the training session, used during development to compare candidates; the test session is touched once, after model selection, to report the final number.

10	What would happen with a completely different person's hand?	Likely lower accuracy than 95.5%, since the model has only ever seen one person's hand geometry and gesture style; this is an explicit, disclosed limitation, not a claim of universal accuracy.
11	Why could the model fail under rotation if the features are supposed to be rotation-tolerant?	"Tolerant" is not "invariant"; the features reduce but do not eliminate rotation sensitivity, and normalization deliberately does not rotate the hand to a canonical pose, since orientation itself matters for gestures like thumbs-up.
12	Why WebSocket instead of repeatedly calling REST APIs?	A WebSocket is one persistent connection for a continuous, bidirectional stream; repeated REST calls would mean a new connection and full HTTP headers for every single frame, roughly ten times a second, with no natural session state.
13	What happens if the backend crashes mid-connection?	The WebSocket connection drops; the frontend's <code>useGestureSocket</code> detects the close and begins reconnecting with exponential backoff, showing a "reconnecting" state rather than a silent failure.
14	What happens if the model file is missing entirely?	<code>MLClassifier</code> raises <code>ModelUnavailable</code> on construction; the pipeline logs a warning and serves the rule-based classifier instead, automatically.
15	Why Docker if Render can run Python natively?	MediaPipe and OpenCV need specific system libraries not guaranteed on a generic Python runtime; Docker makes those dependencies explicit and identical locally and in production.

#	Question	Answer
16	Why separate Vercel and Render instead of one host?	Vercel is built for static frontends; Render runs long-lived Docker containers, which the WebSocket backend needs; neither platform alone does both jobs well.
17	What happens when Render goes to sleep?	The container stops after about 15 minutes idle; the next request triggers a fresh start, up to about 30 seconds, shown in the UI as a "waking up" message.
18	Is the webcam video stored on the server?	No, never; only downscaled single frames pass through in memory during processing, and only 21 normalized landmark points, never pixels, are ever written to disk, in the dataset-recording script.
19	What are the privacy implications of this app?	Low but non-zero: hand landmarks are biometric-adjacent data; no video/images are stored anywhere, and the deployed app itself persists nothing server-side about any user's session.

20	If landmarks are not images, why call it computer vision at all?	The landmark detection stage (MediaPipe on the raw decoded frame) genuinely is computer vision; what happens after, working on the 21 points, is geometry and classification, both are legitimately part of the same CV/ML pipeline.
21	Doesn't a rule-based classifier mean it's not really "using AI"?	Correct, precisely, the rule-based engine is not AI/ML at all, it is hand-written logic; it exists specifically as a non-ML baseline and fallback, and the ML model is the actual machine-learning component, a distinction worth stating clearly, not blurring.
22	If the ML model is better, why keep the rule-based one in production at all?	As the automatic fallback if the ML model is ever unavailable, and as the honest baseline that proves the ML model's improvement is real and measured, not assumed.
23	Is 25 features "feature engineering" or just "using MediaPipe's output directly"?	Genuine feature engineering: none of the 25 values are raw MediaPipe output, they are all computed (ratios, angles, distances) from the 21 points, specifically designed to be rotation- and scale-tolerant.
24	Doesn't 2,258 samples sound too small for real machine learning?	It is small relative to, say, ImageNet-scale datasets, but appropriately sized for 25 well-engineered features and 6 classes; the honest evaluation results and the documented limitation about dataset diversity are exactly how this size is being represented, not oversold.
25	If Logistic Regression is "simple", is this really a serious ML project?	Model simplicity is not the measure of seriousness; the dataset collection, the leakage-safe evaluation methodology, the principled multi-model comparison, and the honest interpretation of results are the actual engineering substance, and a simple model that legitimately wins that comparison is a stronger result than reflexively picking a complex one.
26	Doesn't smoothing over 6 frames just hide a bad classifier?	No, it addresses a different problem, single-frame noise/flicker, not classifier accuracy; a genuinely wrong classifier would still be consistently wrong across the smoothing window, smoothing only stabilizes an already-reasonable stream of predictions.
27	If the confidence score is not a true probability for the rule-based engine, why show it at all?	It is clearly labelled internally as a heuristic match quality score, still useful signal (higher genuinely means a cleaner match to the gesture's template), just not a statistically calibrated probability the way the ML engine's is.
28	Doesn't "provisional: false" in the report just mean you typed false?	No, it is set automatically by <code>train.py</code> based on whether <code>--synthetic</code> was passed; it reflects that this run used the real recorded dataset, not a manually asserted claim.

29	If the app works without an internet connection locally, why does it need HTTPS in production?	Because <code>localhost</code> is a browser-recognized exception to the secure-context rule specifically for local development; a public production URL gets no such exception and must be genuinely HTTPS.
30	Doesn't committing a trained model file contradict "don't commit generated files"?	It is a deliberate, justified exception, small (about 4KB), it is the real production artifact, not an experiment, and committing it means a fresh deploy has a working ML classifier immediately; the raw dataset and any other experimental models remain git-ignored.

22. Debugging / Real-World Troubleshooting Scenarios (30)

Each with a step-by-step diagnostic approach grounded in this project's actual tools and structure.

1. Backend shows offline in the frontend badge.

Step 1: open `<backend-url>/health` directly in a browser, if it does not load, the backend itself is down, check Render's dashboard/logs. Step 2: if `/health` works but the badge still shows offline, check the browser console for a CORS error, meaning `VITE_API_BASE_URL` or `GESTUREFLOW_ALLOWED_ORIGINS` is mismatched. Step 3: confirm `VITE_API_BASE_URL` was actually set at build time on Vercel (a change requires a redeploy to take effect, it is baked in at build time).

2. Frontend loads but the WebSocket never connects.

Step 1: check the browser console/network tab for the WebSocket handshake's status; a close immediately after opening with no data usually means the Origin was rejected (code 1008). Step 2: verify `VITE_WS_URL` uses `wss://` (not `ws://`) and includes the `/ws` path. Step 3: verify `GESTUREFLOW_ALLOWED_ORIGINS` on the backend exactly matches the frontend's real origin, no trailing slash, correct protocol.

3. Camera permission is denied.

Step 1: check the site's permission setting in the browser (the padlock/site-info icon), camera may be blocked at the browser level, not just by the in-page prompt. Step 2: confirm the page is served over HTTPS or localhost; an insecure context never even offers the permission prompt. Step 3: in this project, use the visible "denied" state's own retry action in `CameraView` after fixing the browser-level permission.

4. `/model` works but gestures are never detected.

Step 1: confirm a hand is actually visible and well-lit in frame; check `hand_present` in the result messages (browser devtools network tab, WS frames) to see if MediaPipe is finding a hand at all. Step 2: if `hand_present` is true but `gesture` is always null, sensitivity may be set too strict, try raising it in Settings. Step 3: check the active engine (rule vs ML) in Settings, and try switching, to isolate whether the issue is engine-specific.

5. Gesture recognition feels slow.

Step 1: check the Session Stats panel's server latency (`inference_ms`); if it is high, the backend (possibly a cold-started free-tier instance) is the bottleneck. Step 2: check throughput/dropped frames; a high drop count suggests lowering the target fps in Settings. Step 3: check your own network conditions, since round-trip time to the deployed backend also adds to perceived latency.

6. Frames are being dropped.

Step 1: this is visible directly in `frames_dropped` in the Session Stats panel, confirm it is actually happening and how often. Step 2: lower the target fps in Settings so the backend is not asked to keep up with more frames than it can process. Step 3: if it persists even at a low fps, check server latency, a slow backend (cold start, or genuine overload) is the likely root cause, not the frontend.

7. Render deployment fails to build or start.

Step 1: read the actual Render build/deploy logs first, not guess, the root Dockerfile build is deterministic and the logs name the failing step. Step 2: common causes here: a missing system library for OpenCV (the Dockerfile installs `libgl1/libglu1` for this reason), or the service not binding to the `$PORT` env var Render injects. Step 3: confirm the start command matches the Dockerfile's `CMD`, which runs `uvicorn` against `$PORT`, and that health checks hit `/health`.

8. Vercel frontend cannot connect to the Render backend.

Step 1: confirm the backend is actually reachable by visiting `/health` directly. Step 2: check that `VITE_API_BASE_URL` and `VITE_WS_URL` were set as Vercel project environment variables and that the project was redeployed after setting them (Vite bakes them in at build time, not read at runtime). Step 3: check for a CORS/Origin rejection in the browser console as in scenario 2.

9. A CORS or WebSocket-origin error appears in the console.

Step 1: read the exact error, a REST CORS error and a WS close code 1008 are different problems with the same root cause category. Step 2: for REST, check FastAPI's CORS middleware origins list, sourced from `GESTUREFLOW_ALLOWED_ORIGINS`. Step 3: for WS, check the same env var (the `/ws` handler validates `Origin` against it), and confirm there is no trailing slash or protocol mismatch between the configured origin and the real one.

10. The model file appears to be missing in production.

Step 1: check the `/model` endpoint and the WS `ready` message's `ml_available` field, if false, the backend fell back to the rule engine because it could not load the `.joblib`. Step 2: confirm the Dockerfile actually `COPYs backend/ml/models` into the image (it does), and that the model file is committed to git (it is, un-ignored specifically for deployment) rather than only existing locally. Step 3: check `GESTUREFLOW_MODEL_PATH` if it was overridden to a path that does not exist in the container.

11. Docker works locally but the deployed container fails.

Step 1: compare environment variables between local and deployed, a local run often has different defaults than the platform's actual configured env vars. Step 2: check whether the platform is passing a `$PORT` the app must bind to (Render does), while local runs might hardcode a port. Step 3: check the platform's own build logs for a step that behaves differently in its build environment (e.g., network access, base image layer caching) than a local Docker build.

12. A README image does not display on GitHub.

Step 1: confirm the file actually exists at the exact relative path used in the Markdown image link, and that capitalization matches (GitHub's filesystem is case-sensitive). Step 2: confirm the image was actually committed and pushed, not just present locally/untracked. Step 3: open the raw GitHub URL for the image directly to isolate whether it is a path problem or a rendering/caching problem.

13. git push fails.

Step 1: read the exact error, "non-fast-forward"/"fetch first" means the remote has commits you do not, run `git pull --rebase` first. Step 2: an authentication error means the credential/token needs refreshing, not a code problem. Step 3: a large-file or binary-related rejection means checking `.gitattributes`/repo size limits, relevant here since the PDF itself is a binary file being added to the repo.

14. CI passes locally but fails in GitHub Actions.

Step 1: read the failing job's log in the Actions tab, the backend and frontend are separate jobs, identify which one failed and at which step (lint, typecheck, test, or build). Step 2: check for an environment difference, CI installs dependencies fresh from `requirements.txt/package-lock.json`, so a locally-installed-but-not-declared dependency would fail only in CI. Step 3: reproduce locally with a clean checkout/fresh virtual environment to rule out local machine drift before assuming CI is at fault.

15. `ruff format --check` fails in CI.

Step 1: this means code was committed without being formatted, run `ruff format` locally to auto-fix. Step 2: run `ruff check` too, since lint and format are separate steps in the CI job. Step 3: commit the formatted result and re-push; this is a style-only failure, not a logic bug.

16. The WebSocket connects (a `ready` message arrives) but no gesture results ever arrive after that.

Step 1: confirm the frontend is actually sending frame messages after `ready`, check the network/WS frames tab for outgoing traffic. Step 2: check the browser console for a frame-capture error (canvas/getUserMedia issue) that would silently stop frames being sent. Step 3: check backend logs for an exception while decoding an incoming frame, e.g., unsupported image data, which would stop that connection's result stream without necessarily closing the socket.

17. Confidence is always very low even for a clear gesture.

Step 1: check which classifier engine is active (Settings), the rule engine is comparatively weaker for gestures like OK sign. Step 2: check lighting and hand framing, MediaPipe landmark quality directly drives feature quality for both engines. Step 3: compare behavior against the known per-class metrics in `comparison.json`, a class with genuinely lower recall (e.g., `ok_sign` on the rule engine) is expected to be weaker, not necessarily a bug.

18. The dashboard charts are empty.

Step 1: the dashboard reads from the current in-browser session's recorded history (`useSessionRecorder`), confirm you actually ran a Live session first, in the same browser tab/session. Step 2: check the browser console for a rendering error in the Recharts components. Step 3: confirm gestures were actually being recognized during that session (check the Live page's own gesture card), an empty session naturally produces an empty dashboard.

19. The Model page shows numbers that seem out of date.

Step 1: the Model page reads `GET /model`, which serves `backend/ml/reports/comparison.json` from whatever is actually deployed, confirm the deployed backend is running the latest committed code/model. Step 2: if you retrained locally, confirm `python -m ml.train` actually rewrote `comparison.json` and it was committed and pushed. Step 3: hard-refresh the frontend, in case a stale cached response is being shown.

20. `npm run build` fails on the frontend.

Step 1: read the TypeScript/Vite error output, it usually names the exact file and line. Step 2: run `npm run typecheck` separately to isolate a type error from a bundler/config error. Step 3: confirm `node_modules` is up to date with `package-lock.json` (a fresh `npm ci` rules out local dependency drift).

21. `pytest` cannot find the app module.

Step 1: confirm `pytest` is being run from the `backend/` directory, imports like `from app...` are relative to that root. Step 2: confirm the project's virtual environment is active, so installed dependencies (FastAPI, MediaPipe, etc.) are on the path. Step 3: check for a missing `__init__.py` or an accidental rename in the package structure if it is a

new failure rather than an environment issue.

22. MediaPipe import fails on a freshly set up machine.

Step 1: check the Python version, this project's MediaPipe pin (`mediapipe==0.10.21`) is verified against Python 3.12 on Windows, other versions may not have a compatible wheel. Step 2: reinstall from `backend/requirements.txt` inside a clean virtual environment rather than trying to hand-fix versions. Step 3: check for a missing system dependency if this happens inside Docker/Linux (the Dockerfile installs `libgl1/libglib2.0-0` specifically because OpenCV needs them).

23. The Docker HEALTHCHECK reports unhealthy.

Step 1: exec into the running container (or check platform logs) and confirm the app process actually started without crashing. Step 2: manually curl `/health` from inside the container's network to see if it responds at all. Step 3: check that the HEALTHCHECK's target port matches the port uvicorn is actually bound to (it should track `$PORT`).

24. The local virtual environment will not activate, or the wrong Python runs.

Step 1: confirm you are pointing at the project's own `.venv`, not a system Python or a different project's environment. Step 2: on Windows, confirm the activation script's execution policy is not blocking it (PowerShell) and that the path separators/shell match the activation script used (`Scripts\activate` vs `bin/activate`). Step 3: if the venv is corrupted, delete and recreate it, then reinstall from `requirements.txt`.

25. A local port is already in use when starting the backend or frontend.

Step 1: identify the process holding the port (e.g., `netstat`/Task Manager on Windows) — it is often a previous dev server that did not shut down cleanly. Step 2: stop that process, or run the new one on a different port temporarily. Step 3: if it recurs often, check for a script that spawns a duplicate process instead of reusing one.

26. Moving the sensitivity slider in Settings seems to do nothing.

Step 1: confirm you are actually in a Live session with the WebSocket connected, the slider pushes a `{"type": "config", "min_confidence": x}` message over the live socket, it has no effect if nothing is connected. Step 2: check the browser console/WS frames to confirm that config message is actually being sent. Step 3: remember it changes a confidence threshold, not detection itself, so its effect is subtle for very confident gestures and more visible for borderline ones.

27. The engine toggle (rule vs ML) appears disabled or has no effect.

Step 1: check the WS `ready` message's `ml_available` field, if false (model failed to load on the backend), switching to ML has nothing to switch to. Step 2: confirm the toggle actually sends `{"type": "config", "classifier": "ml"}` (or `"rule"`) over the live socket. Step 3: check that a session is actually connected, the same precondition as scenario 26.

28. The same gesture is consistently recognized as the wrong class.

Step 1: check which engine is active, this is exactly the historical fist-vs-thumbs_up failure mode the rule engine had before it was fixed (see the trick-question section), so first rule out a known/expected weak spot. Step 2: check hand framing/lighting, poor landmark quality degrades both engines similarly. Step 3: if it is reproducible and not one of the known weak spots, it is a legitimate finding worth writing up rather than dismissing.

29. The app behaves differently across browsers (e.g., works in Chrome, not in Safari).

Step 1: check for a browser-specific `getUserMedia`/camera-permission quirk first, camera APIs are the most browser-sensitive part of this app. Step 2: check the console for a WebSocket or canvas API difference. Step 3: **Not determined from the repository**: this project has no documented cross-browser test matrix, so a genuine compatibility bug would need to be diagnosed fresh, not looked up.

30. The production frontend takes an unusually long time to respond on first use after a while.

Step 1: check server latency in the Session Stats panel, a spike on the very first request is consistent with a free-tier Render instance cold-starting after idling. Step 2: confirm it resolves after the instance is warm (subsequent requests return to normal latency). Step 3: distinguish this from a real outage by checking `/health` directly, if it eventually returns 200, it was a cold start, not a failure.

23. Rapid-Fire Round (100 Questions)

Short, 1-2 sentence answers for last-minute revision. Grouped by topic.

General (1-15)

#	Question	Answer
1	What is GestureFlow?	A real-time hand-gesture recognition web app: webcam in, recognized gesture out, over a live browser connection.
2	What problem does it solve?	Demonstrates a full computer-vision + ML pipeline shipped as a real, deployed, working product, not just a notebook.
3	How many gestures does it recognize?	Six: open_palm, fist, thumbs_up, victory, pointing_up, ok_sign.
4	What language is the backend in?	Python, using FastAPI.
5	What language is the frontend in?	TypeScript, using React and Vite.
6	What connects the frontend and backend live?	A WebSocket connection at /ws .
7	What library detects the hand landmarks?	MediaPipe Hands.
8	How many landmarks per hand?	21.
9	Is the CV done client-side or server-side?	Server-side; the browser only captures and streams frames.
10	What are the two gesture-recognition engines?	A hand-written rule-based classifier and a trained ML classifier (logistic regression).
11	Which engine is the production default?	The ML classifier (logistic regression).
12	Where is the frontend deployed?	Vercel.
13	Where is the backend deployed?	Render.
14	Is the source code public?	Yes, on GitHub.
15	Is this a solo project?	Yes.

ML / CV (16-30)

#	Question	Answer
---	----------	--------

16	Which model was selected as best?	Logistic regression.
17	What was its test F1 score?	0.9541 (95.41%).
18	What was its test accuracy?	0.9545 (95.45%).
19	How many samples were collected?	2,258.
20	How many recording sessions?	Two, s1 and s2.
21	How was train/test split done?	By session (group split), not a random row-wise split.
22	Why split by session, not randomly?	To prevent data leakage, since frames from the same session are highly correlated.
23	How many engineered features?	25.
24	What do the 25 features describe?	Geometric properties of the hand derived from the 21 landmarks, e.g., finger extension, distances, angles.
25	What models were compared besides logistic regression?	SVM (RBF kernel), random forest, a most-frequent-class dummy baseline, and the rule-based classifier.
26	What was the rule-based classifier's test F1?	0.8327 (accuracy 0.8635), after the thumb-protrusion fix.
27	What library trained the ML model?	scikit-learn.
28	How is the trained model stored?	As a <code>.joblib</code> file, committed to the repository.
29	What guards against a train/inference feature mismatch?	A <code>FEATURE_VERSION</code> constant checked at load/inference time.
30	What was the known rule-classifier bug that got fixed?	It frequently misread a closed fist as thumbs_up; fixed by requiring the thumb tip to protrude past the curled fingertips.

Numbers & Metrics (31-45)

#	Question	Answer
31	SVM (RBF) test accuracy / F1?	90.81% / 0.9081.
32	Random forest test accuracy / F1?	84.25% / 0.8306.
33	Most-frequent dummy baseline accuracy / F1?	15.57% / 0.0449.
34	Logistic regression precision / recall?	95.49% / 95.38%.

35	How many samples in session s1?	1,115 (used as the training split).
36	How many samples in session s2?	1,143 (used as the test split).
37	How many backend tests?	93.
38	How many frontend tests?	93.
39	Total tests across the project?	186.
40	What was the thumb-protrusion threshold in the bug fix?	Greater than 0.55 (normalized).
41	How many gesture classes?	6.
42	How many MediaPipe hand landmarks?	21.
43	How many engineered ML features?	25.
44	What is FEATURE_VERSION currently?	1.
45	What default frame rate does the frontend target for streaming?	Roughly 10 fps (throttled capture), adjustable in Settings.

Backend / API (46-60)

#	Question	Answer
46	What web framework runs the backend?	FastAPI, served by Uvicorn.
47	What does <code>GET /health</code> return?	A simple status payload used for health checks (Docker HEALTHCHECK and platform probes).
48	What does <code>GET /gestures</code> return?	The list of supported gesture ids/names/descriptions.
49	What does <code>GET /model</code> return?	The contents of <code>comparison.json</code> , the ML evaluation report.
50	What protocol does <code>/ws</code> use?	WebSocket.
51	What is the first message sent over <code>/ws</code> ?	A <code>ready</code> message, including which classifier is active and whether ML is available.
52	How does a client change settings mid-session?	By sending a <code>{"type": "config", ...}</code> message over the open socket (e.g., <code>min_confidence</code> , <code>classifier</code>).
53	What closes a WebSocket connection from a disallowed origin?	Close code 1008, enforced by an Origin check against <code>GESTUREFLOW_ALLOWED_ORIGINS</code> .

54	What env var lists allowed frontend origins?	<code>GESTUREFLOW_ALLOWED_ORIGINS</code> .
55	What env var can disable the origin check entirely?	<code>GESTUREFLOW_WS_ALLOW_ANY_ORIGIN</code> .
56	What env var selects the active classifier?	<code>GESTUREFLOW_CLASSIFIER</code> .
57	What happens if the ML model fails to load?	The pipeline falls back to the rule-based classifier automatically.
58	What image format does the frontend send per frame?	A downscaled JPEG (around 320px wide).
59	What CV library decodes/processes frames server-side?	OpenCV, alongside MediaPipe.
60	Where does interactive API documentation live?	FastAPI's auto-generated <code>/docs</code> (Swagger UI) and <code>/redoc</code> .

Frontend (61-75)

#	Question	Answer
61	What frontend framework is used?	React.
62	What build tool bundles the frontend?	Vite.
63	What styling approach is used?	Tailwind CSS.
64	What handles client-side routing?	React Router.
65	What charting library powers the dashboard?	Recharts.
66	What browser API captures the webcam?	<code>getUserMedia</code> .
67	What draws the hand skeleton overlay?	A canvas overlay drawn from the landmarks returned over the WebSocket.
68	Where are user settings (sensitivity, engine, fps) stored?	In browser <code>localStorage</code> , via a settings context.
69	Where is session history (for the dashboard) stored?	In-memory for the current browser session, via a session-recorder context.
70	What does "Live" mean in the UI status?	The WebSocket is connected and frames are actively being streamed and processed.
71	What does "Idle" mean in the UI status?	No active gesture is currently being recognized in the stream (connection can still be live).

72	What does "Server latency" show?	The backend's per-frame inference time, reported back over the WebSocket.
73	What does "Throughput" show?	The rate of frames actually being processed per second.
74	What does "Frames dropped" show?	Frames the client chose not to send (e.g., while a previous frame's response was still in flight).
75	What does "Detections" show?	A running count of recognized (non-idle) gesture results in the current session.

Deployment / DevOps (76-90)

#	Question	Answer
76	What is the live frontend URL?	https://gesture-recognition-app-tawny.vercel.app/
77	What is the live backend URL?	https://gesture-recognition-app-8ksr.onrender.com
78	What is the backend's health check path?	<code>/health</code>
79	What packages the backend for deployment?	A root <code>Dockerfile</code> .
80	Does the container run as root?	No, it runs as a dedicated non-root <code>appuser</code> .
81	What does the Dockerfile's HEALTHCHECK call?	The backend's <code>/health</code> endpoint.
82	What system libraries does the Dockerfile install for OpenCV?	<code>libgl1</code> and <code>libgl1-mesa-glx</code> .
83	Where does the frontend read its API/WS URLs from at build time?	<code>VITE_API_BASE_URL</code> and <code>VITE_WS_URL</code> environment variables.
84	What config file tells Vercel how to build/serve the SPA?	<code>frontend/vercel.json</code> .
85	Is there a hosted database?	No, the project is explicitly stateless / has no database.
86	Where is the full deployment runbook documented?	<code>docs/DEPLOYMENT.md</code> .
87	What is committed to git for deployment that is normally git-ignored during development?	The trained <code>gesture_clf.joblib</code> model file (datasets themselves stay local/ignored).
88	What CI provider runs automated checks?	GitHub Actions.
89	What free-tier behavior can affect first-request latency on Render?	Cold starts after a period of inactivity.

90	Is HTTPS required for the camera to work in production?	Yes, browsers require a secure context (HTTPS, or localhost as the sole exception) for <code>getUserMedia</code> .
----	---	--

Testing & Misc (91-100)

#	Question	Answer
91	What test framework runs backend tests?	pytest.
92	What test framework runs frontend tests?	Vitest, with React Testing Library.
93	Do frontend tests use a real camera?	No, browser-only APIs (camera, canvas, WebSocket) are mocked in unit tests.
94	What verifies backend code style/lint?	ruff (both <code>ruff check</code> and <code>ruff format --check</code>).
95	What verifies frontend lint and types?	oxlint and the TypeScript compiler (<code>npm run typecheck</code>).
96	What test fixtures drive the CV/rule-classifier unit tests?	Synthetically constructed hand landmark fixtures, not live camera input.
97	Is there a dedicated load-testing or concurrency test suite?	Not determined from the repository.
98	Is there a code license?	Yes, MIT License.
99	Where is the architecture/pipeline explained inside the app itself?	The About page.
100	What is the working brand name of the project?	GestureFlow (the repository itself is named <code>gesture-recognition-app</code>).

24. Important Numbers to Memorize

One-page numbers cheat sheet

Metric	Value
Gesture classes	6 (open_palm, fist, thumbs_up, victory, pointing_up, ok_sign)
MediaPipe landmarks per hand	21
Engineered ML features	25 (FEATURE_VERSION = 1)
Total samples collected	2,258
Sessions	2 (s1 = 1,115 train, s2 = 1,143 test)
Split method	Group-by-session (GroupShuffleSplit), not random row split
Selected model	Logistic Regression
Logreg test accuracy	95.45%
Logreg test F1 (weighted)	95.41%
Logreg precision / recall	95.49% / 95.38%
SVM (RBF) accuracy / F1	90.81% / 90.81%
Random forest accuracy / F1	84.25% / 83.06%
Rule-based accuracy / F1	86.35% / 83.27% (after fix)
Most-frequent dummy accuracy / F1	15.57% / 4.49%
Thumb-protrusion fix threshold	> 0.55 (normalized)
Backend tests	93
Frontend tests	93
Total tests	186
Frontend hosting	Vercel
Backend hosting	Render (Docker)

Live frontend URL	gesture-recognition-app-tawny.vercel.app
Live backend URL	gesture-recognition-app-8ksr.onrender.com
Health check path	/health
Model report path	/model

If you remember nothing else

Logistic regression, 95.4% F1, 2,258 samples, 2 sessions split by session (not randomly), 6 gestures, 25 features from 21 MediaPipe landmarks, server-side CV over a WebSocket, deployed on Vercel + Render.

25. Final One-Page Cheat Sheet

PROJECT	GestureFlow — real-time hand gesture recognition web app
OBJECTIVE	Recognize 6 hand gestures live from a webcam and show them in the browser in real time
INPUT	Live webcam video, captured in-browser, throttled and downscaled to JPEG frames
PROCESS	Frame → WebSocket → backend decodes → MediaPipe finds 21 landmarks → 25 features engineered → Classifier (rule or ML) predicts gesture + confidence → smoothing
OUTPUT	Gesture label, confidence score, and landmark positions streamed back to the browser
FRONTEND	React + Vite + TypeScript + Tailwind + React Router + Recharts, hosted on Vercel
BACKEND	FastAPI + Uvicorn + OpenCV + MediaPipe, hosted on Render via Docker
CV	MediaPipe Hands — 21 landmarks per hand, server-side
ML	scikit-learn — logistic regression selected over SVM (RBF), random forest, and a dummy baseline
MODEL	Logistic regression on 25 engineered geometric features, saved as a committed <code>.joblib</code> file
DATASET	153 self-collected samples across 2 sessions (s1/s2), split by session to avoid leakage
BEST RESULT	95.45% test accuracy / 0.9541 test F1 (logistic regression, on unseen session s2)
API	REST: <code>/health</code> , <code>/gestures</code> , <code>/model</code> ; WebSocket: <code>/ws</code>
WEBSOCKET	Streams frames up, streams landmarks + gesture + confidence + stats down; origin-checked
DEPLOYMENT	Frontend on Vercel, backend on Render (Docker, non-root, HEALTHCHECK on <code>/health</code>)
TESTING	93 tests total (93 backend pytest + 93 frontend Vitest), plus CI lint/typecheck/build
LIMITATIONS	Small self-collected dataset, single-hand only, unknown behavior in unusual lighting/gloves, no load testing
FUTURE SCOPE	Larger/more diverse dataset, more gesture classes, two-hand support, gesture-driven demo interaction

10 things I MUST remember before viva

1. The CV runs server-side (backend), not in the browser — the browser only captures and streams frames.
2. The pipeline is: frame → MediaPipe (21 landmarks) → 25 engineered features → classifier → smoothing → result.
3. Two classifiers exist — rule-based (hand-written thresholds) and ML (trained logistic regression) — and can be switched live.
4. The dataset was split by session, not randomly, specifically to prevent data leakage.

5. Logistic regression won not because it is fancy, but because it generalized best to the unseen test session — simpler models can beat more complex ones on small data.
6. The rule-based classifier had a real, fixed bug: confusing fist with thumbs_up, solved with a thumb-protrusion threshold.
7. Everything is honestly labeled: metrics come from a committed `comparison.json`, not memory or guesswork.
8. The app is stateless — no database — session data lives only in the browser for that session.
9. Deployment is two separate platforms (Vercel + Render) connected by explicit environment variables and an origin allow-list.
10. Anything not actually in the repository is called out as "Not determined from the repository," not invented.